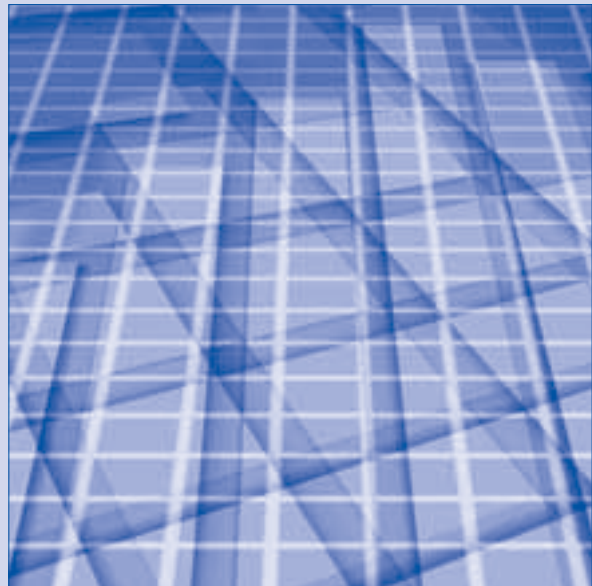




M359 Block 2

UNDERGRADUATE COMPUTING

Relational databases: theory and practice



Introducing relational
theory

Block
2

This publication forms part of an Open University course M359 *Relational databases: theory and practice*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)870 333 4340, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure: tel. +44 (0)1908 858785; fax +44 (0)1908 858787; email ouwenq@open.ac.uk

Sybase, iAnywhere and SQL Anywhere are trademarks of Sybase, Inc.; Java is a trademark of Sun Microsystems, Inc. Other product and company names may appear in the M359 course material. Rather than use a trademark symbol with every occurrence of a trademarked name, we use the names only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The Open University
Walton Hall, Milton Keynes
MK7 6AA

First published 2006, second edition 2009.

Copyright © 2006, 2009 The Open University

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>.

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by S&P Enterprises (rfod) Limited, Glos.

Printed and bound in the United Kingdom by Martins the Printers, Berwick-upon-Tweed.

ISBN 978 0 7492 5488 9

2.1

CONTENTS

1	Introduction	5
1.1	Looking back: The Entity–Relationship model	5
1.2	Looking forward: Relational representations	6
1.3	Outline of this block	7
2	The structure of relational representations	8
2.1	The structure of relations	8
2.2	Domains	16
2.3	Developing a relational representation	19
2.4	Candidate keys, primary keys and alternate keys	20
2.5	Representing relationships using foreign keys	23
2.6	Representing relationships by relations	31
2.7	Summary	36
3	Manipulating relations	38
3.1	The select and project operators	39
3.2	The join and rename operators	43
3.3	The divide operator	49
3.4	Set operators union , intersection and difference	51
3.5	The times operator	54
3.6	Summary	55
4	Constraints	56
4.1	Candidate and foreign key constraints	56
4.2	Tuple constraints	58
4.3	General constraints	59
4.4	Summary	62
5	Normal forms	64
5.1	Motivation	64
5.2	Single-valued facts and functional dependencies	66
5.3	First and second normal forms (1NF and 2NF)	74
5.4	Third normal form (3NF)	77
5.5	Boyce–Codd normal form (BCNF)	80
5.6	Summary	82
	Block summary	84
	Solutions to Exercises	85
	Index	108

M359 COURSE TEAM

This course was produced by the following team (affiliated to The Open University, unless otherwise stated):

Course team

Kevin Waugh Course Team Chair and Author
Ian Cooke Author
Mike Newton Author
Judith Segal Author
Steven Self Author
Alistair Willis Author
Kay Bromley Academic Editor
Ralph Greenwell Course Manager and Accessibility Consultant

External assessor

Barry Lowden University of Essex

Critical readers

Sue Barrass
Peter Blachford
Terry Burbidge
Pauline Butcher
Pauline Curtis
Hugh Darwen
Ivan Dunn
Gillian Mills
Ron Rogerson

LTS Media team

Andrew Seddon Media Project Manager
Steve Rycroft Editor
Andrew Whitehead Designer and Graphic Artist
Phillip Howe Composer
Kamy Yazdanjoo Software Developer
Sue Stavert Technical Testing Team

Thanks are due to the Desktop Publishing Unit of the Faculty of Mathematics and Computing.

1 Introduction

1.1 Looking back: The Entity–Relationship model

In *Block 1*, you were introduced to the idea of a conceptual data model. A conceptual data model aims to capture the essential structure of the data that you are interested in. It may be used as a communication tool, so that the database developer can articulate clearly their understanding of the nature of the data and validate this understanding with the customer.

Block 1 introduced a particular type of conceptual data model: the Entity–Relationship (E–R) model. This model represents:

- ▶ The entity types and their properties. Examples of entity types in the Hospital conceptual model that you met in *Block 1* are **Ward** and **Nurse**, having properties (attributes) **WardNo**, **WardName**, **NumberOfBeds**, and **StaffNo**, **NurseName**, respectively. In the E–R model, the entity types and associated attributes can be written as follows:

Ward(WardNo, WardName, NumberOfBeds)

Nurse(StaffNo, NurseName)

WardNo and **StaffNo** are the respective identifiers.

- ▶ Relationships between entity types including degree and participation conditions. For example, in the Hospital conceptual model, there is a relationship between **Ward** and **Nurse** called **StaffedBy**, which represents the facts that each ward is staffed by one or more nurses, and each nurse staffs exactly one ward – see Figure 1.1.



Figure 1.1 Fragment of an E–R diagram, illustrating how relationships are represented

- ▶ Constraints on the values which might be taken by the properties so that they accurately reflect the constraints of real life. For example: the staff number of a nurse may only be allowed to take values between 000 and 999 according to the requirements specification; the number of beds can never be negative.

The model may also be accompanied by a statement of assumptions – which should be clarified by the customer – and a statement of limitations which delineates the context in which the model is valid.

1.2 Looking forward: Relational representations

This block is concerned with the relational representation of data. You may think of this block as following on directly from *Block 1*: you have structured the data as an Entity–Relationship (E–R) model and now you want to transform that model into a representation which leads fairly directly to an implementable database design. In terms of the three-schema architecture which you met in *Block 1*, the implementable database design forms the logical schema.

In a relational representation, data is essentially structured by means of a set of tables, with relationships among tables being represented by means of shared values of certain data items – in much the same way as all your bank statements and related documents are associated by means of your unique account number, as illustrated in Figure 1.2. A table is, in fact, a concrete representation of an abstract concept called a *relation*, which you met briefly in *Block 1* and which we will discuss further in Section 2.

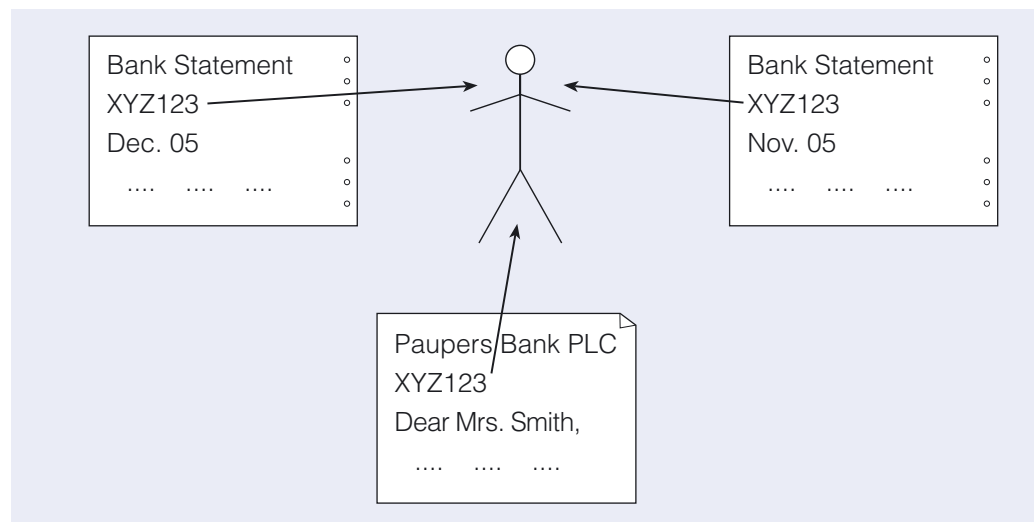


Figure 1.2 Tying together related objects

We should note here that the *implementation* of a relational representation raises some issues which this block does not address. For example, in the relational theory discussed in this block, we assume that every property of interest of every occurrence of an entity type under consideration has a known value. In the real world, this may not be true. An occurrence may not have a particular property or you may not know the value of a particular property. For example, if the entity type were of a person and one of the properties you were interested in was email address, not everyone might have an email address. Or you might know that Joe Bloggs is on email but not know his address at the time you enter his details into the database. In *Blocks 3* and *4*, you will see how database developers commonly deal with such missing information. A further practical problem is that data definition and data manipulation languages (such as the various dialects of SQL) differ in how they implement the theory and how much of the theory they implement. This is discussed further in *Block 3*. Finally, a relational representation which is consistent with the theory might not satisfy the performance needs of a real database – this issue will be considered in *Block 4*.

1.3 Outline of this block

The structure of this block is as follows. In Section 2, we shall consider how to represent the structure of the data in a relational representation, taking as our starting point the E–R conceptual models that you met in *Block 1*. You may find that representing the entity types and their attributes is rather easy and intuitive – you simply use tables as we said above. Rather more difficult is the representation of relationships. You may recall from *Block 1* and Figure 1.1 that relationships are represented in the E–R diagram in our notation by lines, with or without crows' feet to represent degrees, with empty or filled circles to represent participation conditions. In Subsections 2.5 and 2.6, we discuss how such relationships might be represented in tabular form – that is, using relations – and how the form of representation depends (as you might expect) on the degree and participation conditions of the relationship.

We should stress here that you don't have to start with an E–R model in order to construct a relational representation, and neither does an E–R model have to lead to a relational representation: they are independent concepts giving you different views of the data. Either might, in certain circumstances, be considered as an end in itself. An E–R model, as we have already said, might be used as a communication tool. As for relational representations, there is a hope that, in the future, database developers may use a system which implements a relational representation directly, that is, without needing to first translate it into a database definition language such as SQL. However, going from E–R model to relational representation to implementation (as in *Blocks 1, 2 and 3* of this course) is a natural progression.

In Section 3, we explore how relations can be manipulated and introduce a set of operators for this purpose. In Section 4, we consider how constraints on the data might be represented as relational algebra expressions, that is, expressions involving relations and operators. In Section 5, we introduce the idea of normal forms, in particular, first, second, third and Boyce–Codd normal forms. The aim of normal forms is to avoid some of the pitfalls of poor database design which you learned about in *Block 1*. For example, if your relational representation is transformed into a database where data items are replicated, then this may lead to practical problems such as the maintenance of consistency. As a concrete example, if a student changes her name from Julia Glum to Julia Happy, and if data concerning student names is replicated, how do you know that you have changed all instances of Julia Glum?

As noted in the *Course Guide*, this block is less descriptive and more concerned with technical details than *Block 1*. Since we will be focusing heavily on theory, you do not need access to a computer, but you may find it helpful to refer to the relevant summary database card. When it comes to the exercises, we encourage you to avoid simply reading the solutions: doing the exercises is an integral part of the learning experience, so write down solutions for all the exercises as you progress through the block.

It's also worth emphasising that the sections of this block are not of equal length. You will probably spend about a third of your time in studying this block on Section 2, another third on Sections 3 and 4, and the final third on Section 5. And don't forget to leave time for doing the assignment!

2

The structure of relational representations

You have already met the concepts of a relation (briefly) and of an entity attribute in Block 1.

The domain construct is not the same as the domain of discourse, as we shall see.

The structure of a relational representation is based on three constructs: **relation**, **attribute** and **domain**.

We begin by giving some informal definitions of these constructs, accompanied by examples, in order to build up to the more formal definitions that we shall give later. We shall discuss the fundamental construct of relation together with its attributes in Subsection 2.1, before introducing the domain construct and the formal definition of relation in Subsection 2.2.

2.1

The structure of relations

A relation can be pictured as a form of table, with attributes being named columns of the table. Such a table can be used to represent a set of occurrences of an entity type, with each row representing one occurrence.

The first example that we shall look at is the *Enrolment* relation. Figure 2.1 depicts the *Enrolment* relation as a table. The table has three columns, labelled *StudentId*, *CourseCode* and *EnrolmentDate*. That is, the table depicts the *Enrolment* relation and its three named attributes. We should note that this depiction of the relation, the table, shows both rows and columns in an order whereas a relation has no such order – but we shall discuss this in more detail later.

Enrolment

StudentId	CourseCode	EnrolmentDate
s01	c4	Jan 12, 2005
s02	c5	Jan 1, 2005
s02	c7	Jun 12, 2005
s05	c2	Jun 4, 2004
s05	c7	Oct 18, 2004
s07	c4	Dec 12, 2004
s09	c4	Dec 16, 2004
s09	c2	Dec 18, 2004
s09	c7	Dec 15, 2004
s10	c7	Jun 20, 2004
s10	c4	May 5, 2004
s22	c2	Mar 15, 2002
s38	c2	Sep 18, 2003
s38	c5	Mar 9, 2004
s46	c2	Mar 1, 2002
s57	c4	Jun 30, 2001
s57	c5	Jan 20, 2003

Figure 2.1 The *Enrolment* relation depicted as a table

Incidentally, here the term *attribute* is being used as a construct in a relation whereas previously, in *Block 1*, you met it as an attribute of an entity type. Since they closely correspond this should not cause a problem but occasionally we may need to spell out rather pedantically that we are referring to the attribute of an entity type or to the attribute of a relation, as appropriate.

In Figure 2.1, each row of the table corresponds to an occurrence of the entity type **Enrolment** in the University E–R model, as introduced in *Block 1*. Each column of the table (corresponding to each attribute of the relation) represents the values of a particular attribute of the **Enrolment** entity type. The column headings for the table (that is, the names of the attributes of the relation) correspond to the attributes that comprise the **Enrolment** entity type.

We may write down the individual rows using an angled-bracket notation. Thus, the row <s01, c4, 'Jan 12, 2005'> corresponds to one occurrence of the **Enrolment** entity type, namely the occurrence for which the value of the identifier (**StudentId**, **CourseCode**) is (s01, c4). Within this row, the value c4 (that is, a value of the *CourseCode* attribute in the relation) corresponds to the value of the **CourseCode** attribute for that entity. Similarly the value 'Jan 12, 2005' corresponds to the value of the attribute **EnrolmentDate** in the entity. Commas act as separators for the attribute values. Where commas are part of a string value, as in the *EnrolmentDate* column above, quotation marks are used to delineate the string. This should make it clear where a comma is part of a string value and where it is separating attribute values. Where there isn't any such ambiguity, we will omit the quotation marks.

We should emphasise that tables, such as that in Figure 2.1, are only a convenient depiction of a relation. In particular, the orders of the rows and columns as inevitably shown in a table have no significance to a relation. In fact, a row in a relational table represents a **tuple** in a relation, where a tuple is a set of values, one for each of the relation's attributes. As you may already know, all the elements of a set are distinct and the order of the elements is irrelevant. That is, the set {a, b, c} is identical to the set {b, c, a}. So the same tuple can be represented by the row <s01, c4, 'Jan 12, 2005'> in the table of Figure 2.1 and by (say) the row <'Jan 12, 2005', s01, c4> in a table where the column headings are in the order *EnrolmentDate*, *StudentId* and *CourseCode*. A relation is a set of such tuples – in the same way that a table can be considered to be a set of rows. Again, because the elements of a set are not ordered, the rows of a table can be in any order and still depict the same relation. We shall return to the formal definition of a relation later, in Subsection 2.2.

Despite the fact that the order of the attributes is not significant, we shall often use the angled-bracket notation to depict a row of a table (and hence the corresponding tuple), provided there is no ambiguity in the given context about how the values match up with the attributes.

EXERCISE 2.1

How many occurrences of the **Enrolment** entity type are represented in the *Enrolment* relation depicted as a table in Figure 2.1? Is there any significance in the order in which the rows are printed?

You will have noticed that we have chosen the same name for the relation as for the entity type, and the same names for the columns (that is, for the attributes of the relation) as for the attributes of the entity type. We did this so as to reinforce the correspondence between entity types and relations. However, this is just a question of choice: the names do *not* have to be the same. When the names are the same, it is important to remember that, for example, the *Enrolment* relation is different from the

Note that we're not concerned with implementation here, so the exact nature of the quotation marks (single or double) is not relevant. All that is necessary is that they clearly demarcate the string.



Enrolment entity type. A similar distinction holds between the names of the attributes of an entity type and the names of the attributes of a relation.

In order to reinforce this distinction we have printed E–R model names in a different style to relational names. For example, by **Enrolment** we intend a reference to an entity type, while by *Enrolment* we intend a reference to a relation. In your own work you will need to make a similar distinction, usually by including either the phrase ‘entity type’ or the word ‘relation’ as appropriate.



EXERCISE 2.2

According to the convention adopted in this course, is *Student* the name of a relation or an entity type?

We now explore further the properties of relations in terms of the properties of tables representing relations.

Properties of relations

As you have seen, a relation consists of attributes and may be depicted as a form of table. We shall call a table representing a relation, a **relational table**. A relational table is not, however, just *any* kind of table. Specifically, it is a table that adheres to a set of rules, as follows.

- 1 Each value in the table is *atomic*; that is, for each row, the value within a column is always *one* value and never a *group* of values. For example, the row <s07, c4, ‘Dec 12, 2004’> is made up of just one *StudentId* attribute value, one *CourseCode* value and one *EnrolmentDate* value.
- 2 The values within a column (that is, the values of an attribute) are all of the same kind. For example, the values for the *StudentId* column (that is, for the *StudentId* attribute) are strings consisting of 3 characters, the first being the character ‘s’ and the next two, numerals such as ‘0’ or ‘5’.
- 3 Each column of the table has a name, different from any other in the table, by which it may be identified, e.g. *StudentId*.
- 4 Each row is unique, meaning that it is different in some respect from each other row.
- 5 The ordering of rows and columns is not significant. For example, the rows need not have been printed in ascending order of *StudentId*, and *StudentId* need not have been the first column.

The table *Enrolment* is a valid depiction of a relation according to these rules.

We now explore some of the implications of these rules.

Rule 1: atomic entries

The first rule (atomic entries) prevents certain tables from being regarded as the depictions of relations. As an example of a table which contradicts this first rule, consider Figure 2.2.

<i>StudentId</i>	<i>CourseCodes</i>
s01	c4
s05	c2, c7
s07	c4
s09	c4, c2, c7
s10	c7, c4

Figure 2.2 A table which cannot be regarded as a relational table

The table in Figure 2.2 consists of two columns. The first column, *StudentId*, is straightforward and its values conform to the rule. The second column, *CourseCodes*, is a column in which the entry in any given row is sometimes multivalued, for example, in the rows determined by the *StudentId* values s05, s09 and s10. Thus the table in Figure 2.2 does not conform to the first rule because the entries in the *CourseCodes* column are not all atomic. It is therefore not a depiction of a valid relation.

Rule 2: consistent column values

The *Enrolment* table in Figure 2.1 obeys the second rule. Every column of every row has a value and the values within each column are all of the same kind: they are homogeneous. More precisely, we say that all the values within a column are drawn from the same **domain**. (This term will be defined in Subsection 2.2.)

Note the difference between the columns *CourseCodes* in Figure 2.2 and *EnrolmentDate* in Figure 2.1. In the first case, the commas denote different values of the *CourseCodes* attribute, as in 'c2, c7'; in the second, the commas denote the separation of a value into parts, as in 'month and day, year'.

EXERCISE 2.3

Consider the tables in Figures 2.3 and 2.4. Can these tables be regarded as depictions of relations?



<i>StudentId</i>	<i>CourseCode</i>	<i>EnrolmentDate</i>	<i>Tutor</i>
s01	c4	Jan 12, 2005	Jennings
s05	c2	Jun 4, 2004	5212
s05	c7	Oct 18, 2004	5212
s07	c4	Dec 12, 2004	Jennings
s09	c4	Dec 16, 2004	5212

Figure 2.3 A suspect relational table

<i>StudentId</i>	<i>CourseCode</i>	<i>EnrolmentDate</i>	<i>Tutor</i>
s09	c4	Dec 16, 2004	
s09	c2	Dec 18, 2004	Redhead
s09	c7	Dec 15, 2004	Jennings
s10	c4	May 5, 2004	
s10	c7	June 20, 2004	Redhead

Figure 2.4 Another suspect relational table

Rule 3: unique column names

The third rule, which relates to unique column names, means that within a table depicting a relation, a column may be referred to uniquely by its name. Given Rule 5 – which says in part that the ordering of columns is not significant – this is important. You can't refer to (for example) the first column of a table depicting a relation because you don't know the order of the columns in a particular depiction. You can refer to a column only by its name.

Again, the table in Figure 2.1 is consistent with this rule.

Rule 4: distinct rows

The fourth rule means that there are no duplicate rows; any row can be distinguished from any other by its values. That is, there must be a combination of columns that have

Other OU courses may define keys in a somewhat different – but equivalent – way. We shall discuss this equivalent definition in Section 5 of this block.

different values for each row. This is equivalent to saying that any tuple in a relation must be distinguished by its attribute values alone: there is a combination of attributes with different values for each tuple, which will identify the tuple uniquely. This combination of attributes may be all the attributes of the relation. We call a minimal set of such attributes, a **key**.

For example, in the *Enrolment* relation as depicted in Figure 2.1, there is only one key, the pair of attributes (*StudentId*, *CourseCode*). Each value of this pair determines a unique row in the table – a unique tuple – a particular student will enrol on a particular course on only one enrolment date. And this pair is minimal – *StudentId* on its own isn't a key, since a single value of *StudentId* may occur in many rows – a particular student may have enrolled on many courses. Similarly, *CourseCode* isn't a key – a particular course will potentially have many students enrolling on it.

The fourth rule requires that such a key always exists, if the table is to be the depiction of a relation. In particular, one of these keys (if there is more than one) can be chosen to be the **primary key**. Primary keys (of relations) correspond to identifiers (of entities). From now on, if we know which attribute(s) comprise the primary key of a relation, we shall often underline them in the heading of the table depicting the relation, as in Figure 2.5.

Enrolment

<u>StudentId</u>	<u>CourseCode</u>	EnrolmentDate
s01	c4	Jan 12, 2005
s02	c5	Jan 1, 2005
s02	c7	Jun 12, 2005
s05	c2	Jun 4, 2004
s05	c7	Oct 18, 2004
s07	c4	Dec 12, 2004
s09	c4	Dec 16, 2004
s09	c2	Dec 18, 2004
s09	c7	Dec 15, 2004
s10	c7	Jun 20, 2004
s10	c4	May 5, 2004
s22	c2	Mar 15, 2002
s38	c2	Sep 18, 2003
s38	c5	Mar 9, 2004
s46	c2	Mar 1, 2002
s57	c4	Jun 30, 2001
s57	c5	Jan 20, 2003

Figure 2.5 The *Enrolment* relation revisited

We shall say more about the meaning of relations later.

Note that the choice of primary key is determined by the meaning of the relation rather than the particular values of the data. For example, in Figure 2.5, it so happens that each value of *EnrolmentDate* is unique: each such date determines a unique row. But it is clear that this is just a coincidence: there is no reason why there can't be several

students enrolling for several courses on the same date (or one student enrolling for several courses, or several students enrolling for the same course).

Rule 5: insignificance of order

The fifth, and last, rule states that the ordering of rows and columns in a relation is not significant, as we have said before. This means that the table depicted in Figure 2.6 represents the *same* relation (*Enrolment*) as that in Figure 2.5, even though they may be considered to be *different tables*. The rule means that any reference to the ordering of rows and columns in a relation *has no meaning*.

Enrolment

CourseCode	EnrolmentDate	StudentId
c2	Jun 4, 2004	s05
c2	Dec 18, 2004	s09
c2	Mar 15, 2002	s22
c2	Sep 18, 2003	s38
c2	Mar 1, 2002	s46
c4	Jan 12, 2005	s01
c4	Dec 12, 2004	s07
c4	Dec 16, 2004	s09
c4	May 5, 2004	s10
c4	Jun 30, 2001	s57
c5	Jan 1, 2005	s02
c5	Mar 9, 2004	s38
c5	Jan 20, 2003	s57
c7	Jun 12, 2005	s02
c7	Oct 18, 2004	s05
c7	Dec 15, 2004	s09
c7	Jun 20, 2004	s10

Figure 2.6 The *Enrolment* relation again

EXERCISE 2.4

Given that you don't know how rows or columns are ordered in a particular depiction of a relation, how can you refer to a particular row or a particular column?



In summary, a relation is an *abstract* structure whereas a table is a *depiction* of such a structure, with certain features (such as the physical ordering of columns and rows) that are merely properties of the depiction rather than of the abstraction.

Relational terminology

The number of attributes of a relation is called the **degree** of a relation. Please note that this is **not** the same as the degree of a relationship in an E-R model. So, since *Enrolment* has the three attributes *StudentId*, *CourseCode* and *EnrolmentDate*, its degree is 3.

As we said before, rows in a relational table correspond to tuples in a relation. The **cardinality** of a relation is the number of tuples in the relation. The relation *Enrolment* as depicted in Figure 2.1 has cardinality 17. The terminology is illustrated in Figure 2.7 below.

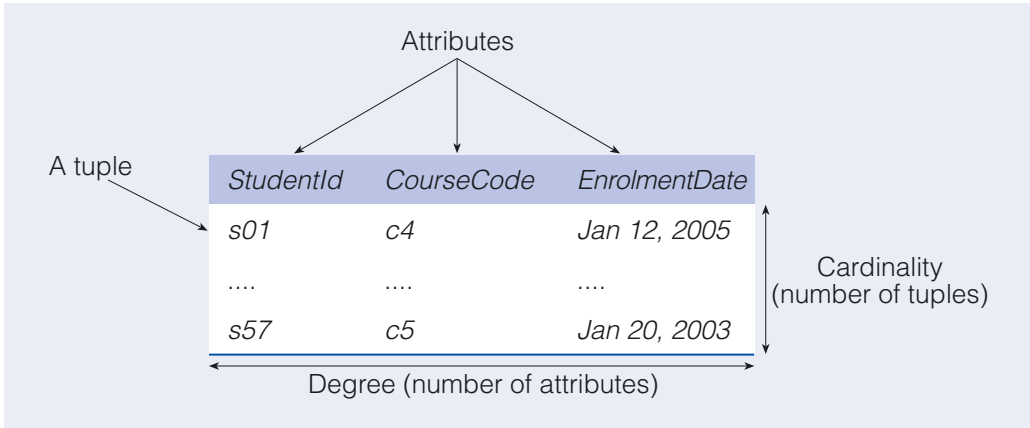


Figure 2.7 Relational terminology



EXERCISE 2.5

What are the degree and cardinality of the *ShortRegion* relation depicted in Figure 2.8?

ShortRegion

RegionNumber	Address	Telephone	EmailAddress
3	Block 9, The Campus, Walton Hill	01670 245365	region3@open.fake.address
4	Suite 2, Fawltly Towers, Torquay	02563 13829	region4@open.fake.address
12	The Office, New York	10898 227191	region12@open.fake.address

Figure 2.8 Table depicting the *ShortRegion* relation



EXERCISE 2.6

The following terms are used to describe aspects of a table:

- (a) Column name
- (b) Column entries
- (c) Row
- (d) Number of columns
- (e) Number of rows

Write down the equivalent relational terms.

EXERCISE 2.7

What is the difference between a table and a relation?



The **heading** of a relation is defined to be the list of its attribute names, which we often label by the name of the relation. The set of tuples of a relation is called the **body** of the relation.

By convention, the heading of a relation is written in a form very similar to that employed for entity type headings:

RelationName(Attribute1, Attribute2, ..., AttributeN)

Remember that we are using a different style of printing for relations from that used for entity types, in order to emphasise that it is a relation that is being written.

When writing down the heading of a relation, it is convenient to indicate the primary key of the relation. This is done by underlining the appropriate attribute(s). By convention, the primary key is placed first. Thus the heading of the *Enrolment* relation is

Enrolment(StudentId, CourseCode, EnrolmentDate)

EXERCISE 2.8

Write down the heading of the *ShortRegion* relation in Figure 2.8.



The meaning of a relation can be defined by specifying when a given tuple belongs to the relation by means of a **natural language predicate**. This defines feasible tuples in terms of the value of the primary key and is best illustrated by an example, as below.

<a, b, c> is a tuple of *Enrolment* if and only if a student with a *StudentId* of 'a' enrolling on a course with code 'b' does so on date 'c'.

Relational theory and logic

You may recognise that 'predicate' is a term used in logic. This is not an accident: relational theory does have a basis in logic. To expand on this, as you may already know, a logical proposition is a statement which has a truth value – it is either true or false. So, for example, 'an elephant is a bird' is a proposition having the truth value false, whereas 'an elephant is a mammal' is a proposition having the truth value true. According to Figure 2.1, 's01 enrolled on c4 on Jan 12, 2005' is a proposition which is true. This latter proposition might be considered as an instantiation of the natural language predicate introduced above, and a relation may be thought of as being the set of all the (known) true propositions which can be instantiated from the natural language predicate.

EXERCISE 2.9

Write down a natural language predicate for the *ShortRegion* relation in Figure 2.8.



Relations and relation variables (relvars)

In this course, we focus on relations as values (albeit values with a rather complex structure – as we have seen, a relation is a set of tuples, where tuples are themselves sets of values). However, some writers on relational theory prefer to talk about relation variables (relvars) rather than relations. The values assigned to relvars are relations – the same relvar can thus represent different relations at different times. Relvars are of special importance when your aim is to produce a relational representation which can be implemented directly as a programming language – in such a scenario, you might be interested in issues of updating, for example. Relations – as values – cannot be updated; relvars can.

2.2 Domains

So far, we have considered informally the concepts of *relation*, *attribute* and *tuple*. We now introduce **domain**. Informally, a domain defines a set of values that a particular attribute can take. You might think of it as constraining these values so that they reflect the reality of the situation. For example, a relation may be representing the entity type of a person; one of its attributes might be age. You wouldn't want the values of this attribute to be negative – or to be greater than (say) 120.

The definition of a domain is as follows:

A domain is a named set of values from which one or more attributes draw their actual values.

It is important to emphasise that a domain is a theoretical construct. We are not, at this point, interested in how a domain might be implemented (implementation issues are considered in future blocks).

If two attributes are defined on the same domain, that is, draw values from the same domain, then these values can be compared – we can say whether or not they are equal. This is *not* true if they are *not* defined on the same domain, even if their values are the same. For example, there may be domains called *Age* and *Height*, both of which may have integer values between 0 and 250, where the values of *Height* are interpreted as centimetres, and the values of *Age* as years. However, we cannot think of a situation in which you would want to check whether a particular value of age was or was not equal to a particular value of height. Thus, it makes sense to define *Age* and *Height* as separate domains, even though they are the same set of values.

Domain definitions should reflect the information contained about the attribute values in the E–R conceptual model and associated requirement documents. For example, in the University conceptual model, the data dictionary or the catalog may record the fact that the attribute **RegionNumber** can only take values between 1 and 12. Domain definitions should also act as a specification for the implementer – we want the implementer to implement *RegionNumber* as the set of integers {1,...,12}. Sometimes, we wish to defer any consideration of the structure of the values of a domain until implementation. The domain *Addresses* is a case in point: we defer until implementation the consideration of whether we wish an address to be a single data object or a more complex object, consisting of a street part, a city part, a postcode part and so on. In this case, we just give the name of the domain.

In Figure 2.9, we present some plausible definitions of domains which will be needed when we build up the relational model of the University. The notation used in Figure 2.9 is not that used in any particular relational DBMS: it is merely meant to be understandable. In some domains, the full set of values can be enumerated, as in *AssignmentNumbers* = {1, 2, 3, 4, 5} or *Credits* = {30, 60}. In others, we make use of abbreviations which are almost universally understood within computing. For example, by {s01...s99}, we mean the enumeration {s01, s02, s03, ..., s99}. Note also the use of curly brackets {} to indicate that a domain is a set.

domains

RegionNumbers = {1...12}
Addresses
TelephoneNumbers = {string of numerals}
EmailAddresses = {string@string}
StudentIds = {s01...s99}
Names = {Family Name}
TitlesOfCourses = {string of alphabetic characters}
Dates = standard dates
StaffNos = {abcd, where a, b, c, d are numerals}
CourseCodes = {c1...c9}
Credits = {30, 60}
Limits = Integer
AssignmentNumbers = {1, 2, 3, 4, 5}
Percentages = {0...100}
Locations = {string of alphabetic characters}
FormNumbers = {SCxyz, where x, y, z are numerals}

Figure 2.9 Some plausible domains for the *University* relation

In Figure 2.9, you might wonder why the domain of *Names* is given as {Family Name}, rather than, for example, {string of alphabetic characters}. The reason for this is to define exactly what needs to be implemented. For example, the name of the student with identifier s01 could be recorded as Antony Aloysius Akeroyd, or as A. A. Akeroyd, or as Antony A. Akeroyd or as Akeroyd Antony A. – but we have taken the decision that only the family name, Akeroyd, will be recorded.

Looking at the *Enrolment* relation in Figure 2.1 in the light of Figure 2.9, the attribute *StudentId* is defined on the domain *StudentIds*, which means that its values can be s01 or s02 and so on, up to s99. Similarly, *CourseCode* is defined on *CourseCodes* and can be c1, c2, ..., c9.

EXERCISE 2.10

Given that, in Figure 2.9, *Locations* and *TitlesOfCourses* are both sets of the same values (both referring to names, though of places and courses respectively), why does it make sense to have them as separate domains?

EXERCISE 2.11

In the *University* relational representation, suppose we have Elvis Holly with staff number 3333 and another member of staff, Buddy Presley, with telephone number 3333. Within the context of Figure 2.9, can we say that Elvis's staff number is the same as Buddy's telephone number?





EXERCISE 2.12

Figure 2.10 depicts a relation *Enrolment1*, where each attribute is declared over the same domain as the attribute with the same name in *Enrolment*. Which of the tuples depicted in Figure 2.10 are legal?

Enrolment1

<i>StudentId</i>	<i>CourseCode</i>	<i>EnrolmentDate</i>
s01	c4	Jan 12, 2005
s07	c4	Dec 12, 2004
s07	c3	Dec 12, 2004
s08	c10	Dec 18, 2004
s07	c4	Dec 15, 2004

Figure 2.10 Table depicting *Enrolment1*

Formal definition of a relation

We are now in a position to give a formal definition of a relation.

A **relation** *R* on domains *D*₁, *D*₂, ..., *D*_{*n*}, not necessarily distinct, consists of a **heading** and a **body**.

The **heading** consists of a fixed set of **attributes** (*A*₁, *A*₂, ..., *A*_{*n*}) such that each attribute *A*_{*i*} corresponds to exactly one **domain** *D*_{*i*} (*i* = 1, 2, ..., *n*).

The **body** consists of a set of distinct tuples each of the form <*A*₁:*v*₁, *A*₂:*v*₂, ..., *A*_{*n*}:*v*_{*n*}>, such that each *v*_{*i*} is a value from the domain *D*_{*i*} corresponding to the attribute *A*_{*i*} (*i* = 1, 2, ..., *n*).

All of this new notation may be confusing, so let us consider an example. In the relation *Enrolment*, the domains *D*₁, *D*₂ and *D*₃ are *StudentIds*, *CourseCodes* and *Dates*. In this case, the domains are distinct, that is, there is a different domain for each attribute, but this isn't necessarily always the case: different attributes may be defined over the same domain. In fact, as you will see in Section 2.5, the representation of relationships depends on different attributes being defined over the same domain so that their values can be matched (and hence values might be shared, as in the example of the bank documents in the Introduction to this block).

According to this formal definition, the heading is the set of attributes (*StudentId*, *CourseCode*, *EnrolmentDate*). The fact that this is a set means that it has no ordering, and hence attributes must be identified by name alone. Thus each attribute must have a unique name. This, of course, corresponds to Rule 3 for a relational table, as described in Subsection 2.1.

The body is the set of tuples, written to include the attribute names for each value. So, for example, instead of writing <s01, c4, 'Jan 12, 2005'>, we write <StudentId: s01, CourseCode: c4, EnrolmentDate: 'Jan 12, 2005'>. This is strictly necessary because of tuples being a *set* of values, and hence having no specified order. If you can't match values with the attributes to which they refer by order considerations, then you have to specify the attributes. This is particularly relevant when you have two attributes defined over the same domain – that is, when two attributes may have the same value. In practice, we often assume the order, as we did in Subsection 2.1.

Individual tuples can be identified by means of the value of the primary key.

2.3 Developing a relational representation

In this subsection, we begin to develop a relational representation. A relational representation consists of the definitions of domains and declarations of relations together with any necessary constraints.

We have already discussed domains and now we will describe how individual relations are declared. As we shall see, such declarations include *at least* the name of the relation, the names of its attributes together with the domains over which they are defined, and an identified primary key. Later, in Subsections 2.5 and 2.6, we shall discuss how relationships can be represented using relations.

We have also already discussed how the concepts of primary keys and domains can enforce constraints on tuples and on the values of attributes. In Section 4, we shall consider some more general constraints.

The first line of a relational representation is simply for identification, as in:

model *University*

The next part of the model consists of a definition of the domains, as in Figure 2.9.

Declaring relations

Having defined the domains, we can now declare the relations. Each declaration has the following form:

```
relation <NameOfRelation>
  <NameOfAttribute1>: <DomainOfAttribute1>
  <NameOfAttribute2>: <DomainOfAttribute2>
  ...
primary key <primary key>
```

Figure 2.11 The basic form of a relation declaration

Note that this syntax does not correspond to any particular relational language.

If the primary key consists of more than one attribute, the attribute names are enclosed in parentheses, and are separated by commas, as in:

primary key (<Attribute1>, <Attribute2>, ...)

If the primary key has only one attribute, then the parentheses may be omitted. By convention, the attributes which compose the primary key are placed at the top of the list of attributes.

So, for example, the declaration of the relation *Enrolment* (as in Figure 2.5) in terms of the domains in Figure 2.9 is:

```
relation Enrolment
  StudentId: StudentIds
  CourseCode: CourseCodes
  EnrolmentDate: Dates
primary key (StudentId, CourseCode)
```

Figure 2.12 The *Enrolment* relation

This is the most basic form of a relation declaration. In later sections we will discuss how this might be extended.

EXERCISE 2.13

What is the essential difference – that is, apart from presentation – between a relational heading as defined at the end of Subsection 2.1, and the basic declaration of a relation as in Figure 2.11?





EXERCISE 2.14

In the style of Figure 2.11, declare the relation *Region*, as in the University model. This has the same heading as the relation *ShortRegion* in Figure 2.8 but a different body.

Note that we can easily derive the heading of a relation from its declaration, and the declaration also gives us much information about the body; it does not, however, tell us exactly which set of tuples constitutes the body.

2.4 Candidate keys, primary keys and alternate keys

In Subsection 2.1, we introduced the concept of keys in the context of Rule 4 for tables depicting relations – that is, the rule that every row in such a table is unique. In terms of relations, this translates into the requirement that every tuple be unique; within a given relation, there must be an attribute (or combination of attributes) such that for any two tuples in the relation, the values of the attribute (or values of the combination) are different. In other words, the value of this attribute (or value of the combination) uniquely define a tuple.

There may be more than one attribute, or combination of attributes, with this property. For example, we may declare a relation *Person*, with attributes *NationalInsuranceNumber*, *Name*, *DateOfBirth*, *Address*, *TelephoneNumber*, *EmailAddress*. In this case, *NationalInsuranceNumber* will certainly uniquely identify a tuple, but so probably will the combination of attributes (*Name*, *DateOfBirth*, *Address*). On the other hand, neither *TelephoneNumber* nor *EmailAddress* will uniquely identify a tuple – people can share both telephone numbers and email addresses. We should point out here that the property of uniqueness is often dependent on the domain of discourse, that is, the ‘closed world’ within which we are developing our relational representation. For example, ‘s01’ as a staff identifier is unique within the University model, but it’s reasonable to suppose that there are plenty of other organisations which have a staff member identified by ‘s01’.

In order to be a key, a combination of attributes must have two properties: not just uniqueness as described above, but also minimality (note that some texts use the term ‘irreducible’ rather than ‘minimal’). This latter property means that there is no proper subset of the combination which guarantees uniqueness. To illustrate with the combination introduced above, (*Name*, *DateOfBirth*, *Address*): clearly, no single one of these attributes fits the bill – many people potentially share the same name, date of birth or address. Similarly no pair of attributes is suitable – different people could have the same name and date of birth, or the same address and name (maybe the address is that of a hostel), or the same date of birth and address (maybe they are twins). So (*Name*, *DateOfBirth*, *Address*) is indeed a minimal set of attributes having the uniqueness property.

We distinguish between different types of keys: candidate, primary and alternate. Informally, a **candidate key** is any key; a **primary key** is a selected candidate key, and an **alternate key** is any candidate key which hasn’t been selected to be the primary key. So in the *Person* example above, there are two candidate keys, *NationalInsuranceNumber* and (*Name*, *DateOfBirth*, *Address*), from which we shall choose *NationalInsuranceNumber* as the primary key, leaving (*Name*, *DateOfBirth*, *Address*) as the alternate key.

You met the term ‘domain of discourse’ in Block 1.

As we pointed out in our discussion on Rule 4 in Subsection 2.1, the concept of uniqueness depends on the meaning of the relation rather than on particular values of the data.

If C is a **subset** of the set of attributes K , then C is either the empty set or a set of attributes, each of which is also an attribute in K . So, for example, if K has set of attributes $\{A_1, A_2, A_3\}$, then C could be $\{A_1\}$ or $\{A_2, A_3\}$, and so on, or even the full set $\{A_1, A_2, A_3\}$. If it is not either the empty set or the full set, then C is said to be a **proper** subset of K . So, for example, $\{A_1\}$ is a proper subset of K , whereas $\{A_1, A_2, A_3\}$ is not.

Here are the formal definitions.

A set of attributes K is a **candidate key** for a relation R if and only if it possesses the following two properties:

- (i) **Uniqueness.** It is illegal for R to contain two distinct tuples with the same value for K .
- (ii) **Minimality.** No proper subset of K has the uniqueness property.

The **primary key** of a relation is one particular key chosen from the candidate keys.

An **alternate key** of a relation is a candidate key which is not the primary key.

EXERCISE 2.15

Given the relation *Staff1(StaffIdentifier, Name, Address, NationalInsuranceNumber)*, list the candidate key(s), choose a primary key and list any alternate keys.



It is important to note that the declaration of primary and alternate keys imposes constraints on the relation to reflect a real life situation. For example, suppose you have the following relation which represents information about general practitioners and their secretaries:

GeneralPractitioner(GPId, GPName, SecId, SecName)

and you are told that *GPId* is the primary key and *SecId* is an alternate key. Then, if you think of the table depicting this relation, you know that each value of *SecId* occurs only once in the table – that is, each secretary works for only one GP. You also know, from the fact that *GPId* is the primary key of the relation, that a GP has only one secretary (since any GP is associated with only one tuple in the relation and any attribute of a relation has only one value). Thus, there is a 1:1 mapping between GPs and secretaries.

Representing an alternate key in the declaration of a relation

Alternate keys are represented in a relational heading declaration in a similar manner to primary keys, using the keywords **alternate key**, as illustrated in Figure 2.13.

relation *GeneralPractitioner*

GPId: GPCodes

GPName: Names

SecId: SecCodes

SecName: Names

primary key *GPId*

alternate key *SecId*

Figure 2.13 Representing an alternate key: the *GeneralPractitioner* relation (assuming suitable domains)

EXERCISE 2.16

Given the following relation *ProgrammingTask*, again assuming suitable domains, what can you deduce about the relationship between a task and a programmer?

relation *ProgrammingTask*

TaskId: TaskCodes

TaskDescription: String

ProgrammerId: ProgrammerCodes

ProgrammerName: Names

primary key *TaskId*

alternate key *ProgrammerId*



If *ProgrammerId* had *not* been declared as an alternate key, which of the following statements would have been true?

- (i) A task may have several programmers allocated to it.
- (ii) A programmer may be allocated to several tasks.
- (iii) A task may have several programmers allocated to it and a programmer may be allocated to several tasks.

The following exercise is intended to give you practice in understanding the constraints imposed by keys. It concerns a relation *Appointments* which records data about patients' appointments with consultants.



EXERCISE 2.17

What is the difference in meaning, as expressed by the definition of the underlined primary keys, between the two relations *Appointments1* and *Appointments2* given below?

Appointments1(PatientId, ApptDate, ApptTime, ConsultantId)

Appointments2(PatientId, ApptDate, ApptTime, ConsultantId)

EXERCISE 2.18

Which of the following statements are true?

- (i) A relation has only one primary key.
- (ii) A relation must have a candidate key.
- (iii) A relation may not have more than one candidate key.
- (iv) A relation must have an alternate key.

Natural language predicates for relations with more than one candidate key

In Subsection 2.1, we discussed how the meaning (semantics) of a relation can be captured in a natural language predicate, where an allowable tuple is expressed in terms of the primary key. Where there is a choice of primary key, that is, more than one candidate key, then there should be multiple expressions, one for each candidate key.

For example, the natural language predicate for the relation *GeneralPractitioner* above is:

<a, b, c, d> is a tuple of *GeneralPractitioner* if and only if

the GP with identifier 'a' has name 'b' and is allocated a secretary with identifier 'c' and name 'd'

and

the secretary with identifier 'c' has name 'd' and works for the GP with identifier 'a' and name 'b'.

In the following Sections, 2.5 and 2.6, we discuss two different ways of representing relationships using relations.

2.5 Representing relationships using foreign keys

We now consider how to represent relationships in a relational representation. In an E–R diagram, relationships are represented by lines between entity types with crow's feet and empty or filled circles as appropriate. In a relational representation, there are only domains, relations, attributes and constraints – how we use these to represent a relationship depends on the degree and participation conditions of the relationship, as we shall now see.

Sometimes relationships can be represented by what might be thought of informally as matching attribute values in different relations, just as in the Introduction to this block we pointed out that the relationship between different documents pertaining to you from your bank can be represented by the same account number (yours!) occurring on each. It is important to note that this can be done only if the attributes are defined over the same domain.

Our first example is of the relationship **Manages** from the University conceptual model that you met in *Block 1*. Figure 2.14 shows the relevant fragment from this model.

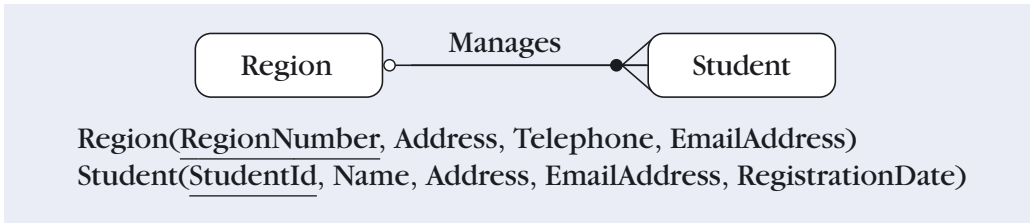


Figure 2.14 Fragment of the E–R model for the University, showing the **Manages** relationship

The diagram tells us that one region may manage zero, one or more students; one student must be managed by one region. Figure 2.15 illustrates some occurrences of the **Manages** relationship.

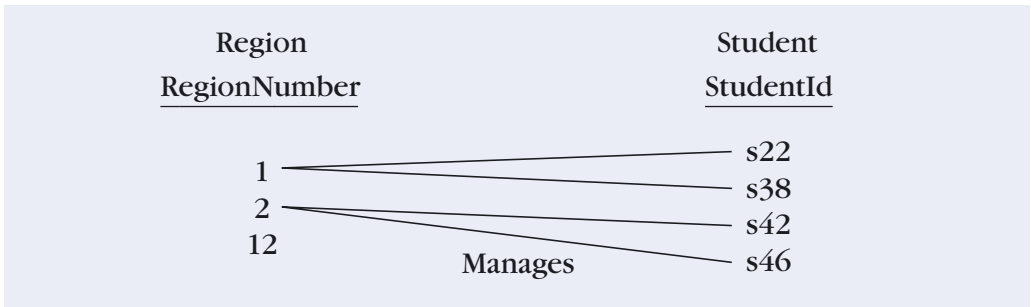


Figure 2.15 Occurrences of the **Manages** relationship

We can represent the **Manages** relationship by adding to each tuple of the *Student* relation, the number of the region which manages that student. So, for example, the tuple <s22, Bryant, '84 Brook Street, Little Hacking', A.Bryant@greenmail.fake.uk, 'Jun 21, 2000'> is extended to <s22, Bryant, '84 Brook Street, Little Hacking', A.Bryant@greenmail.fake.uk, 'Jun 21, 2000', 1>, indicating that the student with *StudentId* s22 is managed by the region with *RegionNumber* 1. The *Student* relation thus has an extra attribute added to the set of attributes of the **Student** entity. This attribute takes its values from the set of values taken by the primary key of *Region*, that is, there must be a (unique) region with *RegionNumber* 1.

The relational heading for *Student* is therefore as follows:

```
Student(StudentId, Name, Address, EmailAddress, RegistrationDate,
        RegionNumber)
```

RegionNumber is said to be a **foreign key** posted (from the primary key of *Region*) into the relation *Student*. We shall give a formal definition of foreign key later.

The relational heading for *Region* is derived directly from the corresponding entity type as follows:

```
Region(RegionNumber, Address, Telephone, EmailAddress)
```



EXERCISE 2.19

Write down the tuple of the *Student* relation corresponding to the entity occurrence <s42, Reddick, '23 Kestrel Lane, Dudley', dave@belwise.fake.co.uk, 'Apr 23, 2002'>. (Hint: the information you need is in Figure 2.15.)

EXERCISE 2.20

Instead of posting the primary key of *Region* as a foreign key into *Student*, could we have posted the primary key of *Student* into *Region* to give the following relational headings?

```
Student(StudentId, Name, Address, EmailAddress, RegistrationDate)
Region(RegionNumber, Address, Telephone, EmailAddress, StudentId)
```

If the participation of **A** in **R** were optional, and we were to post the foreign key from **B** into **A**, there would be the additional problem of what to do if a particular occurrence of **A** were not associated with one of **B**, given that every tuple in the relation representing **A** has to have a value.

Representing a mandatory participation at the :1 end of a 1:*n* relationship has to be done by way of a constraint, as we shall see in Section 4.

Later in this subsection we will deal with the situation in which the participation of a relationship at the :*n* end is optional.

The solution to Exercise 2.20 is important. When we are representing a 1:*n* relationship **R** between entity types **A** and **B**, as in Figure 2.16, we post the foreign key from **A** into **B**, and not the other way round. This is because each occurrence of **B** is associated with a single occurrence of **A** (provided that the participation of **B** in the relationship is mandatory), whereas an occurrence of **A** is associated with potentially many occurrences of **B** – and we can't have attributes in the relation representing **A** taking more than one value.

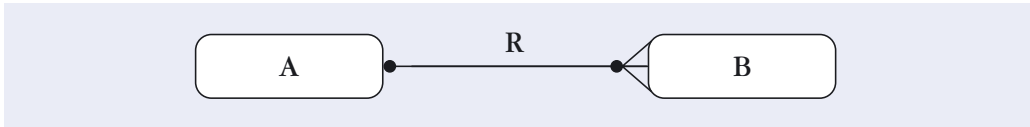


Figure 2.16 A 1:*n* relationship

Declaring a relation to represent relationships using posted foreign keys

In order to declare a relation using a posted foreign key, we add the foreign key to the list of attributes, and then declare it explicitly as a foreign key together with the relation which it references (that is, from where it was posted), below the primary key declaration.

The name of the relationship represented by the foreign key declaration is written above the declaration for the purposes of clarity. The fact that this is a comment, and not part of the declaration, is indicated by enclosing it in curly brackets {...}. The declaration of the *Student* heading is shown in Figure 2.17.


```
relation Student
  StudentId: StudentIds
  Name: Names
  Address: Addresses
  EmailAddress: EmailAddresses
  RegistrationDate: Dates
  RegionNumber: RegionNumbers
  primary key StudentId
  {mandatory participation of Student in Manages relationship}
  foreign key RegionNumber references Region
```

Figure 2.17 Declaration of the *Student* relation

Note that we use comments in the relational model to refer back to constraints in the conceptual model.

You should note that the foreign key attribute in *Student* need not necessarily have the name *RegionNumber*. What is important is that the values of the foreign key must be the same as (some or all of) those of the primary key *RegionNumber* of *Region* (and so must necessarily be defined over the same domain).

This means that for every value of *RegionNumber* appearing in a tuple in the *Student* relation, there must be a tuple in the *Region* relation identified by this number. We couldn't have (for example) the tuple <s99, Bloggs, 'Blogg Palace', Bloggs@bloggs.fake.co, 'Nov 14, 2005', 105> without there being a region with number 105. The concept of enforcement of foreign key constraints is called **referential integrity**. We shall come back to this in more detail later, when we discuss what might happen when we want to delete a region – for example, region 1 – which manages at least one student, that is, it occurs as a value of the foreign key in some tuple of *Student*.

EXERCISE 2.21

Figure 2.18 gives a fragment of a hospital conceptual data model similar to the one to which you were introduced in *Block 1*.



- (a) Write down the relational headings of the relations *WardA* and *PatientA*, taking note of the need to represent the relationship **OccupiedBy**.
- (b) Write down the declarations of the relations in the style of Figure 2.17. You may assume suitable definitions for the domains *WardNos*, *WardNames*, *PatientNumbers* and *PatientNames*, over which are defined the attributes *WardNo*, *WardName*, *PatientId* and *PatientName*, respectively.

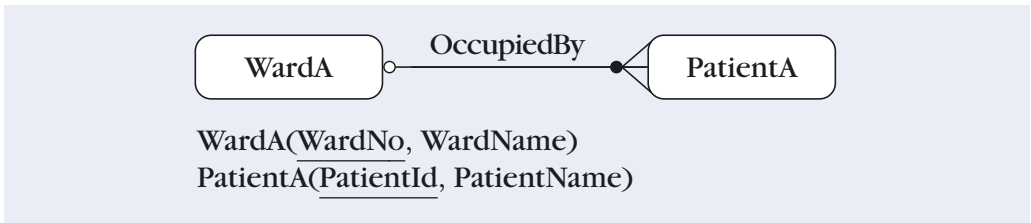


Figure 2.18 Fragment of the Hospital E–R model showing the **OccupiedBy** relationship

EXERCISE 2.22

Given the above example, explain why it is not correct to have a foreign key in *WardA* referencing *PatientA*, that is, why it is not appropriate to have the following relational headings.

```
WardA(WardNo, WardName, PatientId)
PatientA(PatientId, PatientName)
```

We shall now give the formal definition of **foreign key**. You should note from this definition that a foreign key (like a primary key) can be a combination of attributes, rather than just a single attribute, as we have seen so far, and that it can be matched with any candidate key rather than just the primary key of the relation that it references. Later we shall see examples of foreign keys which are combinations of attributes, rather than just a single attribute.

A **foreign key** is an attribute (or combination of attributes) in a relation R_2 whose value in each tuple of R_2 appears as the value of a given candidate key (typically the primary key) of some relation R_1 (where R_1 and R_2 are not necessarily distinct).

The relation having the foreign key is referred to as the **referencing relation** (R_2 in the above definition); the relation from which the foreign key is derived (R_1 above) is referred to as the **referenced relation**.



EXERCISE 2.23

Which is the referenced and which is the referencing relation in the example above concerning the representation of the relationship **Manages** in the relations *Region* and *Student* (as in Figures 2.14 and 2.17)?

The definition of foreign key makes clear that the referenced and referencing relation may be the same – this makes sense when a relationship associates occurrences of the same entity type. There is an example of this in the Hospital conceptual model which you met in *Block 1*, where a nurse may supervise another nurse. We shall return to this example later.

Pre-posted foreign keys

Sometimes a foreign key which matches the values of an attribute (or set of attributes) in one relation with the values of (usually) the primary key in another, does not need to be posted. It corresponds to an attribute which already exists in the entity type corresponding to the first relation. In such a situation, the foreign key may be said to be **pre-posted**.

For example, consider the fragment of the University conceptual data model (which you met in *Block 1*) in Figure 2.19.



Figure 2.19 Fragment of the University conceptual model showing the **StudiedBy** and **EnrolledIn** relationships

In Figure 2.12, we saw that the primary key of *Enrolment* is the pair of attributes (*StudentId*, *CourseCode*), and in Figure 2.17, that the primary key of *Student* is *StudentId*. Although we have not yet defined the relation *Course*, its primary key is *CourseCode*. Figure 2.20 illustrates some occurrences of both the **StudiedBy** and **EnrolledIn** relationships.

Figure 2.20 illustrates that the relationship **StudiedBy** can be represented by matching tuples with the same values of *CourseCode* in both *Course* and *Enrolment*. Similarly the relationship **EnrolledIn** can be represented by matching tuples with the same values of *StudentId* in *Student* and *Enrolment*. The foreign keys, *CourseCode* representing the relationship **StudiedBy** and *StudentId* representing **EnrolledIn**, already exist in the

entity type **Enrolment**. This is due to **Enrolment** being a weak entity type: it cannot exist without the existence of the entity types **Course** and **Student**. *CourseCode* and *StudentId* are thus both pre-posted foreign keys.

You met the concept of weak entity types in Subsection 5.5 of Block 1.

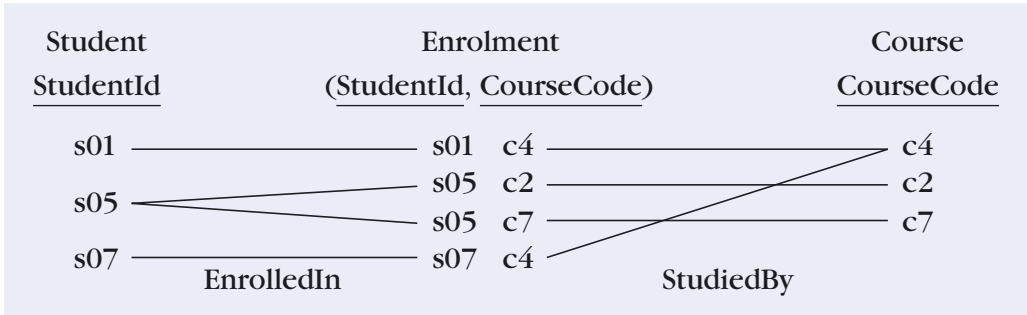


Figure 2.20 Some occurrences of the **StudiedBy** and **EnrolledIn** relationships

EXERCISE 2.24

Declare the relation *Enrolment* in the style of Figure 2.17.



Note that a relationship between a weak entity type and the strong entity type on which it depends cannot necessarily be represented by pre-posted foreign keys, as demonstrated in the exercise below.

EXERCISE 2.25

Consider Figure 2.21, which illustrates some occurrences of a relationship **Mentors** between the entity types **Enrolment** and **Student**. This shows that the student identified by s01 mentors the student with identifier s05 on course c7, and so on.

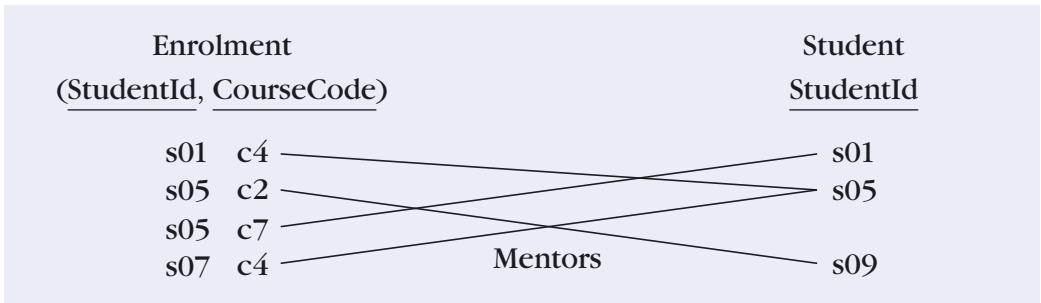


Figure 2.21 Some occurrences of the **Mentors** relationship

Declare the relations *Enrolment* and *Student* so as to represent the relationship **Mentors**.

Representing 1:1 relationships

We have emphasised the fact that when representing a 1:*n* relationship by a foreign key (whether posted or not), the foreign key is declared in the relation at the :*n* end of the relationship and references the primary key in the relation at the :1 end.

One issue in representing 1:1 relationships is: in which relation should the foreign key be declared? As we shall see shortly, the answer to this depends on the participation conditions. Another issue to consider is: in a relation declaration with foreign keys, how do we represent the fact that the relationship represented by the foreign key is 1:1?

See, for example, the note following Exercise 2.20. You should, however, be aware that so far we have only considered 1:*n* relationships where the participation of the relation at the :*n* end is mandatory.

We illustrate both of these issues with an example. Figure 2.22 shows a 1:1 relationship from the Hospital E–R model, and Exercise 2.26 invites you to think about where the foreign key should be posted in this case.

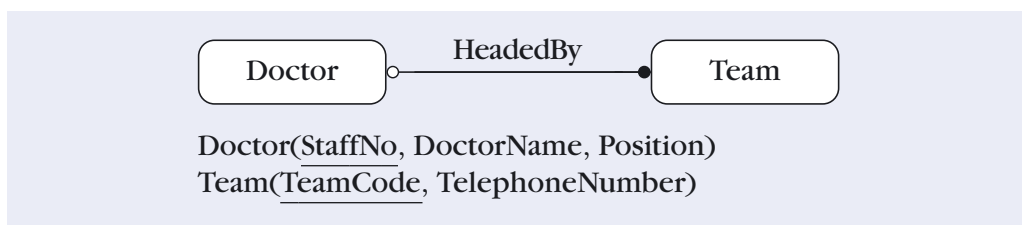


Figure 2.22 A 1:1 relationship from the Hospital E–R model



EXERCISE 2.26

With reference to Figure 2.22, where should the foreign key be posted? That is, which of the following sets of relational headings is allowable?

- (i) Doctor(StaffNo, DoctorName, Position, TeamCode)
Team(TeamCode, TelephoneNumber)
- (ii) Doctor(StaffNo, DoctorName, Position)
Team(TeamCode, TelephoneNumber, StaffNo)

Exercise 2.26 illustrates the general rule that a 1:1 relationship with optional participation at one end and mandatory at the other, is represented by a foreign key in the relation at the mandatory end. Note that if the relationship **HeadedBy** had mandatory participation at both ends, that is, if every team had a head and every doctor headed a team, then both of the pairs of relational headings in Exercise 2.26 would have been correct – you could have chosen to post the foreign key in either relation.

Suppose, however, that the relationship had optional participation at both ends, that is, some teams were not headed by a doctor and some doctors did not head teams. Then neither of the alternatives given would be allowable – some doctors would not be associated with a team and some teams would not be associated with a doctor. In cases such as this, we have to introduce a new relation to represent the relationship, as we shall see in Subsection 2.6.

We now consider how to represent the fact that a relationship is 1:1 in the declaration of a relation. Suppose we were to declare the two relations in Exercise 2.26(ii) in the following way:

relation Doctor

StaffNo: StaffNos

DoctorName: Names

Position: Positions

primary key StaffNo

relation Team

TeamCode: TeamCodes

TelephoneNumber: TelephoneNumbers

StaffNo: StaffNos

primary key TeamCode

{mandatory participation of Team in HeadedBy relationship}

foreign key StaffNo **references** Doctor

Given this declaration, there is nothing to stop a particular *StaffNo*, 111 say, occurring in many tuples of *Team* (for example, in both $\langle t01, 1234, 111 \rangle$ and $\langle t02, 5678, 111 \rangle$), contradicting the fact that **HeadedBy** is 1:1 – there is only one team associated with any doctor who heads a team. That is, any such doctor can only appear once in the table depicting *Team*, so *StaffNo* must be a key for *Team*. Since we have already chosen *TeamCode* to be the primary key for *Team*, *StaffNo* must be an alternate key.

The declaration of the relation *Team* thus becomes:

relation *Team*

TeamCode: *TeamCodes*

TelephoneNumber: *TelephoneNumbers*

StaffNo: *StaffNos*

primary key *TeamCode*

{*HeadedBy* is 1:1}

alternate key *StaffNo*

{*mandatory participation of Team in HeadedBy relationship*}

foreign key *StaffNo* **references** *Doctor*

You met alternate keys in Subsection 2.4 above.

EXERCISE 2.27

Consider the following fragment of a relational model. Derive the associated fragment of the E–R model (diagram and entity types).



relation *Enrolment*

StudentId: *StudentIds*

CourseCode: *CourseCodes*

EnrolmentDate: *Dates*

primary key (*StudentId*, *CourseCode*)

relation *Examination*

StudentId: *StudentIds*

CourseCode: *CourseCodes*

ExaminationLocation: *Locations*

Mark: *Percentages*

primary key (*StudentId*, *CourseCode*)

{*relationship Takes*}

foreign key (*StudentId*, *CourseCode*) **references** *Enrolment*

Some outstanding issues

The technique described above of relying on foreign keys to represent relationships without introducing any further relations does not work in all instances. We shall now explore the nature of those instances, before going on to consider in Subsection 2.6 how the issues they raise are resolved.

In the subsection above we discussed the problems that we would have if the participation of the entity types in the **HeadedBy** relationship was optional at both ends. We have the same problem with a 1:*n* relationship where the participation of the entity type at the :*n* end is optional. For example, suppose we had the following situation, where a ward can be empty and a patient can be an outpatient (that is, not assigned to a particular ward):

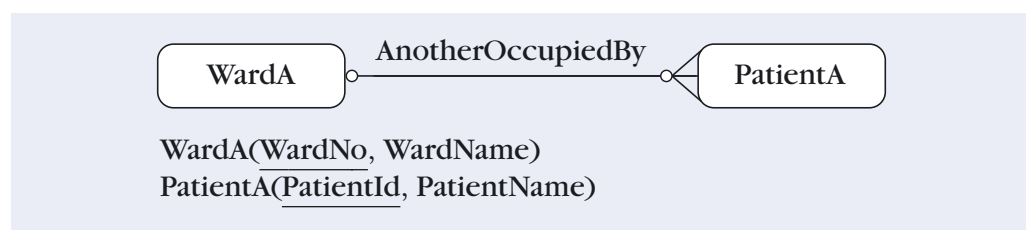


Figure 2.23 A 1: n relationship with optional participation at both ends

In this scenario, the relational headings

WardA(WardNo, WardName)

PatientA(PatientId, PatientName, WardNo)

would not be allowable, because not every patient would be associated with a ward. We shall see how to handle situations such as this in Subsection 2.6.

Another problem which we have not yet tackled is that of $m:n$ (many-to-many) relationships, for example, the **ExaminedBy** relationship below:

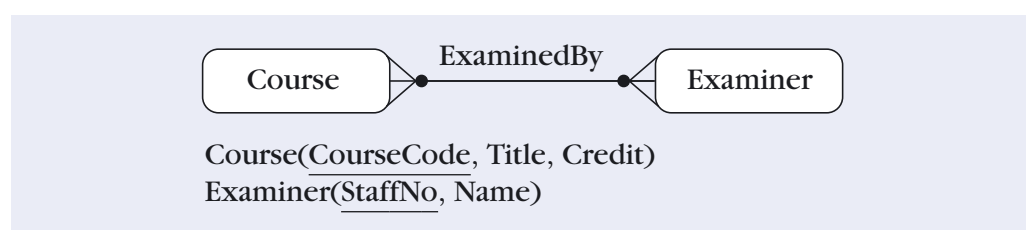


Figure 2.24 The $m:n$ relationship ExaminedBy

In this case, posting the foreign key in either relation is not allowable. Specifically, neither

Course(CourseCode, Title, Credit, StaffNo)

Examiner(StaffNo, Name)

nor

Course(CourseCode, Title, Credit)

Examiner(StaffNo, Name, CourseCode)

is allowable, since one course can be associated with many examiners, and one examiner with many courses.

A common student response in these circumstances is to hedge bets by posting foreign keys in both relations, as in:

Course(CourseCode, Title, Credit, StaffNo)

Examiner(StaffNo, Name, CourseCode)

But clearly this just compounds the problem of illegally having many values for a single attribute in a single tuple.

We need a different mechanism for representing relationships in order to address these outstanding issues. This method, which represents relationships by relations, is often referred to as the 'relation for relationship' mechanism, as we shall now discuss.

2.6 Representing relationships by relations

In this subsection we represent relationships by relations, using foreign keys to match the attribute values (rather than to represent the relationship).

For example, consider the 1:*n* relationship **AnotherOccupiedBy** illustrated in Figure 2.25. Since it has optional participation at both ends, it cannot be represented by posted foreign keys, as we have discussed. Suppose it has the following occurrences:

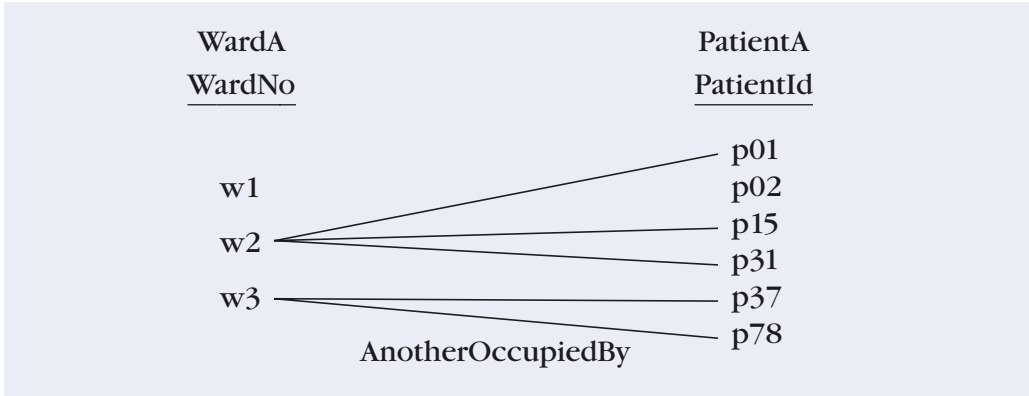


Figure 2.25 Some occurrences of the relationship **AnotherOccupiedBy**

These occurrences can be represented in a relational table as follows:

AnotherOccupiedBy

<i>WardNo</i>	<i>PatientId</i>
<i>w2</i>	<i>p01</i>
<i>w2</i>	<i>p15</i>
<i>w2</i>	<i>p31</i>
<i>w3</i>	<i>p37</i>
<i>w3</i>	<i>p78</i>

Figure 2.26 Table depicting *AnotherOccupiedBy*

From its appearance, this table might tempt you to declare the pair (*WardNo*, *PatientId*) to be the primary key of the relation, but you should resist that temptation. Since **AnotherOccupiedBy** is 1:*n* from **WardA** to **PatientA**, each patient is associated with a unique ward: the primary key is thus *PatientId*. The pair (*WardNo*, *PatientId*) fails the minimality criterion required for a primary key (see Subsection 2.4 above).

The full set of relations for this E–R fragment is given below:

```
relation WardA
  WardNo: WardNos
  WardName: WardNames
  primary key WardNo
relation PatientA
  PatientId: PatientIds
  PatientName: PatientNames
  primary key PatientId
```


relation *AnotherOccupiedBy*
PatientId: *PatientIds*
WardNo: *WardNos*
primary key *PatientId*
foreign key *PatientId* **references** *PatientA*
foreign key *WardNo* **references** *WardA*

We first mentioned referential integrity in Subsection 2.5, in the discussion following on from Figure 2.17.

Referential integrity means that any value of *PatientId* must be matched with one in *PatientA*, that is, that any patient who occurs in the table depicting *AnotherOccupiedBy* must also be in the table depicting *PatientA*, and similarly for *WardNo*. This is illustrated by the following occurrence diagram (Figure 2.27), which incorporates the relation *AnotherOccupiedBy* into Figure 2.26.

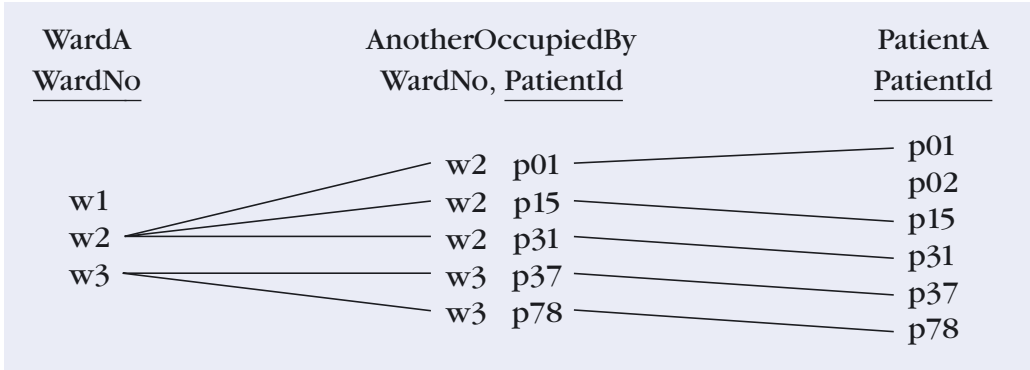


Figure 2.27 *AnotherOccupiedBy* as a relation

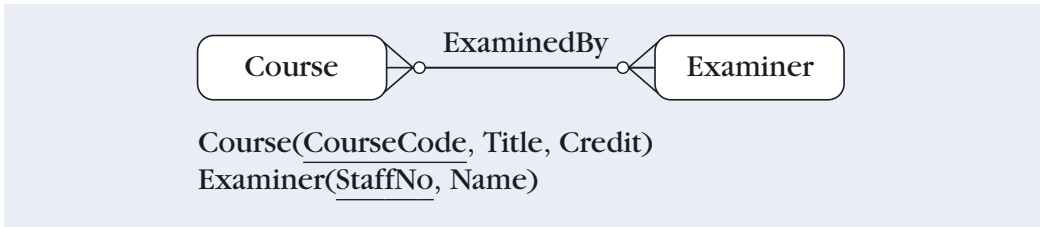


EXERCISE 2.28

Draw an E–R diagram showing the three relations *WardA*, *PatientA* and *AnotherOccupiedBy* above.

EXERCISE 2.29

Declare the relations corresponding to the fragment of an E–R model below.



EXERCISE 2.30

Draw the E–R diagram corresponding to the three relations identified in Solution 2.29.

The new relation which is introduced to represent an *m:n* relationship between entity types **A** and **B** has a special name: it is called an **intersection relation**. The intersection relation has as attributes only those of the primary keys of the relations representing **A** and **B**, which are also foreign keys referencing these relations. The primary key of the intersection relation is the combination of these primary keys.

EXERCISE 2.31

For each of the following fragments of relational representations, draw, if possible, two equivalent E–R diagrams: (a) one with three entity types **House**, **OwnsHouse** and **Person**, and (b) one with two entity types, **House** and **Person**. In each case, decide whether *OwnsHouse* is an intersection relation.

(i)

relation *House**Address*: *Addresses**WhenBuilt*: *Years***primary key** *Address***relation** *Person**Ref*: *NINumber**Name*: *Names***primary key** *Ref***relation** *OwnsHouse**Address*: *Addresses**Ref*: *NINumber***primary key** *Address***foreign key** *Address* **references** *House***foreign key** *Ref* **references** *Person*

(ii)

relation *House**Address*: *Addresses**WhenBuilt*: *Years***primary key** *Address***relation** *Person**Ref*: *NINumber**Name*: *Names***primary key** *Ref***relation** *OwnsHouse**Address*: *Addresses**Ref*: *NINumber**WhenLastSold*: *Years***primary key** (*Address*, *Ref*)**foreign key** *Address* **references** *House***foreign key** *Ref* **references** *Person*

(iii)

relation *House**Address*: *Addresses**WhenBuilt*: *Years***primary key** *Address***relation** *Person**Ref*: *NINumber**Name*: *Names***primary key** *Ref***relation** *OwnsHouse**Address*: *Addresses**Ref*: *NINumber***primary key** (*Address*, *Ref*)**foreign key** *Address* **references** *House***foreign key** *Ref* **references** *Person*

(iv)

relation *House**Address*: *Addresses**WhenBuilt*: *Years***primary key** *Address***relation** *Person**Ref*: *NINumber**Name*: *Names***primary key** *Ref***relation** *OwnsHouse**Address*: *Addresses**Ref*: *NINumber***primary key** *Address***alternate key** *Ref***foreign key** *Address* **references** *House***foreign key** *Ref* **references** *Person*



EXERCISE 2.32

Suppose *C* is a relation which exists solely in order to represent a relationship between entity types **A** and **B**.

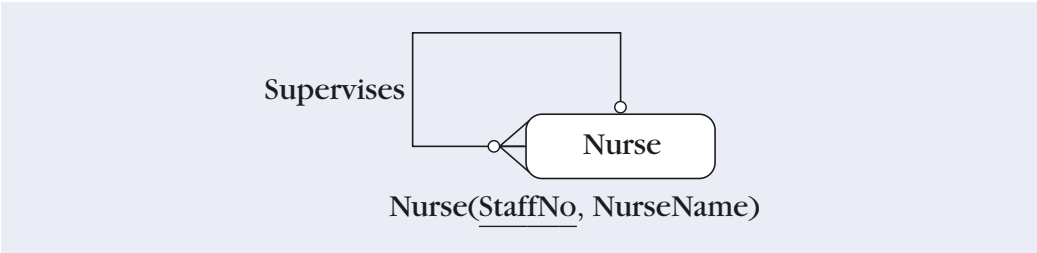
- (i) What are the attributes of *C*?
- (ii) Must the primary key of *C* always be a combination of the primary keys of the relations representing **A** and **B**?
- (iii) What do you know about the relationship if the primary key of *C* is a combination of the primary keys of the relations representing **A** and **B**?

The penultimate exercise in this section is an example of a **recursive relationship**, that is, a relationship which is between an entity type and itself.



EXERCISE 2.33

Declare the relation *Nurse* corresponding to the fragment of the Hospital conceptual data model shown below, where **Supervises** associates occurrences of the entity type **Nurse** with other occurrences (as in, for example, Nurse HighAndMighty supervises Nurse LowAndHumble).



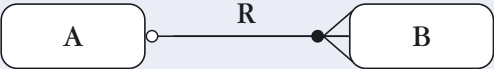
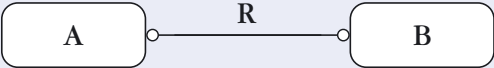
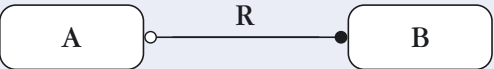
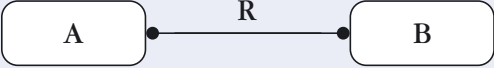
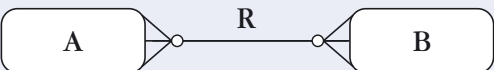
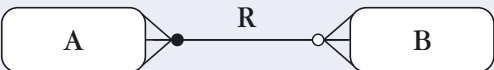
The final exercise of this section revises Subsections 2.5 and 2.6. You should note that we haven't yet considered how to represent some of the mandatory participation conditions – we will discuss this in Section 4.



EXERCISE 2.34

Fill in the gaps in the following table, where we have filled in the first row for you.

Relationship	Method of representing the relationship	Any aspect of the relationship not represented?
(i) 	Foreign key in the relation representing B	The mandatory participation of A in R
(ii) 		

Relationship	Method of representing the relationship	Any aspect of the relationship not represented?
(iii) 		
(iv) 		
(v) 		
(vi) 		
(vii) 		
(viii) 		

2.7 Summary

The context of this section is that we have analysed the structure of the data and the interrelationships between data items and we have produced a conceptual data model, which is an E–R model in this case. We have also taken the decision in this course that our database is going to be a relational one, that is, one based on relational theory. In this section, we have begun to discuss how the conceptual model can be represented relationally.

We have discussed how entities can be represented by *relations*, sets of *tuples*, where a *tuple* is a set of values, one from each *attribute*, drawn from the *domain* of that attribute. A *relation* may be depicted by a table with a particular set of properties. We saw in Subsection 2.1 that these properties are:

- 1 All values in the table must be atomic.
- 2 Every position in the table must have a value and all values in the same column must be of the same kind (from the same domain).
- 3 Each column has a unique name.
- 4 Each row is unique.
- 5 The ordering of rows and columns is not significant.

In Subsection 2.2, we stressed that values of attributes can only be compared if they are drawn from the same domain. For example, we might want to compare the values of the attribute *DateObtainedPilotLicence* with the values of the attribute *DateFlewPlane*, so we would have to ensure that these attributes were defined over the same domain.

In Subsection 2.3 we discussed how relations might be declared, though in subsequent subsections we saw how the basic declaration might be augmented by declarations of alternate and/or foreign keys. In Subsection 2.4, we considered *candidate keys*, which may be *primary* or *alternate keys*, and the *constraints* these place on the tuples of a relation. A particular value of a candidate key can only occur in a single tuple in any relation.

Representing a relationship between two entities in a relational representation is not as straightforward as in an E–R diagram, as we saw in Subsections 2.5 and 2.6. Relationships are fundamentally represented by matching values in foreign and primary keys. Depending on the context, this may or may not involve including another relation.

In the next section, we shall consider how new relations can be derived from old using a set of operators.

LEARNING OUTCOMES

Having studied this section, you should now:

- ▶ Be able to define the relational terms **relation**, **attribute**, **domain**, **tuple**, **key**, **primary key**, **foreign key**, **candidate key** and **alternate key**.
- ▶ Understand how a relation may be depicted by a table and be able to determine whether a given table may depict a relation.
- ▶ Understand how the definitions of domains and keys (both candidate and foreign) constrain data values.
- ▶ Understand and be able to apply two methods for representing relationally a relationship between entity types, by using foreign keys alone (which might be posted or pre-posted) or by the 'relation for relationship' method.
- ▶ Be able to identify when each of the methods for representing a relationship between entity types is applicable.

3

Manipulating relations

In the last section, we discussed the structure of relations in terms of attributes, domains and tuples. In this section, we focus on the operators of relational algebra which derive new relations from old.

If you are unfamiliar with operators

You will already have met operators in the context of arithmetic – although the term ‘operator’ would probably not have been used. Some operators in arithmetic are ‘+’ (add), ‘-’ (subtract), ‘×’ (multiply) and ‘/’ (divide). These all operate on two numbers to give another number. So, for example, $2 + 3 = 5$; $4 \times 7 = 28$. In these examples, 2 and 3 are the operands of ‘+’; 4 and 7 are the operands of ‘×’. We can talk of ‘+’ being applied to operands 2 and 3 to yield 5.

The ‘+’ operator is said to be closed on the set of whole numbers (integers), in the sense that adding one whole number to another gives a third whole number. The operator divide is **not** closed on the set of whole numbers; for example, 1 divided by 2, 0.5, is not a whole number. The operator ‘-’ (subtract or minus) is not closed on the set of positive whole numbers (think of, for example, $1 - 3$).

Terminology: an operator may be invoked, rather than applied.

The operators of relational algebra operate on one or more relations (their **operands**), *not* on individual tuples. The result of applying each operator to a relation (or to a pair of relations, depending on the kind of operator) is itself a relation, that is, the operator is **closed** on the set of relations. So the result of applying one of these operators to a relation can itself be acted on by an operator – the result can itself be an operand, as illustrated in Figure 3.1. We shall see further examples of this below.

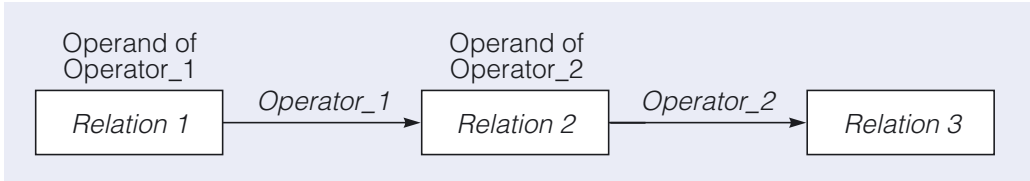


Figure 3.1 The closure property of relational operators. *Operator_1* applied to *Relation 1* yields *Relation 2*, and *Operator_2* applied to *Relation 2* yields *Relation 3*.

Remember that in the context of this block, we are still in the theoretical world. So although most of these operators have direct counterparts in data manipulation languages based on relational principles, such as SQL, not all of them do. Even where the operators are directly implemented, the implementations may not match exactly with their theoretical counterparts. For example, some operators are implemented in SQL so that they may take relational tables as their operands but yield a table which doesn’t represent a relation (because, for example, it contains repeated rows or columns). These issues will be discussed in more depth in *Block 3*.

We shall now look at the relational operators in more detail.

3.1 The select and project operators

The relational operators **select** and **project** are both **unary** operators, which means that they each act on a single relation – have just one relation as the operand.

The select operator

The **select** operator can be thought of as slicing a relation horizontally, picking out those tuples which satisfy a particular condition, called a **selection condition**.

For example, consider the depiction of the *Enrolment* relation that you met in Section 2, which is repeated below in Figure 3.2.

Enrolment

<i>StudentId</i>	<i>CourseCode</i>	<i>EnrolmentDate</i>
s01	c4	Jan 12, 2005
s02	c5	Jan 1, 2005
s02	c7	Jun 12, 2005
s05	c2	Jun 4, 2004
s05	c7	Oct 18, 2004
s07	c4	Dec 12, 2004
s09	c4	Dec 16, 2004
s09	c2	Dec 18, 2004
s09	c7	Dec 15, 2004
s10	c7	Jun 20, 2004
s10	c4	May 5, 2004
s22	c2	Mar 15, 2002
s38	c2	Sep 18, 2003
s38	c5	Mar 9, 2004
s46	c2	Mar 1, 2002
s57	c4	Jun 30, 2001
s57	c5	Jan 20, 2003

Figure 3.2 The *Enrolment* relation again

Now consider the following expression:

select *Enrolment* **where** *CourseCode* = 'c4'

When evaluated, this expression results in the relation represented by the following table:

<i>StudentId</i>	<i>CourseCode</i>	<i>EnrolmentDate</i>
s01	c4	Jan 12, 2005
s07	c4	Dec 12, 2004
s09	c4	Dec 16, 2004
s10	c4	May 5, 2004
s57	c4	Jun 30, 2001

Because SQL uses the term 'select' in a different sense from the way in which it is used here, some writers prefer to give this operator another name, such as 'restrict'.

Later we shall see that the **project** operator may be thought of as slicing a relation vertically.

Here, 'and' is being used in its logical sense. For any propositions A and B, 'A and B' is true if and only if both A and B are true. 'A or B' is true if A is true or B is true or both are true. It is false only if both are false.



As you might expect, where D and E are dates, 'D > E' is true if and only if D comes after (is later than) E.

The natural language predicate for this relation is
<a, 'c4', b> is a tuple of this relation if and only if the student with *StudentId* 'a' enrolled on the course with *CourseCode* c4 on *EnrolmentDate* 'b'.

The general form of a **select** expression is

select <relation> **where** <selection condition>

A **selection condition** consists of a **Boolean expression**, that is, an expression which is either true or false. The selection condition is applied to each tuple of the relation in turn: tuples for which the given Boolean expression is true are retained; those for which it is false are discarded. For example, referring to Figure 3.2, the condition (*StudentId* = 's05' **and** *CourseCode* = 'c2') is true for the tuple <s05, c2, 'Jun 4, 2004'> and false for the tuples <s05, c7, 'Oct 18, 2004'> and <s09, c2, 'Dec 18, 2004'>.

Boolean expressions frequently make use of the comparison operators: '=', '<', '>' and '<>', where '<>' means 'not equal to'. The values on either side of the operator, the operands, must come from the same domain to enable comparison.

EXERCISE 3.1

Write down a table representing the relation which results from the evaluation of the following expression, where *Enrolment* is as in Figure 3.2:

select *Enrolment* **where** *EnrolmentDate* > 'June 1, 2004' **and** *EnrolmentDate* < 'Nov 1, 2004'

EXERCISE 3.2

Write down an expression to select all those students who enrolled either before September 1, 2004, or after January 1, 2005.

EXERCISE 3.3

In Subsection 2.4, you met the relation *GeneralPractitioner* with heading *GeneralPractitioner*(*GPId*, *GPName*, *SecId*, *SecName*). Write an expression to find all those GPs who have the same name as their secretary.

EXERCISE 3.4

In a selection condition, what constraints apply to the operands of the comparison operators? Given Solution 3.3, what implication does this have for the declaration of the relation *GeneralPractitioner*?

The project operator

Just as the **select** operator can be thought of as slicing a relation horizontally, picking out those tuples which satisfy a particular condition, the **project** operator can be thought of as slicing a relation vertically, picking out those attributes which are of interest.

The general form of a **project** expression is

project <relation> **over** <attribute list>

Consider, for example, the following expression:

project *Enrolment* **over** *StudentId*, *CourseCode*

When applied to the relation of Figure 3.2, this expression will give the following relation:

<i>StudentId</i>	<i>CourseCode</i>
<i>s01</i>	<i>c4</i>
<i>s02</i>	<i>c5</i>
<i>s02</i>	<i>c7</i>
<i>s05</i>	<i>c2</i>
<i>s05</i>	<i>c7</i>
<i>s07</i>	<i>c4</i>
<i>s09</i>	<i>c4</i>
<i>s09</i>	<i>c2</i>
<i>s09</i>	<i>c7</i>
<i>s10</i>	<i>c7</i>
<i>s10</i>	<i>c4</i>
<i>s22</i>	<i>c2</i>
<i>s38</i>	<i>c2</i>
<i>s38</i>	<i>c5</i>
<i>s46</i>	<i>c2</i>
<i>s57</i>	<i>c4</i>
<i>s57</i>	<i>c5</i>

EXERCISE 3.5

Write down a table representing the relation which results from the evaluation of the expression:

project *Enrolment* **over** *StudentId*

EXERCISE 3.6

Why can't the solution to Exercise 3.5 have duplicate rows?



Combining expressions

In order to study the effect of more complex expressions, we need to introduce more data. Figure 3.3 depicts the relation *Student*, which you met in Section 2, with some sample data.

Note that in this section we won't be particularly concerned with consideration of primary keys.

Student

<i>StudentId</i>	<i>Name</i>	<i>Address</i>	<i>EmailAddress</i>	<i>RegistrationDate</i>	<i>RegionNumber</i>
s01	Akeroyd	12 Anystreet, Anytown	Akers@tahoo.fake.com	Nov 23, 1999	3
s02	Thompson	8 High Street, Lowville	Pjay@thompson.fake.com	Oct 12, 2004	4
s05	Ellis	34 Globe Road, Smallville	G.Ellis@fake.fake.co.uk	Oct 14, 2003	4
s07	Gillies	29 Straight street, Angletown	Gillies@address.fake.net	Dec 2, 1993	3
s09	Reeves	34 The Crescent, Curville	T.Reeves@nnet.fake.com	Dec 14, 2004	4
s10	Urbach	22 Hilltops, Valley Town	B.Urbach@tnet.fake.fr	May 5, 2003	4
s22	Bryant	84 Brook Street, Little Hacking	A.Bryant@greenmail.fake. uk	Jun 21, 2000	1
s38	Patel	12 Stanley Road, Pitchford	patel122@mailman.fake. uk	Oct 8, 2001	1
s42	Reddick	23 Kestrel Lane, Dudley	dave@belwise.fake.co.uk	Apr 23, 2002	2
s46	Sharp	The Farm, Lower Watley	F.Sharp@bluesky.fake.ac. uk	Feb 14, 2002	2
s57	Patel	4 Lower Crescent, Cinderfield	r.patel@tahoo.fake.com	Nov 5, 2000	4

Figure 3.3 The *Student* relation depicted as a table

We mentioned in the introduction of this section that the closure property of relational algebra operators means that we can use the result of applying one operator as the operand to another.

For example, suppose we want the names of all students in region 4. Applying the **select** operator to *Student* will give us lots of information about the students in region 4: applying **project** will give us the particular bit of information we want. The following expression will do the job:

project (select *Student* where *RegionNumber* = '4') over *Name*

Evaluating such combined expressions is done in an 'inside out' manner, that is, the expression enclosed in the parentheses (or in the innermost pair of parentheses, if there are multiple sets) is evaluated first, as in Figure 3.4.



EXERCISE 3.7

Why won't the following expression evaluate to the answer we want?

select (project *Student* over *Name*) where *RegionNumber* = '4'

EXERCISE 3.8

Write two equivalent relational expressions which will evaluate to give the name, address and registration date of each student who registered after 1 January 2004.

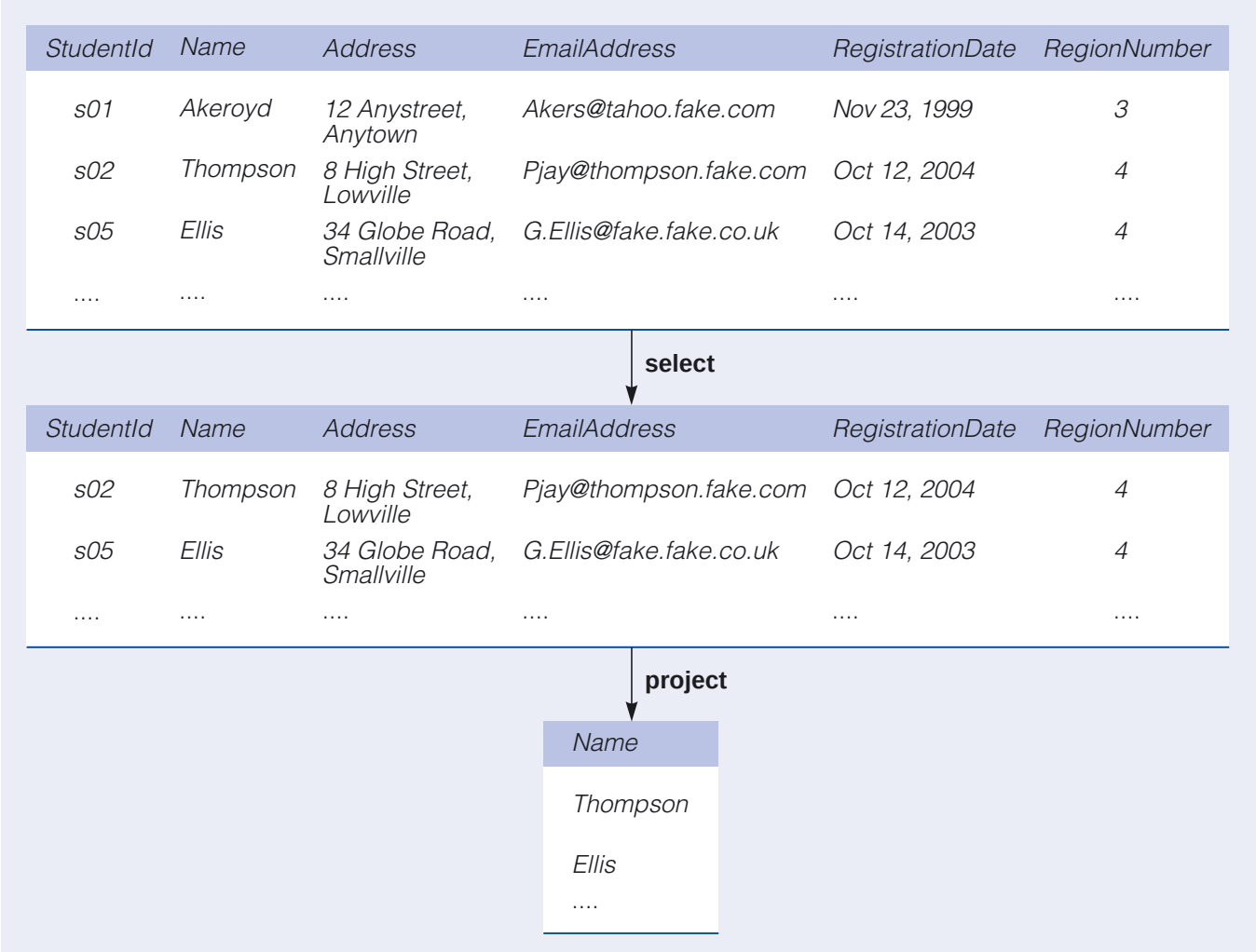


Figure 3.4 Example of combining expressions

We should remark that the order in which you choose to apply the operators in situations such as Exercise 3.8 is irrelevant in a theoretical world. In the real world, where execution time is an issue, order may be relevant. We shall say a little more about this later.

3.2 The join and rename operators

The join operator

The next operator that we will look at is **join**, which is a **binary** operator; this means that it has two relations as operands. There are several forms of the **join** operator, but we shall only consider the **natural join** which we shall simply call the **join**.

We shall illustrate **join** by means of an example. Figure 3.5 shows part of the table depicting the relation resulting from joining *Enrolment*(*StudentId*, *CourseCode*, *EnrolmentDate*) as in Figure 3.2, with *Student*(*StudentId*, *Name*, *Address*, *EmailAddress*, *RegistrationDate*, *RegionNumber*) as in Figure 3.3. Here we are joining the information recorded in the *Enrolment* relation about a particular student with that recorded in the *Student* relation.

Enrolment join Student

StudentId	CourseCode	EnrolmentDate	Name	Address	EmailAddress	RegistrationDate	RegionNumber
s01	c4	Jan 12, 2005	Akeroyd	12...	Akers@...	Nov...	3
s02	c5	Jan 1, 2005	Thompson	8...	Pjay@...	Oct...	4
s02	c7	Jun 12, 2005	Thompson	8...	Pjay@...	Oct...	4
s05	c2	Jun 4, 2004	Ellis	34...	G.Ellis@...	Oct...	4
...	...	...	...	...	...	...	...

Figure 3.5 Part of a table depicting the join of *Enrolment* and *Student*. Some data has been omitted for reasons of space.

You might recall from Subsection 2.1, that the relation *Enrolment* consists of the set of propositions ‘A particular student enrolls on a particular course on a date’, and *Student*, the set ‘A particular student has a name, address ...’. *Enrolment join Student* consists of the set ‘A particular student enrolls on a particular course on a date *and* has name, address ...’.

Figure 3.6 illustrates the joining together of the relational headings; Figure 3.7 shows the joining together of a pair of typical tuples.

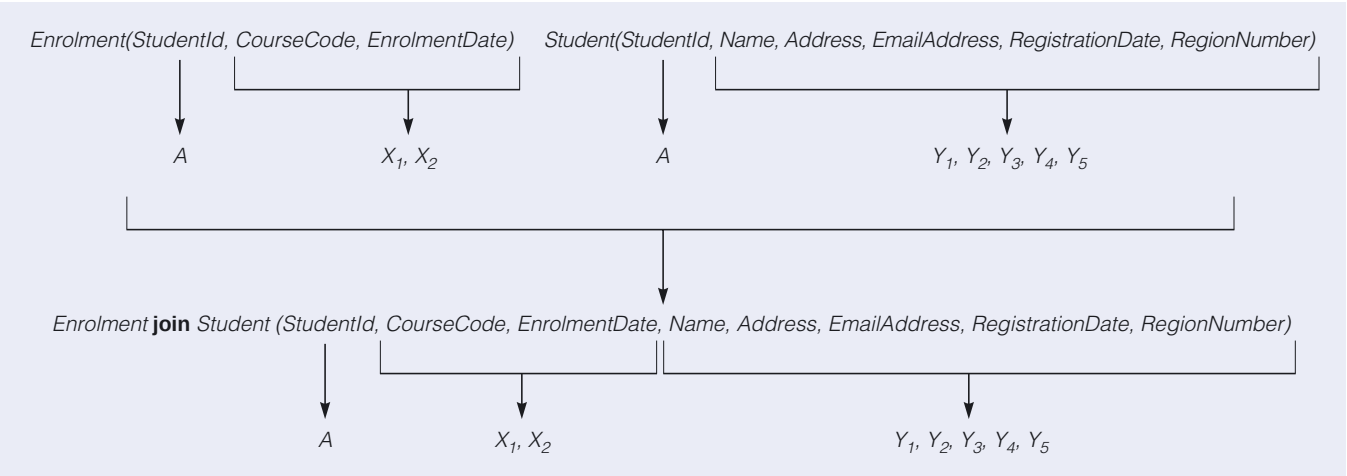


Figure 3.6 Joining together the headings of *Enrolment* and *Student*

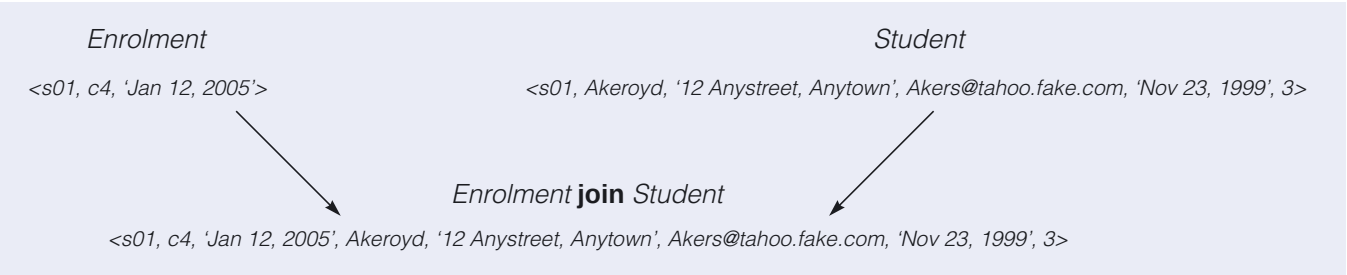


Figure 3.7 Joining together tuples of *Enrolment* and *Student*

The definition of **join** is as follows:

Suppose we have two relations *R1* and *R2* which have a set of attributes {*A*₁, *A*₂, ...} in common (in the same way that *StudentId* is a common attribute of the relations *Student* and *Enrolment*). That is, the heading of *R1* is *R1*(*A*₁, *A*₂, ..., *X*₁, *X*₂, ...) and the heading of *R2* is *R2*(*A*₁, *A*₂, ..., *Y*₁, *Y*₂, ...). None of the attributes *X*₁, *X*₂, ... are the same as any of the attributes *Y*₁, *Y*₂, ... – they have different names and/or different domains.

Then *R1 join R2* is the relation with heading consisting of the set of attributes {*A*₁, *A*₂, ..., *X*₁, *X*₂, ..., *Y*₁, *Y*₂, ...} and body consisting of the set of tuples of the form <*a*₁, *a*₂, ..., *x*₁, *x*₂, ..., *y*₁, *y*₂, ...>, where <*a*₁, *a*₂, ..., *x*₁, *x*₂, ...> is a tuple of *R1* and <*a*₁, *a*₂, ..., *y*₁, *y*₂, ...> is a tuple of *R2*. (*a*₁, *a*₂, ... are values of *A*₁, *A*₂, ..., respectively; *x*₁, *x*₂, ..., are values of *X*₁, *X*₂, ...; and *y*₁, *y*₂, ..., are values of *Y*₁, *Y*₂, ...)

Remember that the order in which we write the attributes of a relation is of no consequence. The attributes in common need not be written first in the list of attributes.

You might remember from Subsection 2.5 (on pre-posted foreign keys) that the relationship **EnrolledIn** between **Student** and **Enrolment** is represented by the foreign key *StudentId* in *Enrolment* matching values of the primary key *StudentId* of *Student*. Figure 3.7 explicitly shows this matching – exactly which tuple in *Student* is matched with which tuple in *Enrolment*.

EXERCISE 3.9

Depict the relation *SmallEnrolment join Examination* in a table, where *SmallEnrolment* has the same heading as *Enrolment* and body as depicted below. The heading of *Examination* is as shown in Exercise 2.27, and the body is depicted below.



SmallEnrolment

<i>StudentId</i>	<i>CourseCode</i>	<i>EnrolmentDate</i>
<i>s05</i>	<i>c2</i>	<i>Jun 4, 2004</i>
<i>s05</i>	<i>c7</i>	<i>Oct 18, 2004</i>
<i>s07</i>	<i>c4</i>	<i>Dec 12, 2004</i>
<i>s09</i>	<i>c4</i>	<i>Dec 16, 2004</i>
<i>s09</i>	<i>c2</i>	<i>Dec 18, 2004</i>

Examination

<i>StudentId</i>	<i>CourseCode</i>	<i>ExaminationLocation</i>	<i>Mark</i>
<i>s07</i>	<i>c4</i>	<i>Bedford</i>	<i>85</i>
<i>s09</i>	<i>c4</i>	<i>Taunton</i>	<i>63</i>
<i>s10</i>	<i>c4</i>	<i>Gateshead</i>	<i>27</i>
<i>s05</i>	<i>c2</i>	<i>Bath</i>	<i>57</i>
<i>s09</i>	<i>c2</i>	<i>New York</i>	<i>56</i>
<i>s09</i>	<i>c7</i>	<i>Taunton</i>	<i>71</i>

There are some problems associated with **join**, which we haven't yet addressed. For example, in Subsection 2.5, we discussed a relation *Region*(*RegionNumber*, *Address*, *Telephone*, *EmailAddress*) and pointed out that the *RegionNumber* attribute in

Student(*StudentId*, *Name*, *Address*, *EmailAddress*, *RegistrationDate*, *RegionNumber*) is a foreign key representing the relationship **Manages** between **Region** and **Student**. We should like it to be possible to associate the details of each student with the details of that region which manages the student using the operator **join** as we did for the relationship **EnrolledIn** in Figure 3.5. Exercise 3.10 explores the consequences of doing this. In the interests of brevity, we introduce a new relation, *SmallRegion*.



EXERCISE 3.10

Figure 3.8 shows the body of a relation *SmallRegion* having the same heading as *Region*.

SmallRegion

<i>RegionNumber</i>	<i>Address</i>	<i>Telephone</i>	<i>EmailAddress</i>
3	Block 9, The Campus, Walton Hill	01670 245365	region3@open.fake.address
12	The Office, New York	10898 227191	region12@open.fake.address

Figure 3.8 The relation *SmallRegion*

Write down a table depicting the relation *Student join SmallRegion*.

The problem in Exercise 3.10 is that we don't want *Address* and *EmailAddress* to be common attributes – neither of these play any part in representing the **Manages** relationship. We can fix the problem by renaming these attributes using a **rename** operator.

The **rename** operator

This is a unary operator which operates on a single relation to return a relation identical to the original except that an attribute name is changed.

Suppose we have a relation *R*, with a set of attributes which includes named attributes *A1*, *A2*, ... which we wish to rename as *NewA1*, *NewA2*, ..., respectively. Then the following relational expression does the job:

R rename (*A1 as NewA1*, *A2 as NewA2*, ...)



EXERCISE 3.11

- (a) Fix the problem identified in Exercise 3.10. That is, write down a relational expression which does yield a relation associating the details of each student with details of the region in *SmallRegion* managing that student.
- (b) Write down a table depicting the relation yielded by the relational expression in (a).

Combining expressions and the use of **alias**

We are now in a position to write arbitrarily complicated relational algebra expressions. In order to do this, you might find it helpful to structure your expression as in the following exercise.

Suppose you are asked to find a relational expression which will give you the names of all the students from region 3 who are taking an examination in Bedford.

EXERCISE 3.12

Which three relations will you need to use in order to find this information? Use the relations as given in the University model (i.e. *Enrolment* rather than *SmallEnrolment*).

In a similar fashion to Exercise 3.9, we can form a relation which associates corresponding tuples in *Enrolment* and *Examination*, that is, *Enrolment* **join** *Examination*. Because we don't want to keep on typing out this expression, we shall give it an **alias** – a temporary name or placeholder – as in the following expression:

ExamAndEnrolDetails **alias** (*Enrolment* **join** *Examination*)

The heading of *ExamAndEnrolDetails*, as in Solution 3.9, is *ExamAndEnrolDetails* (*StudentId*, *CourseCode*, *ExaminationLocation*, *Mark*, *EnrolmentDate*) (where we have ignored any consideration of primary keys).

To complete the task set at the start of this subsection, we now need to:

- (i) Derive a relation associating each student tuple with the corresponding tuple in *ExamAndEnrolDetails* so as to link each student with the relevant enrolment and examination information. We call this relation *StudentExamAndEnrolDetails* and write down its heading.
- (ii) Derive a relation from *StudentExamAndEnrolDetails* which gives the required information (that is, the names of all students from region 3 who are taking an exam in Bedford).
- (iii) Substitute back for *StudentExamAndEnrolDetails* and *ExamAndEnrolDetails* (that is, replace each alias by the original relational expression).

EXERCISE 3.13

Complete the three steps above.

EXERCISE 3.14

Find a relation which gives the titles of the courses studied by students in region 2. Start by selecting those students who are in region 2.

You may find the relevant fragment of the E–R diagram helpful, as shown in Figure 3.9.

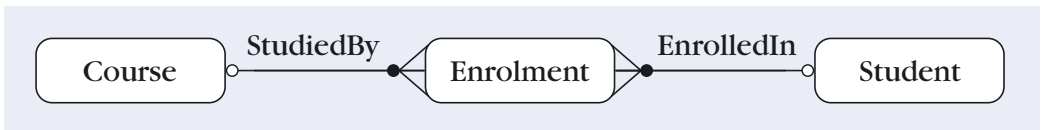


Figure 3.9 Relationships between Course, Enrolment and Student

The following relational heading may be helpful: *Course*(*CourseCode*, *Title*, *Credit*), and you may also find it useful to use aliases as placeholders.



You might find the *Relational headings summary card* useful here.

Recall that the order in which we write the attributes of a relation is unimportant.



We ask you to start in this way simply to cut down the number of different correct answers. The next exercise requires an alternative starting point for the same task.



EXERCISE 3.15

Derive a relation to give the titles of the courses studied by students in region 2. Start by joining all the relevant relations.

A brief note on optimisation

Although this block is strongly geared towards theory, this is an appropriate point to mention briefly one way in which theory might underpin implementation.

For any database management system managing potentially large amounts of data, there is a need to be able to handle user requests in a reasonable timescale. One important component of a relational DBMS is an **optimiser**, which, given a user request, chooses an efficient way to implement that request. We have now seen examples in which different relational algebra expressions are equivalent in that they meet the same request for data. This property of relational algebra provides a relational DBMS with a powerful tool, in that given a user request, the optimiser can appraise all the equivalent relational algebra expressions which meet that request and choose the one which can be implemented most efficiently.

Joining a relation with itself

We saw an example of a recursive relationship, **Supervises** over the entity type **Nurse**, in Exercise 2.33. Figure 3.10 illustrates another recursive relationship, **Appraises** over the entity type **Doctor**. Doctors can appraise 0, 1 or more of their colleagues: every doctor must have an appraiser.

Note that we have introduced a relationship here which does not appear in the standard Hospital model: the *Doctor* relation here is thus slightly different from that of the standard model, both in heading and body.

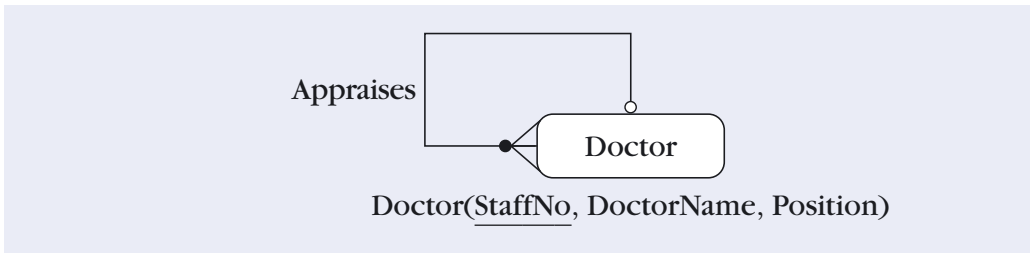


Figure 3.10 The relationship **Appraises**

The relationship **Appraises** may be represented by a foreign key *Appraiser* in *Doctor* referencing *Doctor* so giving the heading of *Doctor* as

Doctor(StaffNo, DoctorName, Position, Appraiser)

Figure 3.11 illustrates this relation.

Doctor

StaffNo	DoctorName	Position	Appraiser
110	Liversage	Consultant	131
131	Kalsi	Consultant	110
156	Hollis	Registrar	110
174	Gibson	Registrar	110
178	Paxton	Registrar	131
389	Wright	House Officer	131

Figure 3.11 The relation *Doctor*

EXERCISE 3.16

Write a relational expression to find the details of all the doctors who are appraised by a doctor called Liversage.

Hint: find the staff number(s) of Liversage, and then find all the doctors which have this number (or numbers) as their value of the attribute *Appraiser*. Remember the **rename** operator.



EXERCISE 3.17

Derive a relation which associates the details of each doctor with the name of their appraiser, as illustrated below.

<i>StaffNo</i>	<i>DoctorName</i>	<i>Position</i>	<i>Appraiser</i>	<i>AppName</i>
110	Liversage	Consultant	131	Kalsi
131	Kalsi	Consultant	110	Liversage
156	Hollis	Registrar	110	Liversage
174	Gibson	Registrar	110	Liversage
178	Paxton	Registrar	131	Kalsi
389	Wright	House Officer	131	Kalsi

Hint for one particular solution:

- (i) find the names and staff numbers of potential appraisers (all the doctors);
- (ii) rename the attributes, in order to ...
- (iii) join the details of each doctor with the name of their appraiser.

3.3 The divide operator

Like **join**, **divide** is a binary operator requiring two operands.

We have seen how we can use the **select** and **project** operators to yield all the students enrolled on a particular course, or all the courses taken by a particular student. But suppose we want to find details of those students who are enrolled on *all* the available courses, or those courses on which *all* the known students are enrolled. In this case, we need the operator **divide**.

We shall illustrate **divide** by an example. For this example, we consider the relation, *SupplyParts* which is the set of tuples $\langle s, p \rangle$ where the supplier identified by s supplies the part identified by p . *SupplyParts* is depicted in Figure 3.12.

SupplyParts

SupplierId	PartId
s1	p4
s5	p2
s5	p7
s7	p4
s9	p4
s9	p2
s9	p7
s10	p7
s10	p4

Figure 3.12 The relation SupplyParts



EXERCISE 3.18 (Revision)

Write a relational expression to derive a relation called AllParts from SupplyParts which yields all the known values of PartId.

A table depicting AllParts is as below:

AllParts

PartId
p4
p2
p7

The expression

divide SupplyParts **by** AllParts

yields the relation depicted by the following table:

SupplierId
s9

That is, the expression yields a list of suppliers who supply all the known parts. In the event, there is only one such supplier.

The general form of a **divide** expression is:

divide <Relation1> **by** <Relation2>

Here, the attributes of Relation2 must be a subset of the attributes of Relation1 (that is, each attribute of Relation2 has the same name and is defined over the same domain as an attribute of Relation1). The **divide** expression evaluates to a relation having all those attributes which are in Relation1 but not in Relation2. So, in the example above, Relation1 (SupplyParts) has attributes SupplierId, PartId, and Relation2 (AllParts) has attribute PartId, and the result of evaluating the given expression has attribute SupplierId.

EXERCISE 3.19

Write a relational expression to find the identifiers of the parts which are supplied by all suppliers.



EXERCISE 3.20

Write a relational expression derived from the relation *Enrolment* to find the identifiers of students who are enrolled on all known courses.

Hint: it might be helpful if you tackle this exercise in stages, using aliases, and then substitute back. The first stage might be to derive the course and student identifier data from *Enrolment*; the second, to derive a relation of all the courses; the third, to apply the **divide** operator.

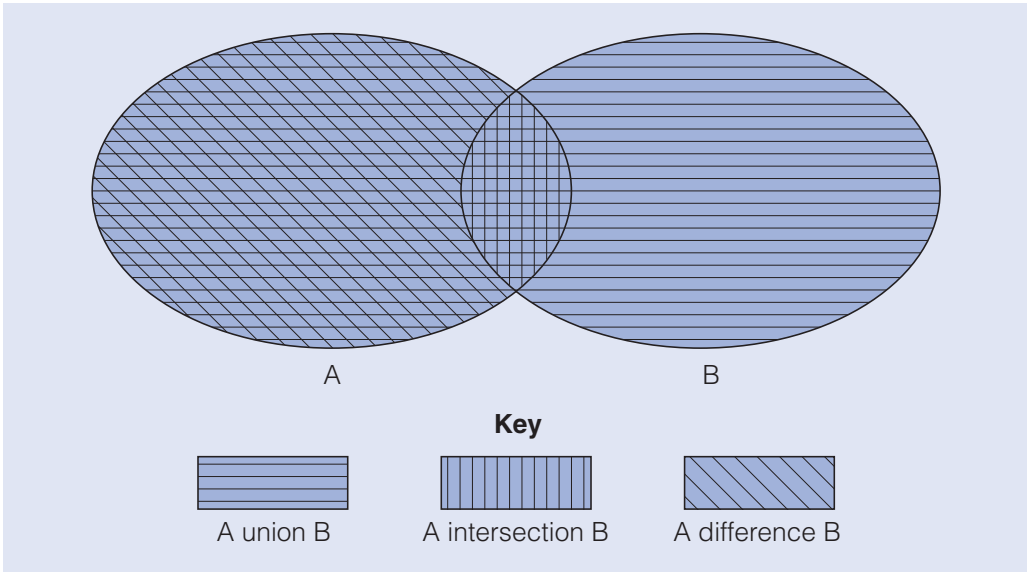
3.4 Set operators union, intersection and difference

The operators we have seen so far – **select**, **project**, **join** and **divide** – are specific to relational algebra. But a relation is a set of tuples, so it seems entirely reasonable to apply the usual set operators to relations, *provided* that these operators have the property of closure, that is, provided that the result of applying such an operator to relations is itself a relation. The set operators that we are particularly interested in are **union**, **intersection** and **difference**.

You might already know their definitions from set theory:

- ▶ The union of two sets A and B is the set consisting of all elements which are either in A or in B or both (with no repeated elements).
- ▶ The intersection of A and B is the set consisting of all those elements which are in both A and B.
- ▶ A difference B is all those elements in A which are not in B.

These are illustrated in Figure 3.13 below.



This type of diagram, as you may already know, is known as a Venn diagram, after the English mathematician John Venn (1834–1923).

Figure 3.13 Venn diagram illustrating the set operators union, intersection and difference

The following exercise (overleaf) provides some practice in using these set operators.



EXERCISE 3.21

- (i) If $A = \{1, 3, 5, 7, 8, 9\}$ and $B = \{1, 2, 3, 4, 5\}$, what are the sets A union B , A intersection B , and A difference B ?
- (ii) What do you know about the relationship between arbitrary sets C and D if C difference D is empty?
- (iii) For arbitrary sets A and B , is A union B the same as B union A , is A intersection B the same as B intersection A , and is A difference B the same as B difference A ?

We have emphasised that we want all operators on relations to have the closure property. This means that we can't take the set of tuples comprising the body of one relation and form a union with the set of tuples comprising the body of any other arbitrary relation – the resulting union is unlikely to be a relation (for example, what would its heading be?). Instead, we insist that the operands to the relational operators **union**, **intersection** and **difference** are **union-compatible**, by which we mean that they have the same set of attributes – that is, they have the same number of attributes and each attribute in one of the operands has the same name and is defined over the same domain as an attribute in the other.

In order to achieve union-compatibility, we may have to use the **rename** operator in the situation where an attribute in one operand is defined over the same domain as an attribute in the other, but has a different name. For example, suppose the two attributes *EnrolmentDate* and *RegistrationDate* are defined over the same domain. Then the two relations

project *Enrolment* **over** *StudentId, EnrolmentDate*

and

project *Student* **over** *StudentId, RegistrationDate*

are not union-compatible but can be very easily be made so by judicious use of **rename**, as in:

project (*Enrolment* **rename** (*EnrolmentDate* **as** *Date*)) **over** *StudentId, Date*

project (*Student* **rename** (*RegistrationDate* **as** *Date*)) **over** *StudentId, Date*

The general form of the **union**, **intersection** and **difference** operators

The general form for each of these operators is very similar:

$\langle \text{Relation1} \rangle$ **union** $\langle \text{Relation2} \rangle$

$\langle \text{Relation1} \rangle$ **intersection** $\langle \text{Relation2} \rangle$

$\langle \text{Relation1} \rangle$ **difference** $\langle \text{Relation2} \rangle$

where, of course, $\langle \text{Relation1} \rangle$ and $\langle \text{Relation2} \rangle$ are union-compatible.

Whenever you are asked to derive a relation where the tuples come from *either this or that* relation, then you should think about applying **union**. When you are asked to derive a relation where each of the tuples is in *both this and that* relation, you should think of **intersection**. When you are asked to find a relation whose tuples come from *this but not that* relation, you should think of **difference**.

For example, suppose you are asked to find the names and email addresses of people who are both members of staff and students (on different courses from those on which they teach, naturally – for example, an associate lecturer on M359 could be taking a course in Art History). You can assume that these people can be identified by their

names and addresses. Given that the heading for the relation *Staff* is *Staff* (*StaffNumber*, *Name*, *Address*, *EmailAddress*, *Telephone*, *RegionNumber*), and given that, in our standard University relational representation, attributes with the same name in *Staff* and *Student* have the same domain, then we can project over *Name*, *Address* to yield a set of tuples which we can intersect with a similar project of *Student* (as in Figure 3.3) giving:

(**project** *Student* **over** *Name*, *Address*) **intersection** (**project** *Staff* **over** *Name*, *Address*)

It is important to note that attributes with the same name do not necessarily have the same domain.

EXERCISE 3.22

Is the following relation equivalent to the relational algebra expression given above?

project (*Student* **intersection** *Staff*) **over** *Name*, *Address*



EXERCISE 3.23

- (i) Form two equivalent relational algebra expressions to find the identifiers of all those students who are either in region 3 or enrolled on course c3 or both. The first expression should involve a set operator; the second expression should not.
- (ii) Form a relational algebra expression which lists the staff numbers of all those doctors who are not appraisers (see Figure 3.10).

EXERCISE 3.24

- (i) Suppose *A* and *B* are the relations depicted by the tables below, where attributes with the same name are defined over the same domain. The natural language predicates of *A* and *B* are the following: *A* tuple <*a*, *b*> belongs to *A* if student '*a*' enrolled on some course on date '*b*'; a tuple <*c*, *d*> belongs to *B* if student '*c*' registered with the University on date '*d*'.

Write down tables depicting *A* **join** *B* and *A* **intersection** *B*.

A

<i>StudentId</i>	<i>Date</i>
<i>Ashwin</i>	<i>Jan 12, 2005</i>
<i>Ashwin</i>	<i>Feb 2, 2005</i>
<i>Beryl</i>	<i>Jun 12, 2005</i>
<i>Beryl</i>	<i>Oct 4, 2005</i>
<i>Carol</i>	<i>Oct 18, 2005</i>
<i>Dave</i>	<i>Dec 12, 2005</i>

B

<i>StudentId</i>	<i>Date</i>
<i>Ashwin</i>	<i>Jan 12, 2005</i>
<i>Beryl</i>	<i>Jun 12, 2005</i>
<i>Carol</i>	<i>Oct 18, 2005</i>
<i>Dave</i>	<i>Dec 12, 2005</i>

- (ii) Suppose *R* and *T* are any two union-compatible relations. What is the connection between *R* **join** *T* and *R* **intersection** *T*?

3.5

The **times** operator

To give you a richer picture of the relational algebra, we shall now briefly describe the operator **times**.

Cartesian product and the **times** operator

You might know that the Cartesian product of two sets A and B , written $A \times B$, consists of the set of all pairs (a, b) where a is an element of A and b is an element of B (and the order of the elements in each pair is important). So if A is the set $\{1, 2, 3\}$ and B is the set $\{4, 5\}$, the Cartesian product $A \times B$ is the set

$\{(1, 4), (1, 5), (2, 4), (2, 5), (3, 4), (3, 5)\}$,

whereas $B \times A$ is the set

$\{(4, 1), (4, 2), (4, 3), (5, 1), (5, 2), (5, 3)\}$.

$A \times B$ and $B \times A$ are not the same because, for example, $(4, 1)$ is not equal to $(1, 4)$.

In relational algebra, the expression

relation1 times relation2

evaluates to a relation whose heading is made up of all the attributes of the headings of both *relation1* and *relation2* and tuples that are made up from every tuple of *relation1* appended to every tuple of *relation2*. Here, we may have to make use of **rename** so as to ensure that no two attributes have the same name.



Since we are concerned with relations, there are no order considerations in defining **times**. We shall explore this point further in Exercise 3.26 below.

EXERCISE 3.25

Given the relations A and B , depicted by the tables below, write down a table depicting the relation A **times** B .

<i>StudentId</i>
<i>s01</i>
<i>s02</i>
<i>s05</i>
<i>s07</i>

<i>CourseCode</i>
<i>c2</i>
<i>c4</i>

EXERCISE 3.26

We noted above that the Cartesian product $A \times B$ is not equal to $B \times A$ for arbitrary sets A and B . (If you are familiar with the term ‘commutative’, this is equivalent to saying that the Cartesian product is not commutative.)

Is the same true for **times**, that is, for arbitrary relations A and B , is A **times** B a different relation from B **times** A ?

EXERCISE 3.27

For the relations A and B presented in Exercise 3.25, write down a table depicting **divide** (A **times** B) **by** B

EXERCISE 3.28

Write a relational expression equivalent to

Enrolment **join** *Student*

using the operators **times**, **rename**, **select** and **project**.

Hint: think of how to eliminate the surplus data in *Enrolment* **times** *Student*.



You might find this exercise particularly challenging!

A historical note on the relational data model

The relational data model was originally developed by an IBM researcher E.F. Codd (Edgar Codd – known as Ted) and described in

Codd, E.F. (1970) 'A relational model of data for large shared data banks', *Communications of the ACM*, vol. 13, no. 6, pp. 377–387.

Codd originally described eight operators: **select**, **project**, **join**, **divide**, **union**, **intersection**, **difference** and **times**. Of these eight operators, he regarded **select**, **project**, **times**, **union** and **difference** as primitive in the sense that the other three operators – **join**, **divide** and **intersection** – can be defined in terms of these five (as we did with **join** in Exercise 3.28, ignoring the issue of renaming).

There has, of course, been a certain amount of tidying-up and refinement of the original model since it was first described, but the seminal work was Codd's.

3.6 Summary

In this section, we introduced a set of theoretical operators {**select**, **project**, **join**, **divide**, **union**, **intersection**, **difference**} which, given that they are all closed on the set of relations, enable us to derive new relations from old. We also introduced the enabling operator **rename**: 'enabling' in the sense that it is of limited use on its own but enables us to apply other operators. Given that relations may be depicted as tables, these operators may be thought of as a means of extracting specific information from tables.

The operators **select**, **project**, **join** and **divide** are specific to relational algebra, whereas **union**, **intersection** and **difference** are closely related to the corresponding set operators. One difference between this latter group and the corresponding set operators is that the operands in the relational setting must be **union-compatible**, that is, have the same set of attributes.

In the examples, we saw how the use of an **alias** might be helpful in breaking down problems.

Finally, we briefly discussed **times**, one of the original primitive relational operators closely related to the Cartesian product of two sets.

LEARNING OUTCOMES

Having studied this section, you should now be able to:

- ▶ Apply the seven operators **select**, **project**, **join**, **divide**, **union**, **intersection** and **difference** to relations in order to find other relations.
- ▶ Apply relational algebra expressions – consisting of relations and operators – to derive relations which represent data having specific properties.
- ▶ Understand the operator **times**.

4 Constraints

In Section 2, we saw how parts of the E–R conceptual model can be transformed into a relational representation depicting relations as tables and using the foreign key mechanism to represent relationships. In that section, we did not consider in any great detail how to ensure that data values represent the real world and how they might be prevented from taking values which are impossible in the real world. **Constraints** help us to address this issue. For example, appropriate constraints on the relevant data values prevent us:

- ▶ from entering a person's age as a negative number;
- ▶ from entering an enrolment date for a particular student taking a particular course *after* the student has taken the exam for that course;
- ▶ from assigning the same student identifier to two different students.

Constraints can also be used to ensure that:

- ▶ a student who submits an assignment for a course is, in fact, enrolled on that course;
- ▶ if a course only has assignments numbered 1 to 5, then a student cannot submit assignment number 6.

These are just a few of the ways in which constraints can help to maintain integrity in our data models – we'll be looking at more detailed examples in this section.

In Section 2, we saw some examples of constraints. One of these is the constraint imposed by the definition of a domain over which the values of an attribute are defined. A domain constraint may be used to enforce the last constraint above: if we have defined the attribute *AssignmentNumber* as taking values from the domain $AssignmentNumbers = \{1...5\}$, then an assignment cannot be given the number 6. We also discussed the constraints associated with candidate and foreign keys. For example, declaring *StudentId* as a primary (and hence candidate) key for *Student* stops us from assigning the same student identifier to two different students.

In this section, we shall consider three categories of constraints: candidate and foreign key constraints, tuple constraints and general constraints.

4.1 Candidate and foreign key constraints

Candidate key constraints

In Section 2, we discussed candidate keys (primary or alternate keys) and foreign keys. As we saw in that section, the declaration of candidate keys constrains the set of tuples in a relation in that no two tuples may have the same value for a candidate key.

EXERCISE 4.1 (Revision)



- (i) In Exercise 2.17, we discussed the difference in semantics (meaning) between the following relations:

Appointment1(PatientId, ApptDate, ApptTime, ConsultantId)

Appointment2(PatientId, ApptDate, ApptTime, ConsultantId)

Given that each relation includes the tuple

<p01, '27 Dec, 2005', 14.30, s13>

determine whether each of the following tuples are allowable in *Appointment1* and/or *Appointment2*:

<p01, '27 Dec, 2005', 15.30, s13>

<p02, '27 Dec, 2005', 14.30, s13>

<p01, '11 Dec, 2005', 14.30, s13>

- (ii) Write down a plausible alternate key for *Appointment1*.

Foreign key constraints

We now turn our attention to the constraints imposed by foreign key declarations. You might recall from the definition of a foreign key, in Subsection 2.5, that the value of a foreign key appearing in a referencing relation must be equal to a value of the referenced candidate key appearing in the referenced relation. For example, you may remember from Figure 2.17 that *RegionNumber* is a foreign key in the relation *Student* referencing the relation *Region* representing the relationship **Manages**. In particular, this implies that for every value of *RegionNumber* appearing in a *Student* table, there must be a corresponding row in a *Region* table with that value as the value of the primary key – if, for example, a student is shown as having *RegionNumber* 57, then there must be a region 57 in the *Region* table.

Such a constraint is referred to as the **referential integrity** rule, which may be written thus:

If a relation R_2 has a foreign key F that references a candidate key P in a relation R_1 , then every value of F appearing in a tuple of R_2 must equal a value of P appearing in a tuple of R_1 .

Note that P is usually the primary key of R_1 , and that R_1 and R_2 need not be different.

A problem arises when you want to delete a row (tuple) from a table depicting the referenced relation. What happens if this tuple is referenced by another tuple? In the example above, what would happen to the student from region 57 if it was decided to delete that region from the *Region* relation? There are several possible answers to this problem, depending on the context.

Restricted effect

With the **restricted effect**, deletion of tuples in the referenced relation is restricted to tuples which are *not* explicitly referenced. Tuples which are so referenced *may not* be deleted. So, in the example above, the tuple in *Region* with primary key value 57 would not be able to be deleted; a tuple with primary key value 117 could be deleted only if no tuple in any referencing relation referenced it.

You may have noticed that this is equivalent to the definition of foreign key which you met earlier.

If the foreign key represents a recursive relationship from one entity type to itself, as in the discussion of the *Appraises* relationship in Subsection 3.2, then R_1 and R_2 will be the same.

Cascade effect

With the **cascade effect**, deletion of a referenced tuple would result in deletion of all the tuples referencing it. So for example, deletion of the tuple in *Region* with primary key value 57 would have the cascade effect of all students in the *Student* relation in region 57 being deleted.

Default effect

The **default effect** is as follows: when a referenced tuple is deleted, then the value of the foreign key attribute – or attributes – in the referencing tuples is set to some default value (which, of course, must appear as a value of the appropriate candidate key in the referenced relation). For example, if the tuple in *Region* with primary key value 57 were deleted, then all students who previously had 57 as the value of their region number would be assigned a new default value, 999, say (assuming that a region with number 999 appears in the table *Region*).

The choice of effect may be declared as an augmentation of the foreign key declaration, as in ‘referential integrity by <effect>’, where effect could be restricted, cascade or default, with the latter augmentation including the value of the default. You will see examples of this in *Block 3*.

These three types of effects may be implemented as ‘referential actions’, that is, actions taken by the DBMS to enforce referential integrity. You will learn more about referential actions in *Block 3*.



EXERCISE 4.2

Which effect is likely to be the most appropriate to preserve referential integrity when tuples are deleted from the referenced relation in the following cases?

- (i) *StudentId* as a foreign key in *Enrolment* referencing *Student* in the University model.
- (ii) *StaffNo* as a foreign key in *Team* referencing *Doctor* (recall, from Figure 2.22, that the foreign key represents the doctor heading the team).

EXERCISE 4.3

Suppose the foreign key *StudentId* in *Enrolment* is augmented by ‘referential integrity by cascade’. What would be the effect on the tables in Figures 3.2 and 3.3 of deleting the following?

- (i) The tuple <s01, c4, ‘Jan 12, 2005’> in *Enrolment*.
- (ii) The tuple <s09, Reeves, ‘34 The Crescent, Curville’, T.Reeves@nnet.fake.com, ‘Dec 14, 2004’, 4> in *Student*.

Updating the value of the primary key of a referenced tuple leads to a consideration of the same sort of issues as if the tuple had been deleted, but we shall not discuss this further.

4.2 Tuple constraints

We have already discussed how values taken by a particular attribute might be constrained by its domain and/or by the declaration of the attribute as a candidate or foreign key. We can also constrain values by means of explicit **tuple constraints**, where we impose Boolean conditions on values of attributes in the same tuple for each tuple of the relation.

For example, suppose we add a new attribute *DateOfBirth* to the relation *Student*, so that its relational heading is now *Student(StudentId, Name, DateOfBirth, Address, EmailAddress, RegistrationDate, RegionNumber)*. An obvious constraint is that we don’t want you to be registered at the University on or before your date of birth (this is

unlike the situation for some over-subscribed nurseries, or so I have been told). This can be expressed as a relational algebra constraint using the key word **constraint** as follows:

constraint *DateOfBirth* < *RegistrationDate*

Each tuple is tested to ensure that the condition is true for that tuple – that the value of *DateOfBirth* is before the value of *RegistrationDate*. The importance of the attribute values coming from the same tuple is clear – it wouldn’t make much sense to compare (for example) my date of birth with your registration date!

These constraint definitions are placed after the key declarations in the relational representation.

EXERCISE 4.4

Suppose we want to ensure that no student can enrol on a course before they are registered. Does the following expression do what we want?

constraint *RegistrationDate* <= *EnrolmentDate*



4.3 General constraints

Participation conditions

In Section 2, we saw how mandatory participation of entities in relationships can sometimes be represented by foreign keys. For example, in Figure 4.1 below, the mandatory participation of **Nurse** in **StaffedBy** is represented by the foreign key *WardNo* in the relation *Nurse*. Now we consider how such mandatory participation is represented when foreign key representation is impossible. This is the case when the mandatory participation is at the 1: end of a 1:*n* relationship, as in the mandatory participation of **Ward** in **StaffedBy** in Figure 4.1, or when both ends of a 1:1 relationship have mandatory participation conditions.

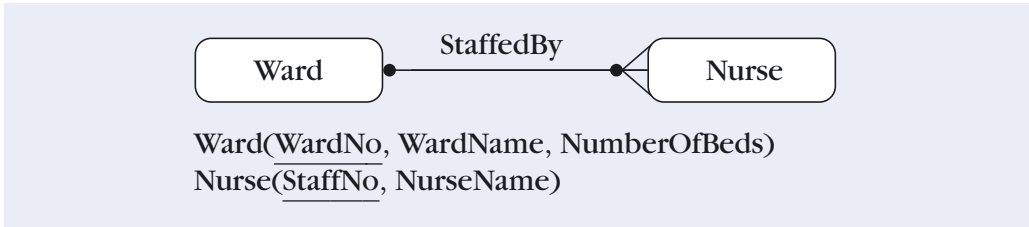


Figure 4.1 Relationship with mandatory participation at both ends

First, let us revise how to represent mandatory participation of **Nurse** in **StaffedBy**.

EXERCISE 4.5 (Revision)

Write down a relational representation of the entity types **Ward** and **Nurse** and the relationship **StaffedBy**, as in Section 2. You may assume the domains *WardNos*, *WardNames*, *BedNumbers*, *StaffNos*, *Names*. Which part of your model represents the fact that the participation of **Nurse** in **StaffedBy** is mandatory?



We want to establish now that every tuple in *Ward* has a matching tuple in *Nurse* – that is, that every value of *WardNo* appearing in *Ward* also appears in some tuple of *Nurse*. Since we are only interested in the attributes with domain *WardNos*, we can find the values taken by these attributes using **project** expressions:

project *Ward* **over** *WardNo*

project *Nurse* **over** *WardNo*

We now want to establish that every value of *WardNo* which appears in *Ward* also appears in *Nurse*.



EXERCISE 4.6 (Revision)

- (i) If every element of a set A is also an element of a set B, what can you say about A difference B?
- (ii) If A and B are sets such that A difference B is empty, what is the connection between elements of A and elements of B?

Putting the information gleaned from Solution 4.6 together and using the phrase ‘is empty’, we have the following constraint which we append to the relation *Ward*:

constraint ((**project** *Ward* **over** *WardNo*) **difference** (**project** *Nurse* **over** *WardNo*)) **is empty**



EXERCISE 4.7



Figure 4.2 An E–R model relating *Course* and *Examiner*

In Exercises 2.29 and 2.30, we established that the following relational representation corresponds to the E–R diagram in Figure 4.2, with suitable entity types:

relation *ExaminedBy*

CourseCode: *CourseCodes*

StaffNo: *StaffNos*

primary key (*CourseCode*, *StaffNo*)

foreign key *CourseCode* **references** *Course*

foreign key *StaffNo* **references** *Examiner*

relation *Course*

CourseCode: *CourseCodes*

Title: *TitlesOfCourses*

Credit: *Credits*

primary key *CourseCode*

relation *Examiner*

StaffNo: *StaffNos*

Name: *Names*

primary key *StaffNo*

Amend this relational representation to represent the following E–R diagram with the same entity types.



Figure 4.3 Mandatory participations in an E–R model relating **Course** and **Examiner**

EXERCISE 4.8

Figure 4.4 depicts a fragment from the Hospital E–R model.

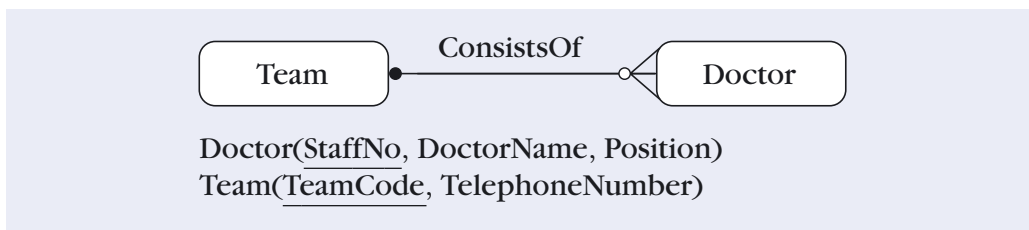


Figure 4.4 Fragment of the Hospital E–R model showing mandatory participation at the :1 end of a 1:*n* relationship

Write down a relational representation of this fragment, assuming the standard domains from the Hospital model. Recall from the discussion in Section 2 that **ConsistsOf** must be represented by a relation. (The participation of **Doctor** in **ConsistsOf** is optional – and hence we can't represent **ConsistsOf** by a foreign key *TeamCode* in *Doctor*, because if we did, some tuples in *Doctor* would have no value for the attribute *TeamCode*.)

Draw an E–R diagram corresponding to your representation with the three entity types **Team**, **Doctor** and **ConsistsOf**.

Other constraints

The following general form of a constraint is often exceedingly useful for expressing constraints other than mandatory participations:

constraint (set of tuples, each of which obeys some undesirable condition) **is empty**

What this says, of course, is that there are no tuples obeying the undesirable condition. Consider, for example, the expression that we met earlier:

constraint ((**project** *Ward* **over** *WardNo*) **difference** (**project** *Nurse* **over** *WardNo*)) **is empty**

This says that there are no tuples in (**project** *Ward* **over** *WardNo*) **difference** (**project** *Nurse* **over** *WardNo*) – that is, that there is no value of *WardNo* appearing in *Ward* which doesn't also appear in *Nurse*.

As another example, consider the situation of Exercise 4.4 above, where we wanted to ensure that a student could not enrol on a course before being registered. Here, the undesirable condition is that a student *has* enrolled on a course before being registered – we want the general form of the constraint to be:

constraint (set of tuples where enrolment date of a student on a course is before that student's registration date) **is empty**

We need to tie together the student's registration information with the enrolment information by joining together the relations *Enrolment* and *Student*, as in:

Enrolment **join** *Student*

Then we need to select those tuples which *do* satisfy our undesirable condition, as in:

select (*Enrolment* **join** *Student*) **where** *EnrolmentDate* < *RegistrationDate*

And then we want to ensure this set is empty, as in:

constraint (**select** (*Enrolment* **join** *Student*) **where** *EnrolmentDate* < *RegistrationDate*) **is empty**



EXERCISE 4.9

Suppose we have relations *Patient1* and *Doctor* with relational headings *Patient1* (*PatientId*, *PatientName*, *ConsultantNo*) and *Doctor*(*StaffNo*, *Name*, *Position*), respectively, where *ConsultantNo* and *StaffNo* are defined over the same domain, and the other domains are as defined in the standard relational representation of the Hospital model. Write down a relational expression to express the constraint that a doctor's *StaffNo* can only appear as a value of *ConsultantNo* if the value of the *Position* attribute of that doctor is 'Consultant'. (This constraint represents the fact that only a consultant can be responsible for a patient.)

Hints:

You want to associate a patient with the doctor who is looking after him/her (whose number is *ConsultantNo*). This might involve a use of the **rename** operator.

Then look for a solution of the form **constraint** (set of tuples, each of which obeys some undesirable condition) **is empty**.

EXERCISE 4.10

Given the relations *Nurse* and *Doctor*, as defined in the standard relational representation for the Hospital model, write a constraint to represent the fact that no nurse can have the same value of the attribute *StaffNo* as any doctor, and vice versa.

4.4 Summary

This section completes the discussion that we started in Section 2, on how an E–R model can be transformed into a relational representation. In this section, we discussed different sorts of constraints: constraints arising from the definition of candidate keys and foreign keys (here we also examined various methods of dealing with the issue of referential integrity), tuple constraints and a short discussion on general constraints of the form **constraint** (...) **is empty**.

In the next section, we shall start looking at the issue of database design.

LEARNING OUTCOMES

Having studied this section, you should now be able to:

- ▶ Define and understand the different ways of ensuring referential integrity.
- ▶ Choose the appropriate method of ensuring referential integrity according to the real-life situation.
- ▶ Represent tuple constraints where appropriate.
- ▶ Represent the mandatory participation of an entity in a relationship when this can't be done using foreign keys.
- ▶ Represent general constraints in the form **constraint (...) is empty**.

5

Normal forms

In this section we begin thinking about the issue of database design by considering relations in normal forms. Students often find the topic of normal forms to be quite difficult. Be aware that you might have to spend more time on this section than on some of the previous sections and that you may have to read parts of the material several times.

Writing about normal forms for students is also quite difficult. It is possible to write long mathematical tomes on this topic – but we don’t do that in this course. In what follows, we try to strike a balance between understanding and rigour.

5.1

Motivation

Recall from Section 3 that the operands to these operators must be union-compatible, that is, they must have the same headings.

In Section 3 of this block, we met a set of relational operators which included the operators **union**, **intersection** and **difference**. Although we didn’t go into this in any great detail in Section 3, these operators can be used to generate new relations with the same headings as the originals but with different bodies. Such new relations are necessary when the information represented by a given relation changes, for example, when data gets updated or deleted or when new data is added.

For example, suppose we have a relation *BasicStudent*, based on the University model, which has heading and body as depicted below.

BasicStudent

StudentId	Name
s1	Ali
s2	Baz
s3	Chuck

We also have the relations *NewStudent*, *ExStudent* and *ChangeStudent*, which have the same heading as *BasicStudent* and bodies as shown below.

NewStudent

StudentId	Name
s4	Ella

ExStudent

StudentId	Name
s2	Baz

ChangeStudent

StudentId	Name
s2	Barbarella

So, compared with the original relation *BasicStudent*, the relation *BasicStudent union NewStudent* has added data about the new student s4:

BasicStudent union NewStudent

<i>StudentId</i>	<i>Name</i>
<i>s1</i>	<i>Ali</i>
<i>s2</i>	<i>Baz</i>
<i>s3</i>	<i>Chuck</i>
<i>s4</i>	<i>Ella</i>

Similarly, *BasicStudent difference ExStudent* has data about s2 deleted:

BasicStudent difference ExStudent

<i>StudentId</i>	<i>Name</i>
<i>s1</i>	<i>Ali</i>
<i>s3</i>	<i>Chuck</i>

And *(BasicStudent difference ExStudent) union ChangeStudent* records changed information about s2.

(BasicStudent difference ExStudent) union ChangeStudent

<i>StudentId</i>	<i>Name</i>
<i>s1</i>	<i>Ali</i>
<i>s2</i>	<i>Barbarella</i>
<i>s3</i>	<i>Chuck</i>

When we are designing a relational database, we have choices about how to design relations. A poor choice can lead to problems such as being unable to record simple facts or easily update information, or the inadvertent loss of information.

For example, suppose we choose to record information in a context similar to that of the University model you've already seen, about students, the courses that they are enrolled on and their tutors for these courses, in a relation *StudentTutorCourse* represented by the table in Figure 5.1. The primary key is *(StudentId, CourseCode)*.

StudentTutorCourse

<i>StudentId</i>	<i>StudentName</i>	<i>CourseCode</i>	<i>TutorId</i>	<i>TutorName</i>
<i>S1</i>	<i>Ashok</i>	<i>C1</i>	<i>T1</i>	<i>Ann</i>
<i>S1</i>	<i>Ashok</i>	<i>C2</i>	<i>T2</i>	<i>Barry</i>
<i>S2</i>	<i>Belinda</i>	<i>C1</i>	<i>T3</i>	<i>Cayley</i>
<i>S3</i>	<i>Charles</i>	<i>C3</i>	<i>T1</i>	<i>Ann</i>

Figure 5.1 Relational table for *StudentTutorCourse*

This choice poses problems, as we now explore in Exercise 5.1.

Note that Figure 5.1 represents data which are similar to, but not identical with, the University model that you've already seen. For example, the latter does not have a separate entity type for Tutors.

Note: we have used capital letters for our identifiers here to make clear that this is not the same as the standard University model.



EXERCISE 5.1

- (i) Suppose a new tutor, Meera, has been appointed on course C2, given the identifier T4, but not yet allocated any students. Why can this information not be recorded in a new relation with the same heading as *StudentTutorCourse* and with the body extended from that of *StudentTutorCourse* so as to include the new information?
- (ii) Tutor T1, Ann, has decided that henceforth she wants to be known as Albert. What problems might this pose for database maintenance?
- (iii) Student S2, Belinda, has decided to withdraw from the university. What problems might this pose?

The problems identified in Solution 5.1 are commonly referred to as **insertion**, **amendment** and **deletion** anomalies, respectively.

What is it about the relation represented by the table in Figure 5.1 which leads to such anomalies? You may have noticed that this relational table contains redundant information – which is the direct cause of the amendment anomaly noted in Solution 5.1(ii).



EXERCISE 5.2

What redundant information is present in the relational table of Figure 5.1?

We saw in Section 2 that the value of a primary key identifies a unique tuple of a relation – it may be thought of as the ‘essence’ of the tuple. But the relation represented by the table in Figure 5.1 contains information which appears essentially unrelated to the primary key (*StudentId*, *CourseCode*), that is, the names of tutors. Also, some of the information in the relation is only associated with part of the primary key. For example, the name of a student is only associated with their identifier and not with the other part of the key, the course code.

What would be the effect of our insisting that, in every tuple, every attribute value is a fact about the whole primary key (unlike the name of a student above) and nothing but the primary key (unlike the name of a tutor, which is basically a fact about an identified tutor)? Would the anomalous behaviour illustrated in Exercise 5.1 disappear? What happens if the relation has more than one candidate key? We will address these and similar questions in this section by examining the consequences of relations having certain types of structure, that is, obeying certain properties. These structures are called **normal forms**, and we shall investigate four of them: first, second, third and Boyce–Codd normal forms.

Before considering these normal forms, we need to discuss the concepts of single-valued facts and functional dependencies.

The relation depicted by Figure 5.1 is only in first normal form, as we shall see.

5.2

Single-valued facts and functional dependencies

A **single-valued fact type**, often abbreviated to **single-valued fact**, is a statement (fact) identifying a property of an entity type which can only take a single value for each entity. For example, consider the following statement:

Each student has exactly one recorded name.

This is a single-valued fact (SVF) type in the University model – the property of recorded name has only one value for each student. On the other hand, look at this statement:

Each name is attached to a student.

This is not an SVF – a common name like ‘John Smith’ might be shared by several different students.

Instances of single-valued facts are called **occurrences**. For example, there are three occurrences of the SVF above illustrated in Figure 5.1:

- ▶ Student S1 has exactly one recorded name, Ashok.
- ▶ Student S2 has exactly one recorded name, Belinda.
- ▶ Student S3 has exactly one recorded name, Charles.

These occurrences may be stated more simply as (for example):

- ▶ Student S1 has name Ashok.

However, strictly speaking, this is ambiguous – it doesn’t preclude S1 from having other (recorded) names.

Ambiguity is often a problem with SVFs. When we are designing a database, we may come across statements in the requirements specification which appear to be single-valued facts, such as the following in a Hospital model:

A consultant has an office.

But beware of the ambiguity inherent in this statement. Further investigation is needed to ascertain whether the statement above is, in fact, a representation of the single-valued fact:

Each consultant has exactly one office.

Or maybe the statement doesn’t represent a single-valued fact at all, but is instead a statement about one particular consultant who has an office whereas others do not (and is thus a property of a particular entity rather than of an entity type).

There may even be a more complex situation where (for example) a consultant is based in a health district and travels around hospitals, and the single-valued fact is actually a statement of a property of the (consultant, hospital) pair:

Each consultant has exactly one office in each hospital.

You might have realised that there is a strong connection between the concept of SVFs and both E–R modelling and relational databases. Regarding E–R modelling, you may recall that an attribute of an entity type is a property of that entity type, and this attribute takes a unique value for each entity. So identifying single-valued facts in a requirements specification helps the E–R modeller to identify attributes. From a relational point of view, if we have a relation $R(\underline{p}, a_1, a_2, \dots)$, then, given the fact that each value of a primary key determines a unique tuple, and each attribute value in a tuple is unique, we can derive the single-valued facts:

Each value of p corresponds to exactly one value of a_1 .

Each value of p corresponds to exactly one value of a_2 .

...

We use the abbreviation ‘SVF’ for single-valued fact type.

The ambiguity inherent in SVFs may be addressed by expressing an SVF as a functional dependency, as we shall see shortly.

Note that p can, of course, be a combination of attributes, as in Exercise 5.3.

EXERCISE 5.3

Write down all the single-valued facts in the relation *StudentTutorCourse*, as depicted in Figure 5.1, which express properties of the primary key (*StudentId*, *CourseCode*).





EXERCISE 5.4

In Exercise 5.2, we identified redundancies in Figure 5.1 (we are told more than once that the student S1 is called Ashok and that the tutor T1 is called Ann). These redundancies are occurrences of two single-valued facts which are not statements about the primary key. What are these two single-valued facts?

With the aim of reducing ambiguity, we may express an SVF as a **functional dependency**.

Informally, an attribute A of a relation R is functionally dependent on a set of attributes $S = \{A_1, \dots, A_n\}$ of R if each value $(a_1, \dots, a_n)$ determines a single value of A , where a_1 is a value of A_1 , a_2 a value of A_2 , and so on.

So, for example, in the relation *StudentTutorCourse*, *TutorId* is functionally dependent on the set $\{StudentId, CourseCode\}$ as each value of $\{StudentId, CourseCode\}$ determines a unique value of *TutorId*. For instance, the value (S1, C2) determines the value T2, and the value (S2, C1) determines the value T3.

Notation: We write $S \mapsto A$ (so $StudentId, CourseCode \mapsto TutorId$).

S is called the **determinant** of the functional dependency (as in ‘ S determines A ’).

Single-valued facts and functional dependencies may be considered to be ways of expressing the same thing at different levels of abstraction. Single-valued facts relate to the real world – they are statements about real-world properties – whereas functional dependencies are formal expressions which belong to the world of relational databases, where a particular functional dependency may or may not hold for a particular relation.

We shall often abbreviate the term functional dependency to FD.

For the sake of brevity, we shall often omit the curly brackets $\{\}$ from around sets.



EXERCISE 5.5

In Exercises 5.3 and 5.4, we identified the following SVFs in the relation depicted by Figure 5.1 (we have numbered the SVFs for ease of reference):

- SVF1: Each student on each course has exactly one name.
- SVF2: Each student on each course has exactly one identified tutor.
- SVF3: Each student on each course has exactly one named tutor.
- SVF4: Each student has exactly one name.
- SVF5: Each tutor has exactly one name.

Write down each of these single-valued facts as functional dependencies in *StudentTutorCourse*. In each case, identify the determinant.

EXERCISE 5.6

One of the following statements is true, and one is false. Identify which is true and which is false, and justify your answers.

- (i) If C is a candidate key of a relation R , then every attribute of R is functionally dependent on C .
- (ii) If every attribute of R is functionally dependent on some subset C of the set of attributes of R , then C is a candidate key.

We discussed candidate keys in Subsection 2.4.

As well as reducing ambiguity, FDs have a further advantage over SVFs in that they may be reasoned about as mathematical entities, as in the properties below.

Properties of functional dependencies

Property 1: combining functional dependencies

Every person has unique values for age, gender (male or female), height and weight. Given the relation $Person(PersonId, Age, Gender, Height, Weight, BodyMassIndex)$, we can express these dependencies either as four separate functional dependencies (where FD1 represents the SVF that each person has a unique age, and so on):

FD1: $PersonId \mapsto Age$

FD2: $PersonId \mapsto Gender$

FD3: $PersonId \mapsto Height$

FD4: $PersonId \mapsto Weight$

or as a single FD:

FD5: $PersonId \mapsto Age, Gender, Height, Weight$

This is an example of the combination property of functional dependencies.

The combination property of functional dependencies may be stated as follows:

If A , B and C are sets of attributes of a relation R , so that $A \mapsto B$ and $A \mapsto C$, then $A \mapsto B, C$.

In the example above, we have that if A is the set consisting of the single attribute $PersonId$, B the set consisting of the single attribute Age , C the set consisting of the single attribute $Gender$, then

$PersonId \mapsto Age$

and

$PersonId \mapsto Gender$

together give

$PersonId \mapsto Age, Gender$

And similarly for the other attributes on the right-hand sides of FD3 and FD4.

Note that we have missed out the curly brackets denoting sets. (If we were feeling pedantic, we might have written, for example, $Age, Gender$ as $\{Age, Gender\}$.)

The converse of the combination property also holds:

If each value of A determines a unique value of the attributes in the union of B with C then it clearly determines a unique value of the attributes of B and C separately.

So, for example, from

$PersonId \mapsto Age, Gender$

we can infer both

$PersonId \mapsto Age$

and

$PersonId \mapsto Gender$

For example, if we know that the person with national insurance number X12345Y is a female aged 21, then we certainly know both that X12345Y is 21 and that X12345Y is female.

We shall consider $BodyMassIndex$ later.

Here, we are using B, C as a shorthand for the set union of B with C . The concept of set union was discussed in Subsection 3.4.

Property 2: extending determinants

Given that a student's name is uniquely determined by their identifier, then it is also uniquely determined by (for example) the combination of their identifier and a tutor identifier. For example, if the student S1 is called Charles, then the student S1 with tutor T1 is still called Charles.

This is an example of the property of extending determinants, which may be stated as follows:

If A , B and C are sets of attributes of a relation R , such that $A \mapsto C$ and A is a subset of B , then $B \mapsto C$.

So in our example, following our usual convention of omitting set brackets, A is *StudentId*, B is *StudentId*, *TutorId* and C is *StudentName*.

It follows from this that:

If it is **not** true that the FD $B \mapsto C$ holds in a relation, then it is **not** true that $A \mapsto C$ holds, where A is any subset of B .

For example, suppose there are two students called John Smith on course C100: one has Janet Brains as a tutor, and the other has Joseph Genius. Here we see that a unique value of the pair student name and course code does not uniquely determine a tutor name, that is:

It is **not** true that $StudentName, CourseCode \mapsto TutorName$ holds in the relevant relation.

Thus, a student name alone does not uniquely determine a tutor name (a student called John Smith may have Janet Brains *or* Joseph Genius as tutor) and neither does a course code alone (C100 is tutored by Janet Brains *and* Joseph Genius). In other words:

Neither $StudentName \mapsto TutorName$ nor $CourseCode \mapsto TutorName$ holds.

Property 3: transitivity

As an example of **transitivity**, we know that values of height and weight determine an individual's body mass index, so we have:

FD6: $Height, Weight \mapsto BodyMassIndex$

But from FD3 $PersonId \mapsto Height$ and FD4 $PersonId \mapsto Weight$, and using Property 1 (the property of combining functional dependencies) we know that

$PersonId \mapsto Height, Weight$

and so we have

FD7: $PersonId \mapsto BodyMassIndex$

This is an example of the transitivity property of FDs which says that if A , B and C are sets of attributes of a relation R such that each value of A determines a unique value of B , and each value of B determines a unique value of C , then each value of A determines a unique value of C . The transitivity property may be stated as follows:

If $A \mapsto B$ and $B \mapsto C$ then $A \mapsto C$.

In our example, A is *PersonId*, B is *Height, Weight* and C is *BodyMassIndex*.

Property 4: augmentation

As an illustration of **augmentation**, given that a particular student identifier determines a student name, then we know that a particular student identifier together with a tutor name determines that student's name and that tutor name. For example, if the student

We discussed subsets in Subsection 2.4.

Accordingly, in Solution 5.5, FD1 may be derived from FD4 by extending the determinant.

S1 is called Charles, then the student S1 with tutor Thomas is the student called Charles with tutor Thomas.

The property of augmentation may be stated as follows:

If A , B and C are sets of attributes of a relation R such that $A \twoheadrightarrow B$, then $A, C \twoheadrightarrow B, C$.

This seemingly trivial property can be very useful in identifying new FDs, and hence in elucidating more of the dependency structure of the data, as we shall see in Exercise 5.9 below.

Finding functional dependencies

Given a relational table, we need to find as many functional dependencies as possible in order to elucidate the dependencies in the data.

For example, consider the following scenario regarding patients' hospital appointments:

A patient is identified by a patient identifier; a consultant is identified by a consultant identifier; a hospital, by a hospital number. A patient's (only) appointment on a particular date is at a particular time and to see a particular consultant. The names of both the patient and the consultant are recorded, as is the name of the hospital. A consultant only works at one hospital on a given date. A consultant only sees one patient on a given date and time.

Note that this example is not directly linked to the Hospital model you have already met.

The data relevant to this scenario for part of a single year is recorded in the relational table shown in Figure 5.2, representing the relation *Appointment*.

	<i>PatientId</i>	<i>PatientName</i>	<i>ApptDate</i>	<i>ApptTime</i>	<i>ConsId</i>	<i>ConsName</i>	<i>HospNo</i>	<i>HospName</i>
1	<i>p01</i>	<i>Balthazar</i>	<i>12/10</i>	<i>14.00</i>	<i>c1</i>	<i>Louella</i>	<i>h1</i>	<i>Faith</i>
2	<i>p01</i>	<i>Balthazar</i>	<i>14/10</i>	<i>13.00</i>	<i>c2</i>	<i>Clementine</i>	<i>h2</i>	<i>Hope</i>
3	<i>p01</i>	<i>Balthazar</i>	<i>09/09</i>	<i>09.00</i>	<i>c3</i>	<i>Nectarine</i>	<i>h3</i>	<i>Charity</i>
4	<i>p02</i>	<i>Cornelius</i>	<i>09/09</i>	<i>14.00</i>	<i>c1</i>	<i>Louella</i>	<i>h4</i>	<i>Flanders</i>
5	<i>p02</i>	<i>Cornelius</i>	<i>14/10</i>	<i>14.00</i>	<i>c2</i>	<i>Clementine</i>	<i>h2</i>	<i>Hope</i>
6	<i>p03</i>	<i>Samuel</i>	<i>16/10</i>	<i>09.00</i>	<i>c2</i>	<i>Clementine</i>	<i>h3</i>	<i>Charity</i>
7	<i>p03</i>	<i>Samuel</i>	<i>13/10</i>	<i>16.00</i>	<i>c1</i>	<i>Louella</i>	<i>h4</i>	<i>Flanders</i>
8	<i>p04</i>	<i>Darcy</i>	<i>12/10</i>	<i>13.00</i>	<i>c1</i>	<i>Louella</i>	<i>h1</i>	<i>Faith</i>
9	<i>p05</i>	<i>Schultz</i>	<i>12/09</i>	<i>13.00</i>	<i>c3</i>	<i>Nectarine</i>	<i>h2</i>	<i>Hope</i>
10	<i>p05</i>	<i>Schultz</i>	<i>12/10</i>	<i>14.00</i>	<i>c3</i>	<i>Nectarine</i>	<i>h2</i>	<i>Hope</i>
11	<i>p06</i>	<i>Samuel</i>	<i>23/11</i>	<i>17.00</i>	<i>c4</i>	<i>Louella</i>	<i>h3</i>	<i>Charity</i>

Figure 5.2 Table representing the *Appointment* relation – row numbers have been included for future reference

We do not, as yet, know the primary key of the relation (though you may be able to have a good guess) – nor do we know whether there are any alternate keys.

The scenario gives us the following single-valued facts about appointments:

SVF1: Each patient on a given date has an appointment at a particular time.

SVF2: Each patient on a given date has an appointment with a particular consultant.

SVF3: Each patient has exactly one name.

SVF4: Each consultant has exactly one name.

In any relational representation of *Appointment*, these single-valued facts must appear as a type of constraint, for example, as a candidate key or a more general constraint.

SVF5: Each hospital has exactly one name.

SVF6: Each consultant on a given date works at exactly one hospital.

SVF7: On a given date and time, a given consultant sees exactly one patient.



EXERCISE 5.7

Write the seven single-valued facts above as seven FDs which hold in *Appointment*.

To check for further functional dependencies, we now look for those which may be derived from transitivity. To do this, we identify FDs where the right-hand side corresponds to the left-hand side of another FD (so we have the pattern $A \mapsto B$ and $B \mapsto C$). For example, from Solution 5.7, the right-hand side of FD2 corresponds to the left-hand side of FD4. Therefore, by transitivity on FD2 and FD4 ($PatientId, ApptDate \mapsto ConsId$ and $ConsId \mapsto ConsName$), we get

FD8: $PatientId, ApptDate \mapsto ConsName$



EXERCISE 5.8

Write down two more FDs which can be derived by transitivity on the known FDs: FD1 to FD8.

We now make use of augmentation (Property 4) to find more FDs.



EXERCISE 5.9

- (i) Derive a new FD, FD11, from FD2, FD6 using augmentation and transitivity.
- (ii) Use FD11 and transitivity with another FD to derive FD12.

To recap, we have identified the following dependencies between data items in the table of Figure 5.2:

FD1 $PatientId, ApptDate \mapsto ApptTime$

FD2: $PatientId, ApptDate \mapsto ConsId$

FD3: $PatientId \mapsto PatientName$

FD4: $ConsId \mapsto ConsName$

FD5: $HospNo \mapsto HospName$

FD6: $ConsId, ApptDate \mapsto HospNo$

FD7: $ConsId, ApptDate, ApptTime \mapsto PatientId$

FD8: $PatientId, ApptDate \mapsto ConsName$

FD9: $ConsId, ApptDate \mapsto HospName$

FD10: $ConsId, ApptDate, ApptTime \mapsto PatientName$

FD11: $PatientId, ApptDate \mapsto HospNo$

FD12: $PatientId, ApptDate \mapsto HospName$

We found these FDs by:

- ▶ Identifying the SVFs in the scenario.
- ▶ Translating the SVFs into FDs.
- ▶ Looking for new FDs using transitivity.
- ▶ Looking for new FDs which could be derived by transitivity if they were augmented.

EXERCISE 5.10



Recall from Subsection 2.4 that a candidate key of a relation R is a minimal set of attributes C , so that each value of C determines a unique tuple of R , depicted as a unique row in a relational table. So, if $\{A_1, A_2, \dots, A_n\}$ is the set of attributes of R and $C \mapsto A_1, A_2, \dots, A_n$, then if C is minimal, it's a candidate key. Recall also from Property 1, the combining of functional dependencies, that if $C \mapsto A_1$, $C \mapsto A_2, \dots$, $C \mapsto A_n$, then $C \mapsto A_1, A_2, \dots, A_n$.

Using this information and the list of functional dependencies above, identify two candidate keys for the relation *Appointment*. Justify your answer.

EXERCISE 5.11

Consider the relation *ClassCourse*(ClassName, CourseCode, ClassRoom, ClassTeacherCode, ClassTeacherName, CourseName, CourseTeacherCode, CourseTeacherName), which concerns information relating to the administration of a school. Here, a class is a set of pupils and a course is a subject that they study. So, for example, Class 1 could study courses Maths, English, Art, Science, etc.

ClassName uniquely identifies a class, and *CourseCode*, *ClassTeacherCode* and *CourseTeacherCode* uniquely identify a course, class teacher and course teacher, respectively.

The relation represents a scenario from which the following single-valued facts are derived:

- SVF1: Each class has exactly one classroom.
- SVF2: Each class has exactly one class teacher.
- SVF3: Each class teacher has exactly one name.
- SVF4: Each course has exactly one name.
- SVF5: Each class taking each course has exactly one course teacher.
- SVF6: Each course teacher has exactly one name.
- SVF7: Each class teacher has exactly one class.
- SVF8: Each classroom has exactly one class.

- (i) Write these SVFs as FDs.
- (ii) Derive five more FDs (FD9 to FD13) by transitivity. Omit any trivial FDs (for example, of the form $A \mapsto A$ or $A, B \mapsto A$).
- (iii) Derive
 - (a) $ClassRoom, CourseCode \mapsto CourseTeacherCode$
 - (b) $ClassTeacherCode, CourseCode \mapsto CourseTeacherCode$
 by augmentation and transitivity using your current list of FDs.

We should note here that it is usually impractical to find the complete set of functional dependencies on a relation, unless the relation has only a few attributes. If R has just two attributes A and B , then we only have to test whether $A \mapsto B$ and/or $B \mapsto A$ hold

in R . With more attributes, the number of tests required to find the complete set of non-trivial functional dependencies rises sharply. For example, even if R only has three attributes A , B and C , then we have to determine whether $B \mapsto A$ and $C \mapsto A$ hold, and if they don't, whether $B, C \mapsto A$ does. And similarly for B and then C on the right-hand side of potential FDs. Obviously, the greater the number of attributes, the more potential FDs should be tested. Thus, unless we are considering a relation with only a few attributes, we do not aspire to find a complete set of FDs, but rather a set which suffices for our purpose. We now return to discussing that purpose.

You might recall from Subsection 5.1 that the purpose of this section is to discuss 'good' choices of relations. We suggested that a 'good' relation should enable you to generate new relations with the same heading, but amended body so as to record new occurrences of single-valued facts (such as 'Tutor T4 has name Meera'). A good relation should also allow you to amend data without having to worry about multiple occurrences (so doing away with unnecessarily redundant data), and prevent you from inadvertently losing occurrences of single-valued facts. We further suggested that the 'badness' of the relation depicted in Figure 5.1 is due to the presence of occurrences of single-valued facts which are not statements of properties of the primary key, or of the whole of the primary key. These SVFs correspond to the following FDs:

$$StudentId \mapsto StudentName$$

$$TutorId \mapsto TutorName$$

In the former case, the determinant of the FD is only part of the primary key; in the latter, the determinant does not include even part of the primary key. We will explore these types of FD further in the next subsection, always with the purpose of reducing data redundancy.

5.3 First and second normal forms (1NF and 2NF)

A relation in **first normal form** (1NF) is not of very great interest to us: it is included for historical significance only. The concept of 1NF pre-dates the development of relational theory; the intention of defining 1NF was to eliminate tables with multiple entries in individual cells.

A definition of first normal form is as follows:

A relation is in first normal form (1NF) if and only if it has no duplicate tuples and in each tuple, each value of every attribute is a single value.

From our discussion of relations in Section 2 of this block, we hope it is clear that every relation is, in fact, in 1NF. You may be wondering why, in this case, we bother with first normal forms, but we thought that if we began this discussion with 2NF, you might wonder about 1NF.

A relation in **second normal form** eliminates one of the causes of the redundant data that we identified in *StudentTutorCourse* in Subsection 5.1. Since the FD $StudentId \mapsto StudentName$ has a determinant $StudentId$ which is a proper subset of the primary key $StudentId, CourseCode$, then a value of $StudentId$ on its own does not identify a unique tuple – it can occur in several tuples – and hence we can have several appearances of a particular student's name in a relational table depicting *StudentTutorCourse*. We want to eliminate FDs of this type, where the determinant is a proper subset of the primary key.

Recall from Exercise 5.6 that every attribute is functionally dependent on the primary key. An attribute which is *not* functionally dependent on *any proper subset* of the primary key is said to be **fully functionally dependent** on the primary key. So,

This is equivalent to the definition of first normal form given in other OU courses.

Recall that we commonly omit the set brackets { and } – so by $StudentId, CourseCode$ we actually mean the set $\{StudentId, CourseCode\}$.

for example, from the depiction of *StudentTutorCourse* in Figure 5.1, we see that student S1 corresponds to more than one tutor, as does course C1, hence neither $StudentId \mapsto TutorId$ nor $CourseCode \mapsto TutorId$ holds, and thus $TutorId$ in *StudentTutorCourse* is fully functionally dependent on $StudentId, CourseCode$ since it is not functionally dependent on any subset.

In fact, the definition of fully functional dependent is more general than this – the relevant determinant need not be the primary key. The definition is as follows:

If A and B are sets of attributes of a relation R such that $A \mapsto B$ and B is not functionally dependent on any proper subset of A , then B is said to be fully functionally dependent on A .

This implies that if B is functionally dependent on a proper subset of A , then B is not fully functionally dependent on A .

So in *StudentTutorCourse*, the attribute *StudentName* is not fully functionally dependent on the primary key as $StudentId \mapsto StudentName$ holds. We say that the determinant of $StudentId, CourseCode \mapsto StudentName$ is **reducible** since it has a subset, the single attribute *StudentId*, which may itself be taken as the determinant of the FD $StudentId \mapsto StudentName$.

A relation in second normal form is characterised by not having any functional dependencies similar to $StudentId \mapsto StudentName$. The formal definition is:

A relation is in second normal form (2NF) if and only if every non-primary key attribute is fully functionally dependent on the primary key.

That is, if P is the primary key of a relation in 2NF, it must be an irreducible determinant for any FD of the form $P \mapsto A$. This is clearly the case when P consists of a single attribute – it cannot then be reduced. A relation where the primary key consists of a single attribute thus must be in 2NF.

We are ignoring the abnormal case when the primary key is the empty set.

EXERCISE 5.12

We established in Exercise 5.10 that the relation *Appointment* represented in Figure 5.2 has two candidate keys as follows:

- (i) *PatientId, ApptDate* giving the relation *Appointment(PatientId, ApptDate, PatientName, ApptTime, ConsId, ConsName, HospNo, HospName)*.
- (ii) *ConsId, ApptDate, ApptTime* giving the relation *Appointment(ConsId, ApptDate, ApptTime, PatientId, PatientName, ConsName, HospNo, HospName)*.

In each case, identify the functional dependencies from the list given after Exercise 5.9 which prevent *Appointment* being in 2NF.

Hint: look for functional dependencies where the determinant is a subset of the primary key.



If we want to eliminate redundancies caused by reducible determinants, we can do so by forming new relations, each of which has an offending reduced determinant as the primary key, and the attributes determined by this primary key as the non-primary attributes. These latter attributes are ‘stripped out’ of the original relation.

For example, in the relation of Figure 5.1, we can address the offending FD $StudentId \mapsto StudentName$ by projecting as follows:

Student2 **alias** (**project** *StudentTutorCourse* **over** *StudentId, StudentName*)

StudentTutorCourse2 **alias** (**project** *StudentTutorCourse* **over** *StudentId, CourseCode, TutorId, TutorName*)

We have used *Student2* as the name of the relation to reflect the fact that it describes students and to indicate that it is in 2NF.

giving
Student2(*StudentId*, *StudentName*)
and
StudentTutorCourse2(*StudentId*, *CourseCode*, *TutorId*, *TutorName*)
respectively.

You might have expected us to partition the relation as follows:

Student2'(*StudentId*, *StudentName*)
StudentTutorCourse2'(*CourseCode*, *TutorId*, *TutorName*)

but if we do this, we lose any link between the two resulting tables and thus valuable information is lost.

The pair of **project** expressions gives two tables as in Figure 5.3:

Student2

<i>StudentId</i>	<i>StudentName</i>
<i>S1</i>	<i>Ashok</i>
<i>S2</i>	<i>Belinda</i>
<i>S3</i>	<i>Charles</i>

StudentTutorCourse2

<i>StudentId</i>	<i>CourseCode</i>	<i>TutorId</i>	<i>TutorName</i>
<i>S1</i>	<i>C1</i>	<i>T1</i>	<i>Ann</i>
<i>S1</i>	<i>C2</i>	<i>T2</i>	<i>Barry</i>
<i>S2</i>	<i>C1</i>	<i>T3</i>	<i>Cayley</i>
<i>S3</i>	<i>C3</i>	<i>T1</i>	<i>Ann</i>

Figure 5.3 Tables representing the *Student2* and *StudentTutorCourse2* relations



You met **join** in Subsection 3.2.

EXERCISE 5.13

Write down a table representing the relation resulting from evaluating the following expression:

StudentTutorCourse2 **join** *Student2*

Beware! In general, taking a relation, projecting it to yield two relations and then joining those two relations, does not always return the original relation, as the following example illustrates.



EXERCISE 5.14

Given the relation *StudentTutorCourse* represented by the table in Figure 5.1, write down tables depicting the relations which result when the following expressions are evaluated:

- (i) **project** *StudentTutorCourse* **over** *StudentId*, *StudentName*, *CourseCode*
- (ii) **project** *StudentTutorCourse* **over** *CourseCode*, *TutorId*, *TutorName*
- (iii) (**project** *StudentTutorCourse* **over** *StudentId*, *StudentName*, *CourseCode*) **join** (**project** *StudentTutorCourse* **over** *CourseCode*, *TutorId*, *TutorName*)

The decomposition of the relation *StudentTutorCourse* into the relations depicted by the two tables of Solution 5.14 (i) and (ii) is called a **lossy decomposition**, in that the join of the projected relations *loses information* (as opposed to losing tuples) from the original relation. Specifically, the information that is lost is that regarding which tuples *aren't* in the original relation.

The opposite of a lossy decomposition is a **non-loss decomposition**. We saw an example of this above where we decomposed *StudentTutorCourse* into the relations *StudentTutorCourse2* and *Student2*, and saw that joining these two relations yielded the original *StudentTutorCourse*.

A theorem of I.J. Heath establishes that when we decompose a relation *R* into relations *S* and *T* by projecting *R*:

- ▶ first, over the left- and right-hand sides of a functional dependency to give *S*,
- ▶ and then over all the attributes of *R* except those which form the right-hand side of the FD, to give *T*,

we are guaranteed a non-loss decomposition. In the example of *StudentTutorCourse*, *S* is *Student2*, the result of projecting *StudentTutorCourse* over the left- and right-hand sides of the FD *StudentId* $\mapsto$ *StudentName*, and *T* is *StudentTutorCourse2*, the result of projecting *StudentTutorCourse* over all its attributes except *StudentName*. Heath's theorem guarantees the result of Exercise 5.13, that this is a non-loss decomposition.

I.J. Heath,
'Unacceptable file
operations in a
relational database',
*Proc. 1971 ACM
SIGFIDET Workshop
on Data Description,
Access and Control*.

EXERCISE 5.15

- (i) Using Solution 5.12(i), decompose *Appointment* into two relations which, when joined, yield *Appointment*.
- (ii) Using Solution 5.12(ii), decompose *Appointment* into three relations which, when joined, yield *Appointment*.
- (iii) Establish that the three relations in (ii) are in 2NF.



If we look at the table in Figure 5.3 depicting the relation *StudentTutorCourse2*, we see that even though the relation is in 2NF – we leave this as an exercise for the keen reader to establish (the argument is as in Solution 5.15(iii)) – we still have redundancies: we are told twice that tutor T1 is called Ann. And this is due to the FD *TutorId* $\mapsto$ *TutorName*, which has nothing directly to do with the primary key. Relations which do not permit this type of FD are said to be in third normal form, as we shall now discuss.

5.4 Third normal form (3NF)

Intuitively, we want to eliminate functional dependencies where the determinant is not the primary key or part thereof. Such dependencies can result in redundant data, as we have seen in the example of *StudentTutorCourse2* above.

Consider the offending FD:

$$TutorId \mapsto TutorName$$

Since *TutorId* (like all attributes) is functionally dependent on the primary key *StudentId*, *CourseCode*, we have

$$StudentId, CourseCode \mapsto TutorId$$

and hence we can derive

$$StudentId, CourseCode \mapsto TutorName$$

by transitivity.

A non-primary key attribute is an attribute which does not form (part of) the primary key.

We can eliminate redundancy arising from such FDs by stipulating that no non-primary key attribute can be derived transitively from the primary key. This leads us to the following (tentative) definition of 3NF: R is in 3NF if for any non-primary key attribute A and primary key P of a relation R , there is no set of attributes B of R such that $P \twoheadrightarrow B$ and $B \twoheadrightarrow A$.

In the example being considered, there *is* such a B (*TutorId*) and so *StudentTutorCourse2* is not in 3NF according to our tentative definition.

But this first idea causes problems with relations having more than one candidate key. For example, in the *Appointment* relation of Figure 5.2, we have two candidate keys, *PatientId*, *ApptDate* and *ConsId*, *ApptDate*, *ApptTime*. If we take the first of these as the primary key, then *any* non-primary attribute A may be derived using transitivity on

$PatientId, ApptDate \twoheadrightarrow ConsId, ApptDate, ApptTime$

and

$ConsId, ApptDate, ApptTime \twoheadrightarrow A$

Hence, if a relation has more than one candidate key, then any non-primary key attribute can be derived transitively from the primary key via an alternate key. We certainly don't want to rule out FDs of the form $AlternateKey \twoheadrightarrow A$, so we amend our definition by adding the stipulation that B cannot be an alternate key.

Exercise 5.16 explores a property of alternate keys, which enables us to determine when B is not an alternate key.

Subsection 2.4 discussed candidate, primary and alternate keys.



EXERCISE 5.16

Suppose that P is the primary key and B an alternate key of a given relation R . Do we have $B \twoheadrightarrow P$?

In general, if there's any attribute X such that $B \twoheadrightarrow X$ does not hold, then B cannot be an alternate key.

Solution 5.16 tells us that if P is the primary key of a given relation and $B \twoheadrightarrow P$ does *not* hold, then B cannot be an alternate key.

So, in order to eliminate the redundancies arising from FDs like $TutorId \twoheadrightarrow TutorName$, we need to rule out the following situation for any non-primary key attribute A and primary key P :

There is a set of attributes B of R where B is not an alternate key, and $P \twoheadrightarrow B$, $B \twoheadrightarrow A$ both hold.

Or, given the result of Exercise 5.16:

There is a set of attributes B of R where $B \twoheadrightarrow P$ does not hold, and $P \twoheadrightarrow B$, $B \twoheadrightarrow A$ both hold.

This situation – which, recall, we do *not* want to hold if we wish to reduce redundancies – is called **transitive dependency**.

A formal definition of transitive dependency is as follows:

An attribute A is **transitively dependent** (TD) on a set of attributes X in a relation R if there is a set of attributes Y such that all the following properties hold:

TD(i) $X \twoheadrightarrow Y$ and $Y \twoheadrightarrow A$.

TD(ii) It is not true that $Y \twoheadrightarrow X$.

TD(iii) A is not an attribute of either X or Y .

As explained above, we include TD(ii) to rule out the situation where Y is an alternate key.

EXERCISE 5.17

Why do you think we include TD(iii) in our definition of transitive dependency? That is, which situations is condition TD(iii) designed to rule out?



Beware! A can be derived by transitivity from $X \mapsto Y$ and $Y \mapsto A$ and yet *not* be transitively dependent on X . This might happen when X and Y are both candidate keys, as we have seen in our discussion of *Appointment*.

We are now in a position to properly define third normal form.

A relation is in third normal form (3NF) if and only if it is in 2NF and no non-primary key attribute is transitively dependent on the primary key.

So, for example, in *StudentTutorCourse2*(StudentId, CourseCode, TutorId, TutorName), as shown in Figure 5.3, *TutorName* is transitively dependent on *StudentId*, *CourseCode*, because:

$StudentId, CourseCode \mapsto TutorId$ and $TutorId \mapsto TutorName$, so TD(i) is satisfied.

It is not true that $TutorId \mapsto StudentId, CourseCode$ (consider T1 in Figure 5.3), so TD(ii) is satisfied.

TutorName is not an attribute of either *TutorId* or *StudentId, CourseCode*, so TD(iii) is satisfied.

The presence of this transitive dependency establishes that *StudentTutorCourse2* is not in 3NF.

In a similar fashion to the way we decomposed a relation which was not in 2NF into a set of relations which were in 2NF (refer to the discussion following Exercise 5.12, and Exercise 5.15), we can decompose a relation which is not in 3NF into relations which are in 3NF by 'stripping out' the offending FDs, as in Exercise 5.18, for example.

In searching the database literature, the author came across at least three non-equivalent definitions of transitive dependency and 3NF. This is the one used on other OU courses, and is also in David Maier (1983) *The theory of relational databases*, Pitman.

By Heath's theorem, we know this is a non-loss decomposition: no information will be lost.

EXERCISE 5.18

We want to project *StudentTutorCourse2*(StudentId, CourseCode, TutorId, TutorName) in Figure 5.3 over subsets of its attributes to form relations *R1* and *R2* which are in 3NF.

If *R1* is *Tutor3*(TutorId, TutorName), what is *R2*?



The next exercise builds on Solution 5.15(ii).

EXERCISE 5.19

- (i) Why is *Appointment2'*(ConsId, ApptDate, ApptTime, PatientId, PatientName) not in 3NF?
- (ii) Decompose *Appointment2'* into two relations which are both in 3NF, and which can be joined to yield the original relation.



This example is adapted from Levene, M. and Loizou, G. (1999) *A guided tour of relational databases and beyond*, Springer.

Usually, a relation in 3NF will have no redundancies – but this isn’t invariably true. For example, consider the table in Figure 5.4, depicting a relation *Address*.

Street	City	Postcode
Hampstead Way	London	NW11
Falloden Way	London	NW11
Oakley Gardens	London	N8
Gower Street	London	WC1E
Gower Street	Bolton	BL1
Amhurst Road	London	E8

Figure 5.4 Table representing the *Address* relation

We shall assume that a street and a city together determine a unique postcode and take *Street*, *City* as the primary key of *Address*. We shall also assume that a postcode determines a unique city.



EXERCISE 5.20

- (i) Is there any redundant data in *Address*?
- (ii) Can any single attribute be a primary key of *Address*?
- (iii) Find an alternate key for *Address*

Now, to examine the potential redundancy in Figure 5.4, let’s look at the non-trivial FDs, based on the above assumptions. The non-trivial FDs are:

FD1: *Street*, *City* $\mapsto$ *Postcode*

FD2: *Postcode* $\mapsto$ *City*

Address is in 2NF (because *Postcode* is not functionally dependent on either *City* or *Street*) and is in 3NF (*City* is not transitively dependent on the primary key as TD(iii) is violated – *City* is an attribute of the primary key). So, here is an example of a table being in 3NF and still exhibiting redundancy.

5.5 Boyce–Codd normal form (BCNF)

In this subsection, we introduce a normal form which eliminates almost all redundancies in our relational tables: **Boyce–Codd normal form (BCNF)**.

A relation is in Boyce–Codd normal form (BCNF) if and only if each irreducible determinant of a non-trivial FD is a candidate key.

Recall that

a determinant of an FD $A \mapsto B$ is irreducible if there is no proper subset S of A such that $S \mapsto B$ holds,

and that

a trivial FD is one in which the right-hand side is a subset of the determinant, that is, it is of the form $A, B \mapsto A$ or $A \mapsto A$.

For example, the relation *Address* as depicted in Figure 5.4 is not in BCNF as the determinant of $Postcode \mapsto City$ is not a candidate key.

Edgar Codd (1923–2003), working at IBM, was the originator of relational databases. Raymond F. Boyce did pioneering work in the development of SEQUEL, the predecessor of SQL.

EXERCISE 5.21

Consider the relation $STC(StudentId, CourseCode, EnrolmentDate, TutorId)$, based on the University model but with the additional constraint that a tutor can only tutor on a single course.

Hence, the FDs represented by STC are as follows:

$StudentId, CourseCode \mapsto EnrolmentDate$

$StudentId, CourseCode \mapsto TutorId$

$TutorId \mapsto CourseCode$

Establish that STC is in 3NF but *not* BCNF.



It might appear at first sight that $StudentId, CourseCode \mapsto TutorId$ and $TutorId \mapsto CourseCode$ together give a transitive dependency. They don't, because TD(iii) is violated.

EXERCISE 5.22

Determine if either of the following relations, as seen in Solution 5.19, are in BCNF, justifying your answers.

(i) $Appointment3'(ConsId, ApptDate, ApptTime, PatientId)$

(ii) $Patient3(PatientId, PatientName)$

We can use our usual 'stripping out offending FDs' non-loss decomposition method to decompose a relation that isn't in BCNF into relations which are.

EXERCISE 5.23

Decompose the $Address$ relation, as in Figure 5.4, into two relations which are in BCNF and which, when joined, will yield $Address$.



You may be concerned that in decomposing $Address$ in Exercise 5.23, the FD $Street, City \mapsto Postcode$ no longer holds in either relation. We thus have to write a constraint which explicitly satisfies the SVF that each street in each city has exactly one postcode.

You should be aware that, given a relation, you don't have to go through 2NF and 3NF in order to find BCNF relations which, when joined, yield the original relation. You could just use the technique which we have (very informally) described as 'stripping out the offending FDs', where the offending FDs are those with determinants which are not candidate keys.

Some limitations of BCNF

The first limitation of BCNF is that it does not eliminate *all* redundancies. It can be shown that BCNF *does* guarantee no redundancies caused by multiple occurrences of single-valued facts.

However, not all redundancies arise from multiple occurrences of single-valued facts. For example, consider the table in Figure 5.5 (overleaf), representing the relation $SmallStudentTutorCourse$ with primary key $StudentId, CourseCode$ and the following natural language predicate:

$\langle a, b, c \rangle$ is a tuple of $SmallStudentTutorCourse$ if and only if student a enrolled on course b has tutor c .

SmallStudentTutorCourse

<i>StudentId</i>	<i>CourseCode</i>	<i>TutorId</i>
<i>S1</i>	<i>C1</i>	<i>T1</i>
<i>S1</i>	<i>C2</i>	<i>T2</i>
<i>S1</i>	<i>C3</i>	<i>T1</i>
<i>S2</i>	<i>C1</i>	<i>T3</i>
<i>S3</i>	<i>C3</i>	<i>T1</i>
<i>S4</i>	<i>C1</i>	<i>T1</i>

Figure 5.5 Table representing *SmallStudentTutorCourse*

There is only one non-trivial FD in *SmallStudentTutorCourse*, namely *StudentId*, *CourseCode* $\mapsto$ *TutorId*. The determinant of this FD is the primary key and hence the relation is in BCNF. However, there is redundant data – we are told more than once that T1 tutors on C1 and that T1 tutors on C3. There is no contradiction here, as neither the property of a tutor tutoring a course nor the property of a course having a tutor are single-valued (for example, T1 tutors on C1 and C3; C1 is tutored by T1 and T3).

Another limitation of BCNF is that it doesn't preclude insertion or deletion anomalies in a few cases.

Because of these limitations, there is ongoing interest in defining and investigating other normal forms. In the literature, you may see references to 4NF, 5NF, 6NF, DKNF (domain/key normal forms), optimal and object normal forms.

Many of these are based on the concept of join dependencies (non-loss decompositions) rather than functional dependencies, as is the case for 2NF, 3NF and BCNF. We feel, however, that 2NF, 3NF and BCNF suffice for a first consideration of normalisation, especially since BCNF suffices to prevent redundancy and eliminate insertion and deletion anomalies in almost all cases.

5.6 Summary

In this section, we looked more closely at some theoretical aspects of data modelling, motivating our discussion by considering the insertion, amendment and deletion anomalies which might occur in poorly designed relations. We then considered how to analyse the dependencies (single-valued facts) between different data attributes by way of functional dependencies. We established that redundant repetitions of single-valued facts are eliminated in relations which are in Boyce–Codd normal form (BCNF) (where every determinant is a candidate key). We introduced first normal form (1NF) as a historical footnote, and second (2NF) and third (3NF) normal forms as (not strictly necessary) staging posts to BCNF, noting that each of 2NF and 3NF eliminates a particular cause of redundancy. Finally, we noted that BCNF has some limitations and that other normal forms have been defined.

LEARNING OUTCOMES

Having studied this section, you should now be able to:

- ▶ Explain what is meant by insertion, deletion and amendment anomalies in relations.
- ▶ Understand the terms 'single-valued fact' and 'functional dependency' and the correspondence between them.
- ▶ Transform single-valued facts into functional dependencies, and extend the set of functional dependencies on a given relation using the properties of transitivity and augmentation.
- ▶ Recognise when a relation is in second normal form (2NF), understand the connection between 2NF and the elimination of a cause of data redundancy, and be able to decompose a relation which is not in 2NF into a set of 2NF relations by means of non-loss decomposition.
- ▶ Understand the term 'transitive dependency'.
- ▶ Recognise when a relation is in third normal form (3NF), understand the connection between 3NF and the elimination of a cause of data redundancy, and be able to decompose a relation in 2NF into a set of 3NF relations by means of non-loss decomposition.
- ▶ Recognise when a relation is in Boyce–Codd normal form (BCNF) and understand that BCNF guarantees no repetition of occurrences of single-valued facts, and be able to decompose a relation which is not in BCNF into a set of BCNF relations by means of non-loss decomposition.

Block summary

In Section 1 we discussed how this block fits in with the rest of the course, and in Sections 2 to 4 we introduced you to the theory underpinning relational databases.

If you are more interested in implementation than in theory, you may view the theory as:

- ▶ A staging post between a conceptual model, such as an E–R model, and an implementation of a relational database as a set of tables.
- ▶ The underpinning of an implementation. For example, the way a DBMS optimises queries (so that a user can get information in a reasonable time) is by using equivalent relational algebra expressions.
- ▶ Providing goals to which an implementation should aspire. For example, the ideal DBMS should enable constraints of all the types discussed in Section 4.

In any case, we advise you to take cognisance of the following quote from Leonardo da Vinci:

As quoted in C.J. Date (2005) *Database in depth: relational theory for practitioners*, O'Reilly.

Those who are enamoured of practice without theory are like a pilot who goes into a ship without rudder or compass and never has any certainty where he is going. Practice should always be based upon a sound knowledge of theory.

Finally, as we have just seen, Section 5 had a slightly different focus, concentrating on how relations might be designed so as to reduce the occurrence of redundant data.

Looking forward, *Block 3* addresses implementation issues and *Block 4* looks at issues of practical database design.

Solutions to Exercises

SOLUTION 2.1

There are 17 occurrences of the **Enrolment** entity type represented in the *Enrolment* relation in Figure 2.1, since each row represents a distinct occurrence. There is no significance in the order in which the rows are printed – they could be printed in any order and still depict the same relation.

SOLUTION 2.2

According to the convention adopted in this course, *Student* is the name of a relation.

SOLUTION 2.3

Whether or not Figure 2.3 depicts a relation depends on how the domain of the *Tutor* column has been defined. If it has been defined as the set of all character strings, then all the values in this column come from this set, and the table may depict a relation. If, however, it has been defined as (say) sets of strings of four numerals, then the value ‘Jennings’ does not come from this set and so the table does not depict a relation.

In the table in Figure 2.4, two of the rows have no values for the attribute *Tutor*. According to Rule 2, this table cannot depict a relation.

SOLUTION 2.4

You can refer to a row by means of the value of the primary key in that row (since each value of the primary key determines just one row); you can refer to a column by its name, as in Rule 3.

SOLUTION 2.5

The relation as depicted in Figure 2.8 has degree 4 (four attributes) and cardinality 3 (three tuples).

SOLUTION 2.6

Table term	Equivalent relational term
Column name	Attribute name
Column entries	Values of attributes
Row	Tuple
Number of columns	Degree
Number of rows	Cardinality

SOLUTION 2.7

A relation is an abstract concept consisting of a set of tuples of attribute values in any order. Provided that it obeys the rules above, a table might be a concrete depiction of a relation.

SOLUTION 2.8

ShortRegion(RegionNumber, Address, Telephone, EmailAddress)

NB: do **not** forget to underline the primary key.

SOLUTION 2.9

$\langle a, b, c, d \rangle$ is a tuple of *ShortRegion* if and only if a region identified by *RegionNumber* 'a' has address 'b', telephone number 'c' and email address 'd'.

SOLUTION 2.10

It is sensible to define different domains for *Locations* and *TitlesOfCourses*, even though they are the same set, to emphasise their difference in meaning. It is extremely unlikely that you will ever want to compare the name of a place with the name of a course.

SOLUTION 2.11

No – we can't compare the two values of staff number and telephone number, as they come from different domains.

SOLUTION 2.12

Working downwards in the depiction of the relation in Figure 2.10, the first three tuples in the depiction are legal. The next one is illegal because the value of *CourseCode*, c10, is not permitted by the definition of the domain *CourseCodes*. There is a clash between the second and last tuples as depicted, as we can only have one tuple with primary key (s07, c4).

SOLUTION 2.13

In addition to the information included in the relational heading described in Subsection 2.1, the relation declaration in Figure 2.11 also defines the domains of each attribute, that is, where each attribute derives its values, thus constraining the values. We shall see later that relation declarations may contain further information.

SOLUTION 2.14

relation *Region*
RegionNumber: *RegionNumbers*
Address: *Addresses*
Telephone: *TelephoneNumbers*
EmailAddress: *EmailAddresses*
primary key *RegionNumber*

SOLUTION 2.15

The candidate keys are *StaffIdentifier* and *NationalInsuranceNumber*. Either one could be chosen as the primary key (though you might prefer to choose *StaffIdentifier* as that is under the University's control), and the other one becomes the alternate key.

SOLUTION 2.16

With *ProgrammerId* as an alternate key, there is a 1:1 mapping between tasks and programmers. That is, a particular programmer can occur in only one tuple of the relation – a programmer is associated with only one task, and a task is associated with only one programmer.

If no alternate key is declared, then only statement (ii) is true. Statement (i) is false because every task has only one tuple associated with it and any attribute in that tuple can have only one value; statement (iii) is false because statement (i) is.

SOLUTION 2.17

In *Appointments1*, a patient can only have a single appointment on a given date (and that appointment is at a particular time with a particular consultant); in *Appointments2*, a patient can have multiple appointments on a given date, each potentially with a different consultant.

We can deduce this from the declaration of the primary keys. In the first case, a particular patient and date identifies only one tuple, for which there can be only a single appointment time. In the second case, there can be multiple appointment times for a particular patient on a particular date – each particular patient, date and time identifies a unique tuple.

SOLUTION 2.18

Statements (i) and (ii) are true – a relation must have a (unique) primary key and this is a candidate key. Statements (iii) and (iv) are false – a relation can have more than one candidate key, but if it has only one, then this is the primary key and there is no alternate key.

SOLUTION 2.19

<s42, Reddick, '23 Kestrel Lane, Dudley', dave@belwise.fake.co.uk, 'Apr 23, 2002', 2>

SOLUTION 2.20

Posting the primary key of *Student* into *Region* is impossible because a particular region may manage more than one student – for example, in Figure 2.15, region 1 manages both students s22 and s38. If we had posted the primary key of *Student* into *Region*, the corresponding tuple of *Region* for region 1 would be <1, '57, Longboat Street, Birmingham', 0120 779165, region1@open.fake.address, s22, s38>, which is illegal – remember from Rule 1 in Subsection 2.1 that an attribute of a relation can only take a single value in each tuple, and there is only one tuple representing region 1 as *RegionNumber* is a primary key.

SOLUTION 2.21

(a) *WardA*(*WardNo*, *WardName*)

PatientA(*PatientId*, *PatientName*, *WardNo*)

(b)

relation *WardA*

WardNo: *WardNos*

WardName: *WardNames*

primary key *WardNo*

relation *PatientA*

PatientId: *PatientNumbers*

PatientName: *PatientNames*

WardNo: *WardNos*

primary key *PatientId*

{mandatory participation of *PatientA* in the *OccupiedBy* relationship}

foreign key *WardNo* **references** *WardA*

SOLUTION 2.22

Because the relationship *OccupiedBy* is 1:*n* from **WardA** to **PatientA**, one ward potentially has many patients – but in any tuple of *WardA*, every attribute must have only one value. That is, we can't have a tuple like <w2, Wessex, p01 p15 p31>. In addition, since participation of **WardA** in *OccupiedBy* is optional, there may be a ward with no patients – and in each tuple of *WardA*, every attribute must have a value.

SOLUTION 2.23

Student is the referencing relation; *Region* is the referenced relation.

Note that we use comments in the relational model to refer back to constraints in the conceptual model.

SOLUTION 2.24

relation *Enrolment**StudentId*: *StudentIds**CourseCode*: *CourseCodes**EnrolmentDate*: *Dates***primary key** (*StudentId*, *CourseCode*){mandatory participation of *Enrolment* in *EnrolledIn* relationship}**foreign key** *StudentId* **references** *Student*{mandatory participation of *Enrolment* in *StudiedBy* relationship}**foreign key** *CourseCode* **references** *Course*

SOLUTION 2.25

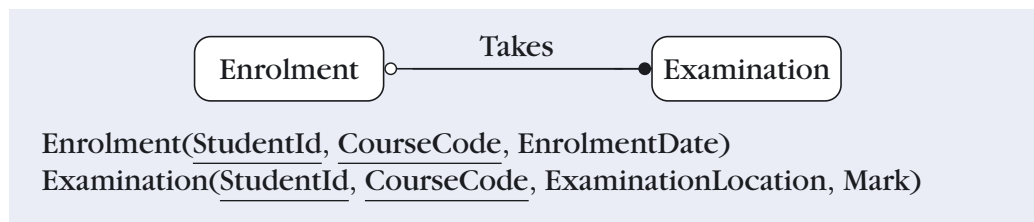
relation *Enrolment**StudentId*: *StudentIds**CourseCode*: *CourseCodes**EnrolmentDate*: *Dates**Mentor*: *StudentIds***primary key** (*StudentId*, *CourseCode*){mandatory participation of *Enrolment* in *EnrolledIn* relationship}**foreign key** *StudentId* **references** *Student*{mandatory participation of *Enrolment* in *StudiedBy* relationship}**foreign key** *CourseCode* **references** *Course*{mandatory participation of *Enrolment* in *Mentors* relationship}**foreign key** *Mentor* **references** *Student***relation** *Student**StudentId*: *StudentIds**Name*: *Names**Address*: *Addresses**EmailAddress*: *EmailAddresses**RegistrationDate*: *Dates**RegionNumber*: *RegionNumbers***primary key** *StudentId*

The point here is that the student who mentors another student on a particular enrolment is not the same person as this other student – and so has to be represented by an explicit foreign key in the *Enrolment* relation.

SOLUTION 2.26

Since the participation of **Doctor** in **HeadedBy** is optional, only (ii) is allowable. (i) is not allowable because not all doctors head teams – not all doctor tuples are associated with tuples in the *Team* relation. If a doctor doesn't head a team, then there will be no value for *TeamCode* in that doctor's tuple, which is illegal.

SOLUTION 2.27



Since (StudentId, CourseCode) is the identifier of both **Enrolment** and **Examination**, a particular value of this pair in **Enrolment** can only match with one pair in **Examination**, as shown below – thus **Takes** must be 1:1.

Enrolment		Examination	
(StudentID, CourseCode)		(StudentID, CourseCode)	
s07	c4	s07	c4
s09	c4	s09	c4

Takes

Some occurrences of the **Takes** relationship

As to the participation conditions, we know that the participation of **Examination** in **Takes** must be mandatory, since the foreign key of every tuple in **Examination** must have a value. In the absence of any information about **Enrolment**, we make the participation of **Enrolment** in **Takes** optional – the default participation condition.

Note that **Examination**, like **Enrolment**, is a weak entity type: it wouldn't exist without the existence of **StudentId** and **CourseCode**.

SOLUTION 2.28



Note that because *PatientID* is the primary key of *AnotherOccupiedBy*, a tuple in *PatientA* can match with at most one tuple in *AnotherOccupiedBy* (and vice versa) – so this matching is 1:1. *WardNo* is **not** the primary key, nor an alternate key, in *AnotherOccupiedBy*, so a tuple in *WardA* may match with many tuples in *AnotherOccupiedBy*. As before, where we have no information about the participation condition of an entity type in a relationship, we settle for the default condition, which is optional.

SOLUTION 2.29

We need to represent **ExaminedBy** by a relation as below.

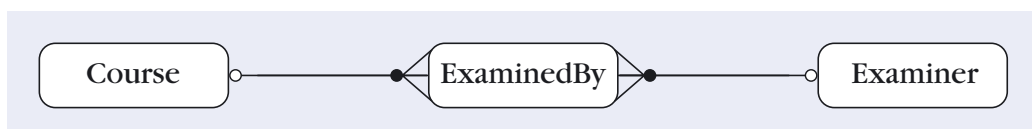
relation *ExaminedBy*
CourseCode: *CourseCodes*
StaffNo: *StaffNos*
primary key (*CourseCode*, *StaffNo*)
foreign key *CourseCode* **references** *Course*
foreign key *StaffNo* **references** *Examiner*

relation *Course*
CourseCode: *CourseCodes*
Title: *TitlesOfCourses*
Credit: *Credits*
primary key *CourseCode*

relation *Examiner*
StaffNo: *StaffNos*
Name: *Names*
primary key *StaffNo*

Note that because the relationship is many-to-many, the primary key in *ExaminedBy* is the pair of attributes (*CourseCode*, *StaffNo*).

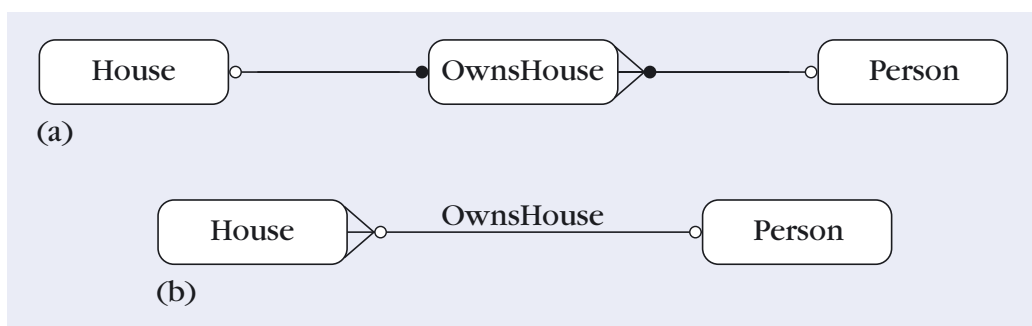
SOLUTION 2.30



Note that this diagram is equivalent to that of Exercise 2.29.

SOLUTION 2.31

(i)



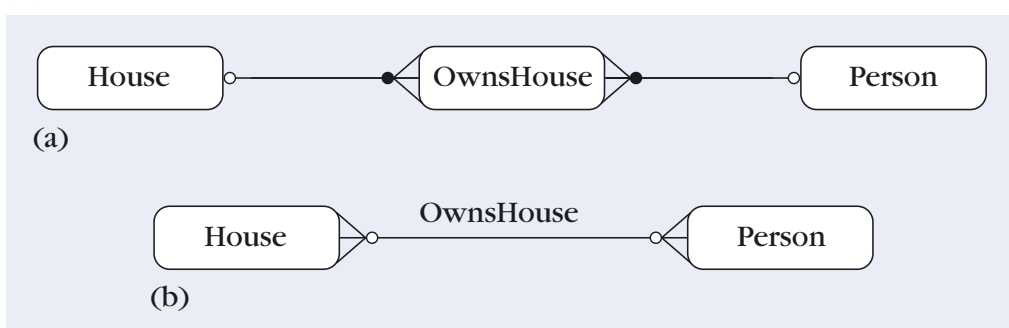
OwnsHouse is not an intersection relation since it doesn't represent an $m:n$ relationship. As we can see from the second diagram, it represents a $1:n$ relationship.

(ii)



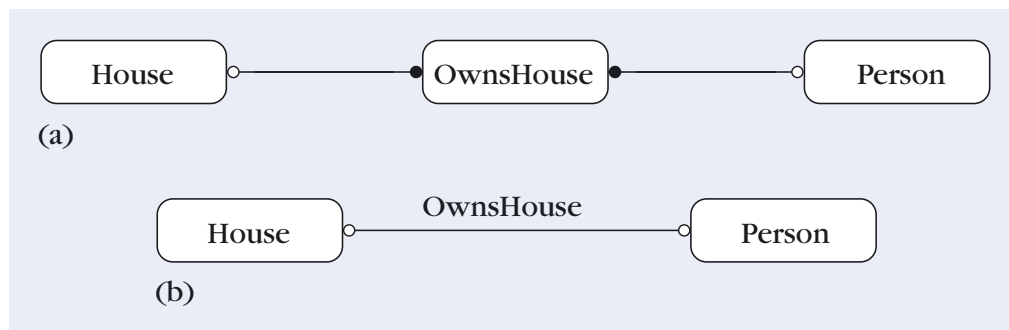
Here, we have to keep the entity type **OwnsHouse** because it records information (**WhenLastSold**) which isn't recorded elsewhere. That is, the fragment of relational representation given cannot be represented by an E-R diagram having only two entities.

(iii)



Here, *OwnsHouse* is an intersection relation, since it exists only to represent an $m:n$ relationship.

(iv)



OwnsHouse is not an intersection relation, as it represents a 1:1, rather than an $m:n$, relationship.

SOLUTION 2.32

- (i) The set of attributes of *C* must be the combination of the primary keys of the relations representing *A* and *B* – no other attributes are possible (since the role of *C* is simply to represent the relationship – the association between occurrences of *A* and *B*).
- (ii) No, the primary key of *C* is not always a combination of the primary keys of the relations representing *A* and *B* – see, for example, Exercise 2.31(i) and its corresponding solution.
- (iii) If the primary key of *C* is a combination of the primary keys of the relations representing *A* and *B*, then the relationship must be $m:n$ (and so *C* is an intersection relation) – one occurrence of *A* must be associated with many of *B*, and one occurrence of *B* must be associated with many of *A*. Otherwise, if it were $1:n$, so that (for example) one occurrence of *A* may be associated with many of *B* but one occurrence of *B* is associated with at most one of *A* (as in Figure 2.25, with *A* being the entity type *WardA* and *B* being the entity type *PatientA*), then the primary key of the relation representing *B* would be the primary key of *C*. A similar argument holds for a 1:1 relationship.

SOLUTION 2.33

relation *Nurse*

StaffNo: *StaffNos*

NurseName: *Names*

primary key *StaffNo*

relation *Supervises*

StaffNo: *StaffNos*

Supervisor: *StaffNos*

primary key *StaffNo*

foreign key *StaffNo* **references** *Nurse*

foreign key *Supervisor* **references** *Nurse*

Because *Supervises* is a relationship with optional participation at the $:n$ (many) end, we must represent it by a relation, as in the example of *AnotherOccupiedBy* discussed at the beginning of this subsection.

SOLUTION 2.34

Relationship	Method of representing the relationship	Any aspect of the relationship not represented?
(i) 	Foreign key in the relation representing B	The mandatory participation of A in R
(ii) 	Relation for relationship	No
(iii) 	Foreign key in the relation representing B	No
(iv) 	Relation for relationship	No
(v) 	Foreign key in the relation representing B . May need to declare an alternate key.	No
(vi) 	Foreign key in either the relation representing A or that representing B . May need to declare an alternate key.	Mandatory participation in R of whichever entity type is not represented by the relation which includes the foreign key
(vii) 	Relation for relationship	No
(viii) 	Relation for relationship	Mandatory participation of A in R

We should point out that in those relationships of Solution 2.34 where we have stipulated the straightforward ‘Foreign key’ method of representation, there is no technical reason why we couldn’t just as well have used the ‘Relation for relationship’ method. We chose not to in the interests of economy, because the latter representation method introduces a new relation and the former does not. Where we have stipulated the ‘Relation for relationship’ method in Solution 2.34, there is no choice – the straightforward foreign key mechanism does not work, for reasons that we have explained in Subsections 2.5 and 2.6.

SOLUTION 3.1

Here, the selection condition is that the enrolment date must be after June 1, 2004 but before November 1, 2004.

<i>StudentId</i>	<i>CourseCode</i>	<i>EnrolmentDate</i>
<i>s05</i>	<i>c2</i>	<i>Jun 4, 2004</i>
<i>s05</i>	<i>c7</i>	<i>Oct 18, 2004</i>
<i>s10</i>	<i>c7</i>	<i>Jun 20, 2004</i>

The order of the rows in the table is irrelevant, of course, since the table represents a relation.

SOLUTION 3.2

select *Enrolment* **where** *EnrolmentDate* < ‘Sep 1, 2004’ **or** *EnrolmentDate* > ‘Jan 1, 2005’

SOLUTION 3.3

select *GeneralPractitioner* **where** *GPName* = *SecName*

SOLUTION 3.4

The operands of the comparison operators must be from the same domain (see Subsection 2.2). This means that GP names and secretary names cannot be defined over different domains if we wish to write expressions such as that of Solution 3.3.

SOLUTION 3.5

<i>StudentId</i>
<i>s01</i>
<i>s02</i>
<i>s05</i>
<i>s07</i>
<i>s09</i>
<i>s10</i>
<i>s22</i>
<i>s38</i>
<i>s46</i>
<i>s57</i>

SOLUTION 3.6

The result of applying **project** must be a relation because of the closure property of relational operators and a table depicting a relation cannot have duplicate rows.

SOLUTION 3.7

The innermost expression, **project** *Student* **over** *Name*, will evaluate to give a relation with the single attribute *Name* – so there will be no attribute *RegionNumber* for the selection condition.

SOLUTION 3.8

Here, we can either apply **project** first and then **select** as in the following:

select (**project** *Student* **over** *Name*, *Address*, *RegistrationDate*) **where** *RegistrationDate* > 'Jan 1, 2004'

Or **select** first and then **project**, as in:

project (**select** *Student* **where** *RegistrationDate* > 'Jan 1, 2004') **over** *Name*, *Address*, *RegistrationDate*

SOLUTION 3.9

SmallEnrolment **join** *Examination*

<i>StudentId</i>	<i>CourseCode</i>	<i>EnrolmentDate</i>	<i>ExaminationLocation</i>	<i>Mark</i>
s05	c2	Jun 4, 2004	Bath	57
s07	c4	Dec 12, 2004	Bedford	85
s09	c4	Dec 16, 2004	Taunton	63
s09	c2	Dec 18, 2004	New York	56

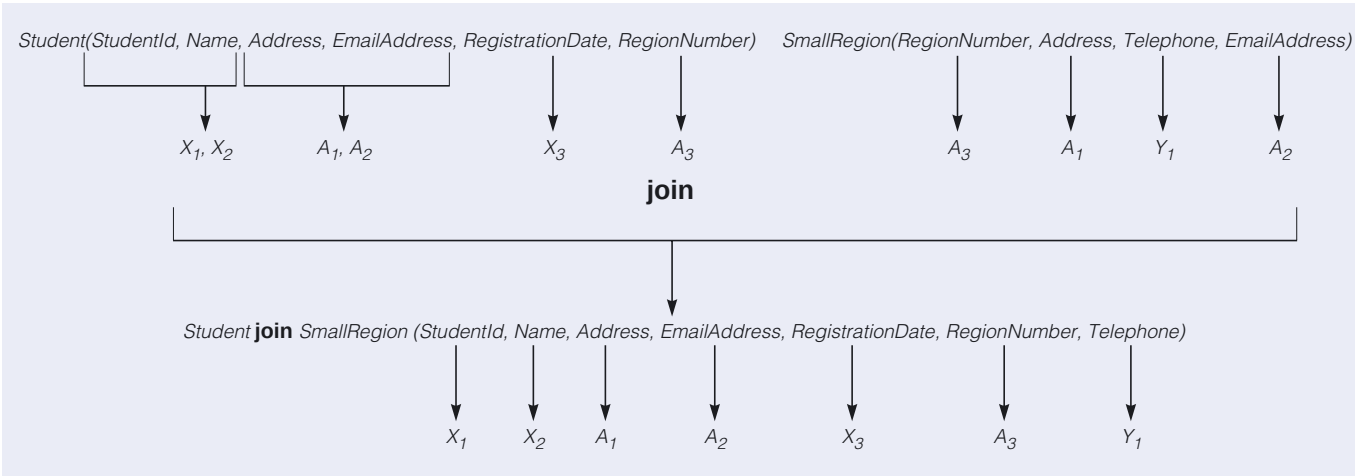
As usual, the orders of the rows and columns have no significance.

SOLUTION 3.10

The following table (with empty body) depicts the relation *Student* **join** *SmallRegion*:

<i>StudentId</i>	<i>Name</i>	<i>Address</i>	<i>EmailAddress</i>	<i>RegistrationDate</i>	<i>RegionNumber</i>	<i>Telephone</i>
------------------	-------------	----------------	---------------------	-------------------------	---------------------	------------------

The joining together of the relational headings is illustrated below, using the notation of Figure 3.6, with *A*₁, *A*₂ and *A*₃ labelling the common attributes.



Of course, this illustration is for explanatory purposes only – it is not part of the required solution.

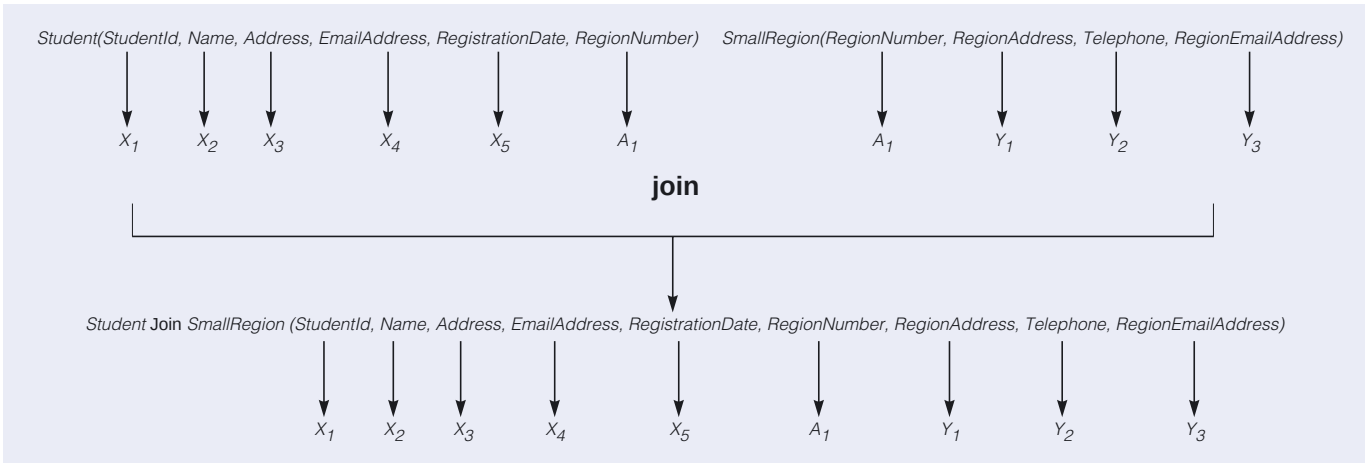
The relation is empty – has no body – as the set of common attributes is {*Address*, *EmailAddress*, *RegionNumber*}, and no student has the same address and email address as the region managing them.

SOLUTION 3.11

(a) *Student* **join** (*SmallRegion* **rename** (*Address* **as** *RegionAddress*, *EmailAddress* **as** *RegionEmailAddress*))

An alternative solution is to rename the appropriate attributes of *Student* instead of those of *Small Region*.

The heading of this relation is illustrated below:



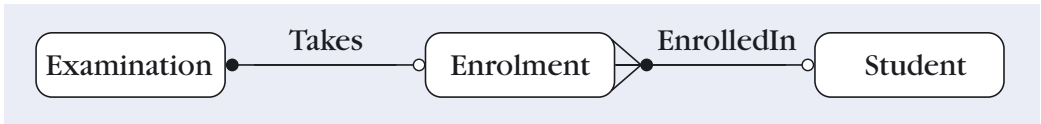
(b) The table resulting from evaluating this expression is as below:

Stud...	Name	Add...	EmailAdd...	Reg...	RegionNum...	RegionAdd...	Tel...	RegionEmailAdd...
s01	Akeroyd	12...	Akers@...	Nov...	3	Block 9...	01670...	region3@...
s07	Gillies	29...	Gillies@....	Dec...	3	Block 9...	01670...	region3@...

(We have left out some of the data in this solution because of space considerations.)

SOLUTION 3.12

You need the relation *Examination*, as in Exercise 3.9, which contains information about the location of the examination, *Student* obviously (Figure 3.3), and *Enrolment* (Figure 3.2) which links the other two relations via the relationships **EnrolledIn** and **Takes**, as shown below.



Relationships between Examination, Enrolment and Student

SOLUTION 3.13

(i) We can join *ExamAndEnrolDetails* and *Student* over their common attribute *StudentId*, as in

StudentExamAndEnrolDetails **alias** (*ExamAndEnrolDetails* **join** *Student*)

This will give the heading

StudentExamAndEnrolDetails(*StudentId*, *CourseCode*, *ExaminationLocation*, *Mark*, *EnrolmentDate*, *Name*, *Address*, *EmailAddress*, *RegistrationDate*, *RegionNumber*)

There is also a solution that requires just two relations, *Student* and *Examination*. *Student* contains the required information about the student and their region number; *Examination* contains the information required about the examination location. There is an association, using the *StudentID* attribute that appears in the relations *Student* and *Examination*, that links the information about a student who takes an examination with the information about the same student in the *Student* relation. Using this direct association avoids the need to use the *Enrolment* relation from the conceptual data model.

(ii) **project (select** *StudentExamAndEnrolDetails* **where** *RegionNumber* = '3' **and** *ExaminationLocation* = 'Bedford') **over** *Name*

(iii) Substituting for *StudentExamAndEnrolDetails* gives

project (select (*ExamAndEnrolDetails* **join** *Student*) **where** *RegionNumber* = '3' **and** *ExaminationLocation* = 'Bedford') **over** *Name*

Then substituting for *ExamAndEnrolDetails* gives

project (select ((*Enrolment* **join** *Examination*) **join** *Student*) **where** *RegionNumber* = '3' **and** *ExaminationLocation* = 'Bedford') **over** *Name*

We should note that the answer given here is not unique: there are other algebraic expressions which are equivalent, that is, which will always evaluate to the same relation and can be depicted by the same table.

SOLUTION 3.14

There are several equivalent solutions. Here are the steps towards constructing one, where we have included comments after the *//* symbols:

Region2Students **alias** (**select** *Student* **where** *RegionNumber* = '2')

// Region2Students is the relation of all the tuples from *Student* where the student is *//* from region 2.

Region2StudentsEnrol **alias** (*Region2Students* **join** *Enrolment*)

// Region2StudentsEnrol gives the enrolment and student details of students from *//* region 2.

Region2Courses **alias** (*Region2StudentsEnrol* **join** *Course*)

// Region2Courses adds the course details of students from region 2 to the existing *//* information by joining the relations over the common attribute *CourseCode*.

project *Region2Courses* **over** *Title*

Now substituting back, first for *Region2Courses* and then for *Region2StudentsEnrol* and *Region2Students*:

project (*Region2StudentsEnrol* **join** *Course*) **over** *Title*

project ((*Region2Students* **join** *Enrolment*) **join** *Course*) **over** *Title*

project (((**select** *Student* **where** *RegionNumber* = '2') **join** *Enrolment*) **join** *Course*) **over** *Title*

SOLUTION 3.15

Joining all the relations gives:

(*Student* **join** *Enrolment*) **join** *Course*

We then want to select just those tuples referencing region 2:

select ((*Student* **join** *Enrolment*) **join** *Course*) **where** *RegionNumber* = '2'

And then project over the titles of courses:

project (select ((*Student* **join** *Enrolment*) **join** *Course*) **where** *RegionNumber* = '2') **over** *Title*

An alternative is
Student **join**
(*Enrolment* **join**
Course)

SOLUTION 3.16

To find Liversage's staff number, we first select Liversage's particulars:

select *Doctor* **where** *DoctorName* = 'Liversage'

Then project these particulars over *StaffNo* (using an alias for the sake of brevity):

LiversageStaffNo **alias** (**project** (**select** *Doctor* **where** *DoctorName* = 'Liversage') **over** *StaffNo*)

This gives the relation as below, with the given data:

LiversageStaffNo

<i>StaffNo</i>
110

We then find all those doctors appraised by Liversage. This involves deriving all those tuples where the value of the attribute *Appraiser* is Liversage's staff number – we need *Appraiser* in *Doctor* and *StaffNumber* in *LiversageStaffNo* to be a common attribute.

This involves a use of **rename**, as below:

Doctor **join** (*LiversageStaffNo* **rename** (*StaffNo* **as** *Appraiser*))

Expanding the alias gives

Doctor **join** ((**project** (**select** *Doctor* **where** *DoctorName* = 'Liversage') **over** *StaffNo*) **rename** (*StaffNo* **as** *Appraiser*))

Given Figure 3.11, this evaluates to

Doctor

<i>StaffNo</i>	<i>DoctorName</i>	<i>Position</i>	<i>Appraiser</i>
131	Kalsi	Consultant	110
156	Hollis	Registrar	110
174	Gibson	Registrar	110

Of course, your solution does not have to include tables since we have only asked for a relation. We have included tables for illustrative purposes only.

SOLUTION 3.17

There are many solutions: here's one.

(i) A **alias** (**project** *Doctor* **over** *StaffNo*, *DoctorName*)

giving

A

<i>StaffNo</i>	<i>DoctorName</i>
110	Liversage
131	Kalsi
156	Hollis
174	Gibson
178	Paxton
389	Wright

Again, note that these tables are provided for illustrative purposes only.

(ii) *B* **alias** (*A* **rename** (*StaffNo as Appraiser, DoctorName as AppName*))
giving

B

<i>Appraiser</i>	<i>AppName</i>
110	Liversage
131	Kalsi
156	Hollis
174	Gibson
178	Paxton
389	Wright

(iii) *Doctor* **join** *B*
giving
Doctor **join** ((*project Doctor over StaffNo, DoctorName*) **rename** (*StaffNo as Appraiser, DoctorName as AppName*))

SOLUTION 3.18

AllParts **alias** (*project SupplyParts over PartId*)

SOLUTION 3.19

divide *SupplyParts* **by** (*project SupplyParts over SupplierId*)

Given the data in Figure 3.12, this will yield a relation with heading (*PartId*) and empty body.

SOLUTION 3.20

StudentCourses **alias** (*project Enrolment over StudentId, CourseCode*)

AllCourses **alias** (*project Course over CourseCode*)

divide *StudentCourses* **by** *AllCourses*

Substituting back, we have

divide (*project Enrolment over StudentId, CourseCode*) **by** (*project Course over CourseCode*)

SOLUTION 3.21

- (i) $A \cup B = \{1, 2, 3, 4, 5, 7, 8, 9\}$, $A \cap B = \{1, 3, 5\}$ and $A - B = \{7, 8, 9\}$.
- (ii) If $C - D$ is empty, then there is no element of C which is not in D , that is, every element of C is in D , so C is a subset of D .
- (iii) $A \cup B$ is the same as $B \cup A$; $A \cap B$ is the same as $B \cap A$; $A - B$ is *not* equal to $B - A$ in general. If you have met the term ‘commutative’ before (and don’t worry if you haven’t), you will see that the set operators union and intersection are commutative, whereas difference is not.

SOLUTION 3.22

No – this is not an allowable relational algebra expression, as *Student* and *Staff* are not union-compatible (and cannot be made to be so using **rename**). For example, *Staff* has an attribute *Telephone* which doesn’t match with any attribute in the relation *Student*, since *Telephone* is defined over the domain *TelephoneNumbers* and no attribute in *Student* is defined over *TelephoneNumbers*.

SOLUTION 3.23

- (i) The first expression involves a **union** as follows:

```
(project (select Student where RegionNumber = '3') over StudentId)
union
(project (select Enrolment where CourseCode = 'c3') over StudentId)
```

The second expression involves a **join**:

```
project (select (Student join Enrolment) where RegionNumber = '3' or
CourseCode = 'c3') over StudentId
```

- (ii) First find the staff numbers of all the doctors and then 'take away' all those doctors who are appraisers (with judicious use of **rename**), as in:

```
(project Doctor over StaffNo) difference ((project Doctor over Appraiser) rename
(Appraiser as StaffNo))
```

SOLUTION 3.24

- (i)

A join B

StudentId	Date
Ashwin	Jan 12, 2005
Beryl	Jun 12, 2005
Carol	Oct 18, 2005
Dave	Dec 12, 2005

A intersection B

StudentId	Date
Ashwin	Jan 12, 2005
Beryl	Jun 12, 2005
Carol	Oct 18, 2005
Dave	Dec 12, 2005

- (ii) Since R and T have the same set of attributes – all attributes in common –
 $R \text{ join } T = R \text{ intersection } T$

SOLUTION 3.25

A times B

StudentId	CourseCode
s01	c2
s02	c2
s05	c2
s07	c2
s01	c4
s02	c4
s05	c4
s07	c4

SOLUTION 3.26

For arbitrary relations A and B , $A \textbf{ times } B = B \textbf{ times } A$. That is, the relational operator **times** is commutative, unlike the corresponding mathematical operator (the Cartesian product). This is because the order of attributes and their corresponding values is immaterial in a relation.

SOLUTION 3.27

StudentId
s01
s02
s05
s07

That is, the relation **divide** ($A \textbf{ times } B$) **by** B is the relation A in this case.

SOLUTION 3.28

project
(**select** ($Enrolment \textbf{ times } (Student \textbf{ rename } (StudentId \textbf{ as } StudentIdent)))$) **where**
 $StudentId = StudentIdent$)
over $StudentId, CourseCode, EnrolmentDate, Name, Address, EmailAddress,$
 $RegistrationDate, RegionNumber$

Well done if you got this correct! It's a considerably harder exercise than you'll meet in the TMAs or examination for this course.

SOLUTION 4.1

- (i) Given that each relation includes the tuple $\langle p01, '27 \text{ Dec, } 2005', 14.30, s13 \rangle$, $\langle p01, '27 \text{ Dec, } 2005', 15.30, s13 \rangle$ is not an allowable tuple in *Appointment1* as it has the same value for the primary key as $\langle p01, '27 \text{ Dec, } 2005', 14.30, s13 \rangle$; it is an allowable tuple for *Appointment2*.
 $\langle p02, '27 \text{ Dec, } 2005', 14.30, s13 \rangle$ is an allowable tuple for both – with the constraints as given, there's nothing to stop a consultant seeing different patients at the same time and date.
 $\langle p01, '11 \text{ Dec, } 2005', 14.30, s13 \rangle$ is an allowable tuple for both.
- (ii) A plausible alternate key, representing the semantics that a consultant cannot see more than one patient at a particular time on a particular date, is the combination (*ConsultantId*, *ApptDate*, *ApptTime*).

SOLUTION 4.2

- (i) When a tuple is deleted from *Student*, then presumably this is equivalent to a student leaving the University. So the cascade effect seems the most appropriate here: all enrolments involving this student should be deleted.
- (ii) Here, when a doctor leaves the hospital, it is plausible that the team remains, but needs a new head. So the default effect is probably most appropriate.

SOLUTION 4.3

- (i) The given tuple would be deleted from *Enrolment* – so the corresponding row would be deleted from the table in Figure 3.2. The relation *Student* as depicted in Figure 3.3 would remain unchanged as no tuple in *Student* references any in *Enrolment* – the referencing is the other way round.

- (ii) Here, all the tuples in *Enrolment* referencing the given tuple are deleted. That is, the tuples <s09, c4, 'Dec 16, 2004'>, <s09, c2, 'Dec 18, 2004'> and <s09, c7, 'Dec 15, 2004'> are deleted.

SOLUTION 4.4

This is not an acceptable tuple constraint if the two attributes are from two different relations (*Student* and *Enrolment*). We are going to have to use a **join** to derive a new relation subsuming *Student* and *Enrolment* (see Subsection 4.3).

SOLUTION 4.5

relation *Nurse*

StaffNo: *StaffNos*

NurseName: *Names*

WardNo: *WardNos*

primary key *StaffNo*

{*relationship StaffedBy*}

foreign key *WardNo* **references** *Ward*

relation *Ward*

WardNo: *WardNos*

WardName: *WardNames*

NumberOfBeds: *BedNumbers*

primary key *WardNo*

The mandatory participation of **Nurse** in **StaffedBy** is represented by the foreign key *WardNo* – every tuple in *Nurse* has a value for *WardNo*, so has a matching tuple in *Ward*.

SOLUTION 4.6

- (i) A difference B is empty.
- (ii) Conversely, if A difference B is empty, then every element of A is an element of B (see Subsection 3.4 and Exercise 3.21).

SOLUTION 4.7

relation *ExaminedBy*

CourseCode: *CourseCodes*

StaffNo: *StaffNos*

primary key (*CourseCode*, *StaffNo*)

foreign key *CourseCode* **references** *Course*

foreign key *StaffNo* **references** *Examiner*

relation *Course*

CourseCode: *CourseCodes*

Title: *TitlesOfCourses*

Credit: *Credits*

primary key *CourseCode*

constraint ((**project** *Course* **over** *CourseCode*) **difference** (**project** *ExaminedBy* **over** *CourseCode*)) **is empty**

relation *Examiner*

StaffNo: *StaffNos*

Name: *Names*

primary key *StaffNo*

constraint ((**project** *Examiner* **over** *StaffNo*) **difference** (**project** *ExaminedBy* **over** *StaffNo*)) **is empty**

SOLUTION 4.8

relation *ConsistsOf*

StaffNo: *StaffNos*

TeamCode: *TeamCodes*

primary key *StaffNo*

foreign key *StaffNo* **references** *Doctor*

foreign key *TeamCode* **references** *Team*

relation *Doctor*

StaffNo: *StaffNos*

DoctorName: *Names*

Position: *Positions*

primary key *StaffNo*

relation *Team*

TeamCode: *TeamCodes*

TelephoneNumber: *TelephoneNumbers*

primary key *TeamCode*

{mandatory participation of *Team* in *ConsistsOf* relationship}

constraint ((**project** *Team* **over** *TeamCode*) **difference** (**project** *ConsistsOf* **over** *TeamCode*)) **is empty**



SOLUTION 4.9

constraint (**select** (*Patient1* **join** (*Doctor* **rename** (*StaffNo* **as** *ConsultantNo*)) **where** *Position* <> 'Consultant') **is empty**

SOLUTION 4.10

constraint ((**project** *Nurse* **over** *StaffNo*) **intersection** (**project** *Doctor* **over** *StaffNo*)) **is empty**

This may well not be the only solution.

SOLUTION 5.1

- (i) You can't record this information because it has no associated values for *StudentId* or *StudentName*.
- (ii) The problem this poses is that tutor T1 occurs many times (potentially) in the relational table and each occurrence must be changed. It's possible that some occurrences might be missed and thus some recorded data become incorrect.
- (iii) The problem here is that if Belinda withdraws from the university, then the tuple with *StudentId* S2 and *CourseCode* C1 must be deleted – and this tuple is the only one containing the name of the tutor T3. If *StudentTutorCourse* is the only relation in which information about tutors is recorded, then this information about T3 will be lost.

SOLUTION 5.2

In Figure 5.1 we're told more than once that the student with identifier S1 is called Ashok, and that tutor T1 is called Ann.

SOLUTION 5.3

Each student on each course has exactly one name.

Each student on each course has exactly one identified tutor.

Each student on each course has exactly one named tutor.

You could, of course, be more precise here and say (for example) that ‘Each student, identified by *StudentId*, on each course, identified by *CourseCode*, has exactly one name, identified by *StudentName*’ – but this degree of precision adds nothing to our understanding of the real-world data.

SOLUTION 5.4

Each student has exactly one name.

Each tutor has exactly one name.

SOLUTION 5.5

FD1: *StudentId*, *CourseCode* $\mapsto$ *StudentName*

FD2: *StudentId*, *CourseCode* $\mapsto$ *TutorId*

FD3: *StudentId*, *CourseCode* $\mapsto$ *TutorName*

FD4: *StudentId* $\mapsto$ *StudentName*

FD5: *TutorId* $\mapsto$ *TutorName*

The determinants are *StudentId*, *CourseCode* for FD1, FD2 and FD3, *StudentId* for FD4 and *TutorId* for FD5.

SOLUTION 5.6

- (i) This is true – every value of a candidate key determines a unique value for each attribute of *R*.
- (ii) This is false, because *C* may not have the minimality condition necessary for a candidate key. For example, every attribute in the relation *StudentTutorCourse* is functionally dependent on (*StudentId*, *StudentName*, *CourseCode*).

SOLUTION 5.7

FD1: *PatientId*, *ApptDate* $\mapsto$ *ApptTime*

FD2: *PatientId*, *ApptDate* $\mapsto$ *ConsId*

FD3: *PatientId* $\mapsto$ *PatientName*

FD4: *ConsId* $\mapsto$ *ConsName*

FD5: *HospNo* $\mapsto$ *HospName*

FD6: *ConsId*, *ApptDate* $\mapsto$ *HospNo*

FD7: *ConsId*, *ApptDate*, *ApptTime* $\mapsto$ *PatientId*

SOLUTION 5.8

FD9: *ConsId*, *ApptDate* $\mapsto$ *HospName* by transitivity on FD6 and FD5.

FD10: *ConsId*, *ApptDate*, *ApptTime* $\mapsto$ *PatientName* by transitivity on FD7 and FD3.

SOLUTION 5.9

- (i) FD2 *PatientId*, *ApptDate* $\mapsto$ *ConsId* augments to *PatientId*, *ApptDate*, *ApptDate* $\mapsto$ *ConsId*, *ApptDate* which simplifies to *PatientId*, *ApptDate* $\mapsto$ *ConsId*, *ApptDate*.

Using this and FD6, we get

FD11: *PatientId*, *ApptDate* $\mapsto$ *HospNo*
by transitivity.

- (ii) FD12: *PatientId*, *ApptDate* $\mapsto$ *HospName* by transitivity on FD11 and FD5.

Recall that *PatientId*, *ApptDate* is shorthand for the set {*PatientId*, *ApptDate*}, and since sets can have no duplicate elements, {*PatientId*, *ApptDate*, *ApptDate*} = {*PatientId*, *ApptDate*}.

SOLUTION 5.10

First, we want to find a set of attributes C of *Appointment* such that $C \mapsto PatientId$, $C \mapsto PatientName$, $C \mapsto ApptDate$..., $C \mapsto HospName$.

FDs 1, 2, 3 (and Property 2 on extending determinants), 8, 11 and 12 demonstrate that a unique value of *PatientId*, *ApptDate* determines unique values of all the other attributes.

Thus, by Property 1 on combining FDs, a unique value of *PatientId*, *ApptDate* determines a unique row.

Now to consider minimality: *PatientId*, *ApptDate* has the minimal property required of candidate keys, as a value of *PatientId* determines several rows (see, for example, rows 1, 2 and 3 of Figure 5.2), as does a value of *ApptDate* (see, for example, rows 1, 8 and 10), so that neither *PatientId* nor *ApptDate* can be a candidate key.

Similarly, FDs 7, 10, 6, 9 and 4 (with some judicious application of Property 2) demonstrate that a unique value of *ConsId*, *ApptDate*, *ApptTime* determines unique values of the other attributes. Also, *ConsId*, *ApptDate* does not determine unique rows (see rows 1 and 8) and neither does *ApptDate*, *ApptTime* (see rows 1 and 10) nor *ConsId*, *ApptTime* (see rows 1 and 4), and hence neither do their constituent attributes, by Property 2.

So the two candidate keys identified are *ConsId*, *ApptDate*, *ApptTime* and *PatientId*, *ApptDate*.

SOLUTION 5.11

- (i) FD1: *ClassName* $\mapsto$ *ClassRoom*
 FD2: *ClassName* $\mapsto$ *ClassTeacherCode*
 FD3: *ClassTeacherCode* $\mapsto$ *ClassTeacherName*
 FD4: *CourseCode* $\mapsto$ *CourseName*
 FD5: *ClassName*, *CourseCode* $\mapsto$ *CourseTeacherCode*
 FD6: *CourseTeacherCode* $\mapsto$ *CourseTeacherName*
 FD7: *ClassTeacherCode* $\mapsto$ *ClassName*
 FD8: *ClassRoom* $\mapsto$ *ClassName*
- (ii) FD9: *ClassName* $\mapsto$ *ClassTeacherName* by transitivity on FD2 and FD3
 FD10: *ClassName*, *CourseCode* $\mapsto$ *CourseTeacherName* by transitivity on FD5 and FD6
 FD11: *ClassRoom* $\mapsto$ *ClassTeacherCode* by transitivity on FD8 and FD2
 FD12: *ClassRoom* $\mapsto$ *ClassTeacherName* by transitivity on FD11 and FD3
 FD13: *ClassTeacherCode* $\mapsto$ *ClassRoom* by transitivity on FD7 and FD1
- (iii) (a) By augmentation, FD8 *ClassRoom* $\mapsto$ *ClassName* becomes
ClassRoom, *CourseCode* $\mapsto$ *ClassName*, *CourseCode*
 and then by transitivity with FD5, we derive
ClassRoom, *CourseCode* $\mapsto$ *CourseTeacherCode*
- (b) We may augment FD7 to
ClassTeacherCode, *CourseCode* $\mapsto$ *ClassName*, *CourseCode*
 and then by transitivity with FD5, derive
ClassTeacherCode, *CourseCode* $\mapsto$ *CourseTeacherCode*

SOLUTION 5.12

- (i) $PatientId \mapsto PatientName$
 (ii) $ConsId, ApptDate \mapsto HospNo$
 $ConsId, ApptDate \mapsto HospName$
 $ConsId \mapsto ConsName$

SOLUTION 5.13

The table representing the evaluation of *StudentTutorCourse2* **join** *Student2* is the same as the original relational table *StudentTutorCourse* seen in Figure 5.1 (remember that the order of the columns is immaterial).

SOLUTION 5.14

(i)

<i>StudentId</i>	<i>StudentName</i>	<i>CourseCode</i>
<i>S1</i>	<i>Ashok</i>	<i>C1</i>
<i>S1</i>	<i>Ashok</i>	<i>C2</i>
<i>S2</i>	<i>Belinda</i>	<i>C1</i>
<i>S3</i>	<i>Charles</i>	<i>C3</i>

(ii)

<i>CourseCode</i>	<i>TutorId</i>	<i>TutorName</i>
<i>C1</i>	<i>T1</i>	<i>Ann</i>
<i>C2</i>	<i>T2</i>	<i>Barry</i>
<i>C1</i>	<i>T3</i>	<i>Cayley</i>
<i>C3</i>	<i>T1</i>	<i>Ann</i>

(iii)

<i>StudentId</i>	<i>StudentName</i>	<i>CourseCode</i>	<i>TutorId</i>	<i>TutorName</i>
<i>S1</i>	<i>Ashok</i>	<i>C1</i>	<i>T1</i>	<i>Ann</i>
<i>S1</i>	<i>Ashok</i>	<i>C1</i>	<i>T3</i>	<i>Cayley</i>
<i>S1</i>	<i>Ashok</i>	<i>C2</i>	<i>T2</i>	<i>Barry</i>
<i>S2</i>	<i>Belinda</i>	<i>C1</i>	<i>T1</i>	<i>Ann</i>
<i>S2</i>	<i>Belinda</i>	<i>C1</i>	<i>T3</i>	<i>Cayley</i>
<i>S3</i>	<i>Charles</i>	<i>C3</i>	<i>T1</i>	<i>Ann</i>

SOLUTION 5.15

- (i) $Appointment2(\underline{PatientId}, \underline{ApptDate}, \underline{ApptTime}, ConsId, ConsName, HospNo, HospName)$
 $Patient2(\underline{PatientId}, PatientName)$

By Heath's theorem, we know that the join of these two relations yields the original relation.

We have used the name *Appointment2'* to differentiate this relation from the relation *Appointment2* in Solution 5.15(i).

- (ii) *Consultant2(ConsId, ConsName)*

ConsultantHosp2(ConsId, ApptDate, HospNo, HospName)

Appointment2'(ConsId, ApptDate, ApptTime, PatientId, PatientName)

By repeated applications of Heath's theorem, we know that the join of these three relations yields the original relation.

- (iii) *Consultant2* is in 2NF – if a primary key only has a single attribute, then every non-primary attribute must be fully functionally dependent on it.

ConsultantHosp2 is in 2NF – *HospNo* is not functionally dependent on either *ConsId* or *ApptDate*, and neither is *HospName*.

Appointment2' is in 2NF – *PatientId* is not functionally dependent on *ConsId*, *ApptDate* (see, for example, rows 1 and 8 of Figure 5.2) or *ConsId*, *ApptTime* (see rows 1 and 4) or *ApptDate*, *ApptTime* (see rows 1 and 10) or on any single attribute *ConsId*, *ApptDate* or *ApptTime*, and neither is *PatientName*.

SOLUTION 5.16

Since *B* is an alternate key, then a particular value of *B* determines a unique value for each of the attributes – all the attributes are functionally dependent on *B* – and in particular, $B \mapsto P$.

SOLUTION 5.17

We want to rule out trivial functional dependencies, where the right-hand side of the FD is a subset of the left-hand side (the determinant).

SOLUTION 5.18

StudentTutorCourse3(StudentId, CourseCode, TutorId)

SOLUTION 5.19

- (i) *Appointment2'* is not in 3NF because *PatientName* is transitively dependent on the primary key via the FD $PatientId \mapsto PatientName$ as:

$ConsId, ApptDate, ApptTime \mapsto PatientId$ and $PatientId \mapsto PatientName$ (TD(i)).

It is not true that $PatientId \mapsto ConsId, ApptDate, ApptTime$ (TD(ii)).

PatientName is not an attribute of either *PatientId* or *ConsId*, *ApptDate*, *ApptTime* (TD(iii)).

- (ii) We decompose as follows:

Patient3(PatientId, PatientName)

Appointment3'(ConsId, ApptDate, ApptTime, PatientId)

These are both in 3NF. They are both in 2NF, in the former case because the primary key has only one attribute, and in the latter, because the solution to Exercise 5.10 establishes that the FD $ConsId, ApptDate, ApptTime \mapsto PatientId$ has an irreducible determinant. *Patient3* is in 3NF because it only has one FD. *Appointment3'* is in 3NF because any non-trivial FD with *PatientId* on the right-hand side must have a determinant which is a subset of the primary key *ConsId*, *ApptDate*, *ApptTime*, and we have already established that $ConsId, ApptDate, ApptTime \mapsto PatientId$ is the only such possibility.

SOLUTION 5.20

- (i) We are told more than once that NW11 is in London.
 (ii) No – a street name does not uniquely determine a city; a postcode does not uniquely determine a street; a city does not uniquely determine a postcode.
 (iii) *Postcode, Street*

SOLUTION 5.21

STC is in 3NF because it is in 2NF – neither *StudentId* nor *CourseCode* are determinants of any FD – and any transitive dependency would have to involve either $B \mapsto \textit{EnrolmentDate}$ or $B \mapsto \textit{TutorId}$, where *B* is not a candidate key, neither of which holds.

STC is not in BCNF because *TutorId* is not a candidate key.

SOLUTION 5.22

- (i) *Appointment3'*(*ConsId*, *ApptDate*, *ApptTime*, *PatientId*) is in BCNF, as the only non-trivial FDs applicable in this relation from the list preceding Exercise 5.10 are:

$\textit{ConsId}, \textit{ApptDate}, \textit{ApptTime} \mapsto \textit{PatientId}$

$\textit{PatientId}, \textit{ApptDate} \mapsto \textit{ApptTime}$

$\textit{PatientId}, \textit{ApptDate} \mapsto \textit{ConsId}$

In the first case, the determinant is the primary key; in the second and third, the determinant is the alternate key *PatientId*, *ApptDate*.

- (ii) *Patient3*(*PatientId*, *PatientName*) is in BCNF, as the only FD is $\textit{PatientId} \mapsto \textit{PatientName}$.

SOLUTION 5.23

AddressBCNF(*Street*, *Postcode*)

PostcodeBCNF(*Postcode*, *City*)

As usual, Heath's theorem assures us that these relations, when joined, will yield the original relation.

Index

- 1:1 relationship 27
- 1:n relationship 24
- 1NF *see* first normal form
- 2NF *see* second normal form
- 3NF *see* third normal form
- A
 - alias 46
 - alternate key 20–21, 36, 78
 - amendment anomaly 66
 - angled-bracket notation 9
 - atomic 10
 - attribute 8, 36
 - value 67
 - augmentation 70, 72
- B
 - BCNF 80
 - binary operator 43, 49
 - body of a relation 15, 18, 95
 - Boolean expression 40
 - Boyce–Codd normal form 80, 82
- C
 - candidate key 20, 22, 36, 73, 78, 80–81
 - cardinality of a relation 14
 - Cartesian product 54
 - cascade effect 58
 - closed operator 38
 - closure property 42, 52
 - Codd, E.F. 55
 - comparison operators 40
 - conceptual data model 5
 - constraint 5, 21–22, 36, 40, 56, 59, 81
 - candidate key 56
- D
 - data redundancy 74
 - declaring a relation 19, 24, 28
 - default effect 58
 - degree of a relation 13
 - deletion anomaly 66, 82
 - determinant 74
 - extending 70
 - difference operator 52, 60
 - divide operator 49–50, 54
 - domain 8, 11, 16, 36
 - domain of discourse 20
- E
 - Entity–Relationship model 5
- F
 - FD 68
 - first normal form 74
 - foreign key 24, 26–27, 31
 - constraint 57
 - posted 24
 - pre-posted 26
 - fully functionally dependent 74
 - functional dependency 68–69, 77, 82
 - determinant 68, 70, 74, 77, 81
 - trivial 73, 80, 106
- G
 - generating new relations 64, 74
- H
 - heading of a relation 15, 18, 44
- I
 - identifier of entity type 12
 - implementable database design 6
 - implementation of a relational representation 6
 - insertion anomaly 66, 82
 - intersection operator 52
 - intersection relation 32
 - irreducible determinant 75, 80
- J
 - join dependency 82
 - join operator 43
- K
 - key 12
- L
 - logical proposition 15
 - logical schema 6
 - lossy decomposition 77
- M
 - minimality criterion 31
 - m:n* relationship 30
- N
 - natural language predicate 15, 22
 - non-loss decomposition 77, 81–82
 - normal forms 7, 66, 82
- O
 - operand 38, 42
 - operator 7, 38
 - optimisation 48
 - optimiser 48
- P
 - participation condition 59
 - primary key 12, 20, 36, 66–67, 85
 - project operator 40, 49
- R
 - recursive relationship 34, 48
 - reducible determinant 75
 - redundancy 66, 68, 74–75, 77–78, 80–82
 - referenced relation 26, 57
 - referencing relation 26, 57
 - referential integrity 25, 32, 57
 - relation 6–8, 13, 15, 18, 31, 36
 - properties 10
 - variables 16
 - relational algebra 55
 - relational representation 6, 19
 - relational table 8, 10, 13, 15, 36, 38
 - relationship 5, 7, 24, 31

- relvar 16
- rename operator 46, 49, 52
- restricted effect 57
- S
- second normal form 74–75, 79
- select operator 39, 49
- selection condition 39–40
- set 9
- single-valued fact 66, 68, 74, 81
 - ambiguity 67
- strong entity type 27
- SVF 67
- T
- third normal form 77, 79
- times operator 54
- transitive dependency 78–79
- transitivity 70, 72, 77, 79
- tuple 9, 36, 44
- tuple constraint 58
- U
- unary operator 39, 46
- union operator 52
- union-compatible 52
- W
- weak entity type 27

